Project 2 – Quick guide

Acme Analytics Linux Platform · story → commands → screenshots → final understanding

Muhammad Bilal Javed · up to Phase 4 Step 4.3 (part 1)

For fast revision: each step keeps only **why** we did it, the **commands**, the **screenshots** and my **final understanding**. Full details, tables and issue analyses are in the day reports and the complete guide.

Day 1 – 13 Oct: Phase 0 and the network foundation

The same story as a document to download: Project-2-Day-1-story.odt. The detailed technical reports are part 1 and part 2.

The story in one paragraph

Acme Analytics is a small company whose server setup has grown messily: nobody controls what reaches the internet, nothing is documented, and nobody could rebuild a server the same way twice. My job is to build a clean, controlled platform. On Day 1 I first **recorded how the servers looked** before touching anything. Then I built the **network foundation**: every server got a fixed address, the two internal servers lost their own internet connection, and the gateway server gw01 became the **one and only door** to the internet. At the end of the day, app01 – a server with no internet card of its own – could reach the internet through gw01, the outside world saw only gw01's address, and the servers could find each other by name. Along the way I hit and solved ten real problems.

Chapter 0 – The plan and the machines

Where are we?

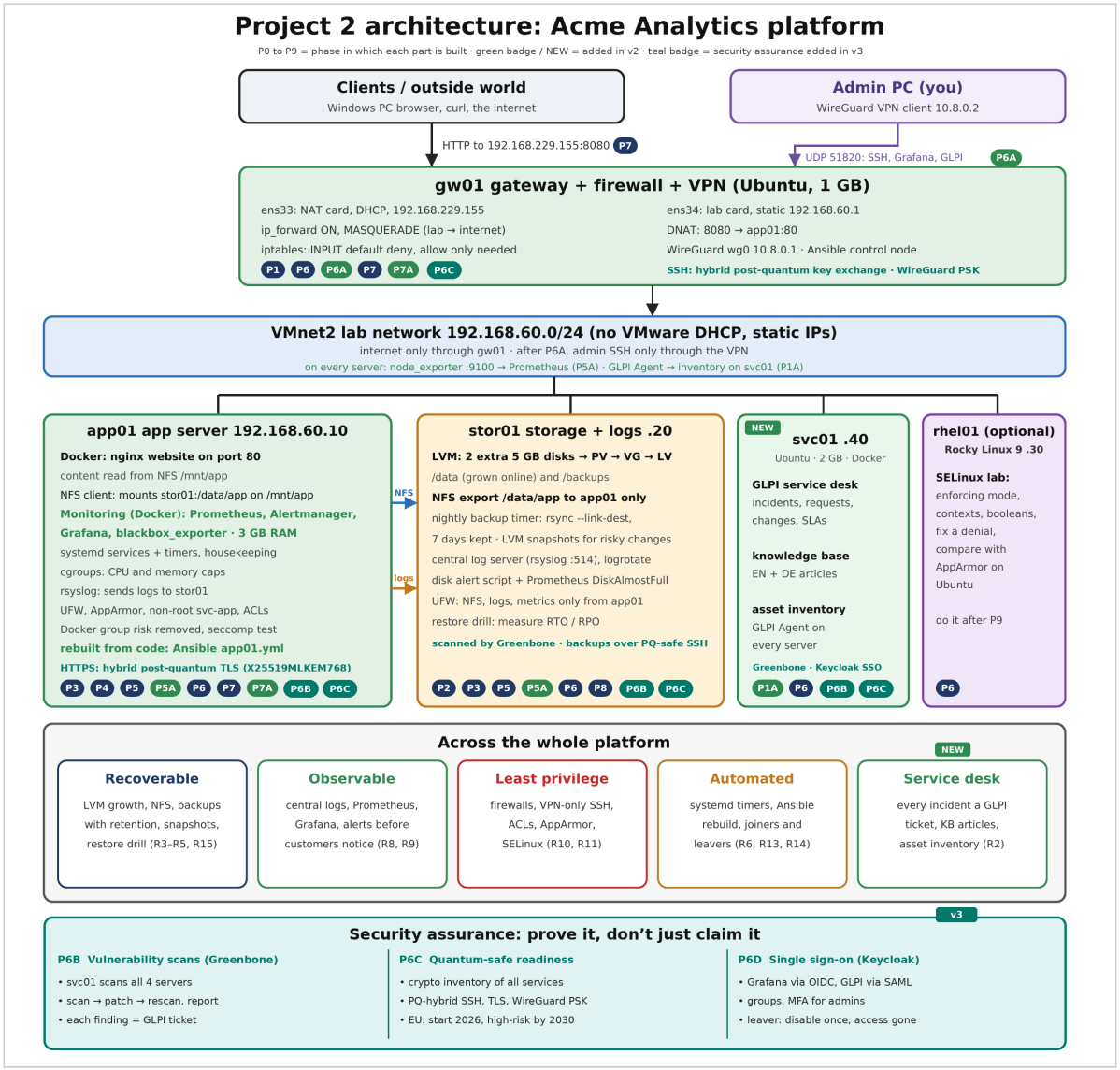


Figure 1 – Target architecture of the whole project. Day 1 builds the gateway, the lab network and the addresses of app01 and stor01.

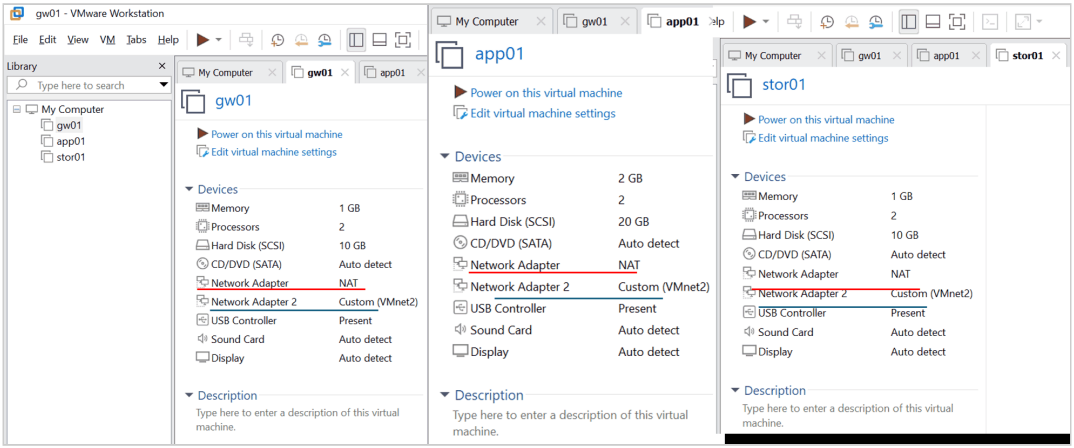


Figure 2 – The three VMs in VMware. Each has two network cards: "Network Adapter" (NAT = internet through Windows) and "Network Adapter 2" (VMnet2 = the private lab network).

Why do we go to the next step?

Before changing anything, a good engineer writes down how the system looks *now*. Otherwise, when something breaks later, there is nothing to compare against.

Chapter 1 – Know the system (Phase 0)

Step 0.1 – Engine, drivers and memory

What I did and why. The **kernel** is the engine of Linux. **Modules** are drivers – small add-ons that let the engine use hardware. I recorded the engine version, the loaded drivers, the memory, and one important switch: whether gw01 may pass traffic between networks.

```
uname -r                    # kernel version only
uname -a                    # everything: name, kernel, build date, CPU type
cat /proc/version           # the kernel's own description (compiler, build)
lsmod | head                # first 10 loaded drivers
free -h                     # memory and swap, in human-readable units
sysctl net.ipv4.ip_forward  # may this machine route traffic? (0 = no, 1 = yes)
```

```
bilal_001@gw01:~$ uname -r
5.15.0-198-generic
bilal_001@gw01:~$ uname -a
Linux gw01 5.15.0-198-generic #208-Ubuntu SMP Fri Sep 4 10:22:38 UTC 2026 x86_64 x86_64 x86_64 GNU/L
inux
bilal_001@gw01:~$ cat /proc/version
Linux version 5.15.0-198-generic (buildd@lcy02-amd64-040) (gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4
.0, GNU ld (GNU Binutils for Ubuntu) 2.38) #208-Ubuntu SMP Fri Sep 4 10:22:38 UTC 2026
bilal_001@gw01:~$ lsmod | head
Module                  Size  Used by
vsock_loopback         16384  0
vmw_vsock_virtio_transport_common    40960  1 vsock_loopback
vmw_vsock_vmci_transport    32768  1
vsock                  49152  5 vmw_vsock_virtio_transport_common,vsock_loopback,vmw_vsock_vmci_tran
sport
intel_rapl_msr         20480  0
binfmt_misc            24576  1
vmw_balloon            24576  0
intel_rapl_common      40960  1 intel_rapl_msr
rapl                   20480  0
bilal_001@gw01:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       299Mi       1.3Gi       1.0Mi       331Mi       1.4Gi
Swap:          1.3Gi          0B       1.3Gi
bilal_001@gw01:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
bilal_001@gw01:~$ _
```

Figure 3 – Baseline of gw01: kernel 5.15.0-198, VMware drivers loaded, 1.9 GiB memory, and ip_forward = 0.

Step 0.2 – How the machine starts

What I did and why. Starting a computer is a relay race: firmware → bootloader (GRUB) → kernel → systemd (the manager) → services → login. I measured how long it takes and saved the details to files as evidence.

```
systemd-analyze                          # total boot time
systemd-analyze blame >> boot-gw01.txt   # time per service, saved to a file
journalctl -b | head -15 >> journal-gw01.txt  # first log lines of this boot, saved
ls /boot                                 # the files used to start Linux
```

```
bilal_001@gw01:~$ systemd-analyze
Startup finished in 17.022s (kernel) + 21.095s (userspace) = 38.117s
graphical.target reached after 19.809s in userspace
bilal_001@gw01:~$ systemd-analyze blame >>boot-gw01.txt
bilal_001@gw01:~$ journalctl -b | head -15 >>journal-gw01.txt
bilal_001@gw01:~$ ls /boot
config-5.15.0-198-generic       initrd.img.old                  vmlinuz-5.15.0-198-generic
grub                            lost+found                      vmlinuz.old
initrd.img                      System.map-5.15.0-198-generic
initrd.img-5.15.0-198-generic   vmlinuz
bilal_001@gw01:~$ _
```

Figure 4 – Boot takes 38.1 s (17.0 s kernel + 21.1 s services). /boot holds the current kernel and the previous one as a fallback.

Step 0.3 – Server mode instead of desktop mode

What I did and why. Linux can start in different modes, called **targets**. A desktop (`graphical.target`) is wasted on a server nobody sits in front of. Server mode (`multi-user.target`) runs everything needed – network, services, logins – without the desktop: faster, lighter and with fewer things an attacker could abuse.

```
systemctl get-default                          # which mode now?
sudo systemctl set-default multi-user.target   # switch to server mode
systemctl get-default                          # check
```

```
bilal_001@gw01:~$ systemctl get default
Unknown command verb get.
bilal_001@gw01:~$ systemctl get-default
graphical.target
bilal_001@gw01:~$ sudo systemctl set-default multi-user.target
[sudo] password for bilal_001:
Created symlink /etc/systemd/system/default.target → /lib/systemd/system/multi-user.target.
bilal_001@gw01:~$ systemctl get-default
multi-user.target
```

Figure 5 – gw01 switched from graphical.target to multi-user.target. The first line shows my typo “get default” (the command is get-default).

Why do we go to the next step?

I now know how gw01 looks. The next question is: **how are the three machines connected?** That is the network – and without a working network nothing else in the project (storage, monitoring, VPN, automation) can work.

Chapter 2 – The starting point of the network

Where are we?

```
hostname    # which machine am I on?
ip -br a    # network cards: name, state, address (brief form)
lsblk       # disks and partitions
```

```
bilal_001@gw01:~$ hostname
gw01
bilal_001@gw01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.155/24 metric 100 fe80::20c:29ff:feac:fa08/64
ens34            DOWN
bilal_001@gw01:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0 63.9M  1 loop /snap/core20/2318
loop1                       7:1    0 63.8M  1 loop /snap/core20/2922
loop2                       7:2    0   87M  1 loop /snap/lxd/29351
loop3                       7:3    0 38.8M  1 loop /snap/snapd/21759
loop4                       7:4    0 44.7M  1 loop /snap/snapd/28254
sda                         8:0    0   10G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0  1.8G  0 part /boot
└─sda3                      8:3    0  8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0  8.2G  0 lvm  /
sr0                        11:0    1    2G  0 rom
bilal_001@gw01:~$ _
```

Figure 6 – gw01 at the start: ens33 (internet side) has 192.168.229.155; ens34 (lab side) is DOWN with no address.

```
bilal_002@app01:~$ hostname
app01
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.152/24 metric 100 fe80::20c:29ff:fee7:59c4/64
ens34            DOWN
bilal_002@app01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0  63.8M  1 loop /snap/core20/2922
loop2                       7:2    0    74M  1 loop /snap/core22/2955
loop3                       7:3    0    87M  1 loop /snap/lxd/29351
loop4                       7:4    0 115.2M  1 loop /snap/lxd/40911
loop5                       7:5    0  38.8M  1 loop /snap/snapd/21759
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    20G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0  18.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0    10G  0 lvm  /
sr0                        11:0    1  1024M  0 rom
bilal_002@app01:~$ _
```

Figure 7 – app01 at the start: it ALSO has its own internet address (192.168.229.152) on ens33; the lab card ens34 is DOWN.

```
bilal_003@stor01:~$ hostname
stor01
bilal_003@stor01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.153/24 metric 100 fe80::20c:29ff:fe08:26cc/64
ens34            DOWN
bilal_003@stor01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0  63.8M  1 loop /snap/core20/2922
loop2                       7:2    0    87M  1 loop /snap/lxd/29351
loop3                       7:3    0    74M  1 loop /snap/core22/2955
loop4                       7:4    0 115.2M  1 loop /snap/lxd/40911
loop5                       7:5    0  38.8M  1 loop /snap/snapd/21759
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    10G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0   8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   8.2G  0 lvm  /
sr0                        11:0    1  1024M  0 rom
bilal_003@stor01:~$ _
```

Figure 8 – stor01 at the start: its own internet address (192.168.229.153), lab card DOWN, and only one disk (sda) – the two extra data disks are missing.

Why do we go to the next step?

The servers need **fixed addresses** on the lab network, like permanent desk-phone numbers. Every later part of the project – firewall rules, file sharing, monitoring, automation – refers to these numbers. If they changed, everything would break.

Chapter 3 – Fixed addresses for every server (Step 1.1)

3.1 gw01 – the gatekeeper gets its inside address

```
ls /etc/netplan                                              # -> 50-cloud-init.yaml
sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml  # backup
echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
```

```
bilal_001@gw01:~$ ls /etc/netplan
50-cloud-init.yaml
bilal_001@gw01:~$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml
bilal_001@gw01:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:

  version: 2

  ethernets:

    ens33:

      dhcp4: true

    ens34:

      addresses: [192.168.60.1/24]
bilal_001@gw01:~$ echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-ne
twork-config.cfg
network: {config: disabled}
```

Figure 9 – gw01: the real file is 50-cloud-init.yaml, backed up, and cloud-init's network handling disabled.

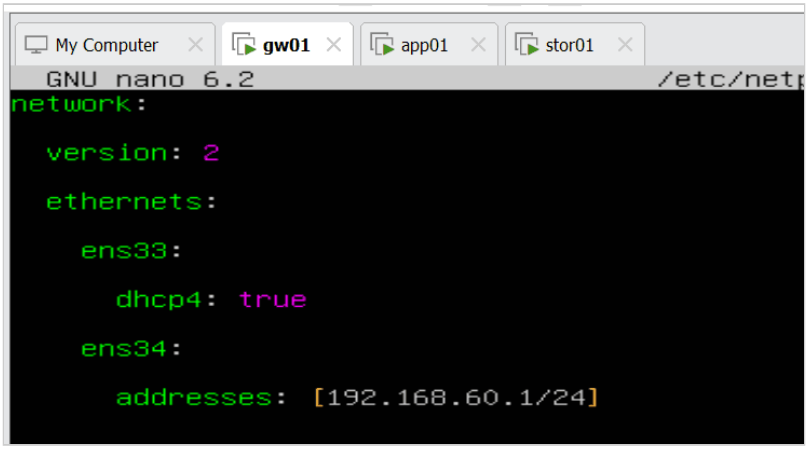


Figure 10 – gw01's netplan file, typed in the nano editor.

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml   # only the administrator may read/write it
sudo netplan try                                 # apply with automatic undo; Enter = keep
ip -br a                                         # check
```

```
bilal_001@gw01:~$ sudo chmod 600 /etc/netplan/50-cloud-init.yaml
bilal_001@gw01:~$ sudo netplan try
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
Do you want to keep these settings?


Press ENTER before the timeout to accept the new configuration


Changes will revert in 115 seconds
Configuration accepted.
bilal_001@gw01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.155/24 metric 100 fe80::20c:29ff:feac:fa08/64
ens34            UP             192.168.60.1/24 fe80::20c:29ff:feac:fa12/64
bilal_001@gw01:~$ _
```

Figure 11 – netplan try accepted; ens34 on gw01 now has 192.168.60.1/24. The Open vSwitch warning is harmless (that software is not installed).

3.2 app01 – the shop window, and an unplugged cable

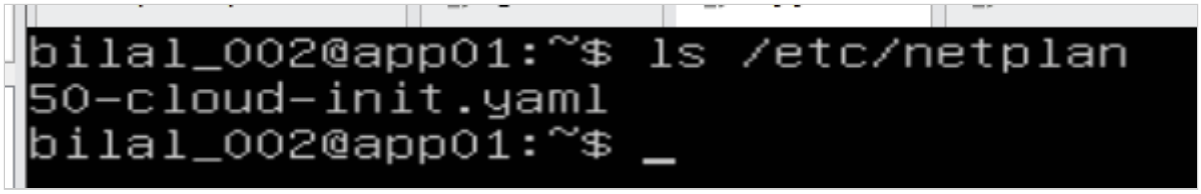


Figure 12 – app01 also uses 50-cloud-init.yaml.

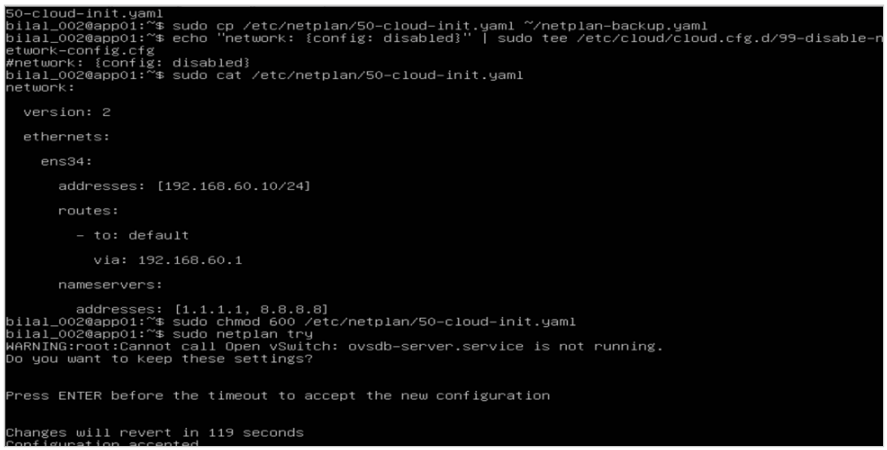


Figure 13 – app01: backup, cloud-init line, the new settings (192.168.60.10, default route via gw01, DNS servers) and netplan try. Note the "#" at the start of the cloud-init line – a typing slip that turns the line into a comment.

```
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.229.152/24 metric 100 fe80::20c:29ff:fee7:59c4/64
ens34            UP             192.168.60.10/24 fe80::20c:29ff:fee7:59ce/64
bilal_002@app01:~$ ping -c 3 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=10.8 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=0.755 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=1.33 ms

--- 192.168.60.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 0.755/4.287/10.779/4.596 ms
bilal_002@app01:~$
```

Figure 14 – app01 can now reach gw01 (3 of 3 replies) – the lab network works. BUT ens33 still has its own internet address 192.168.229.152.

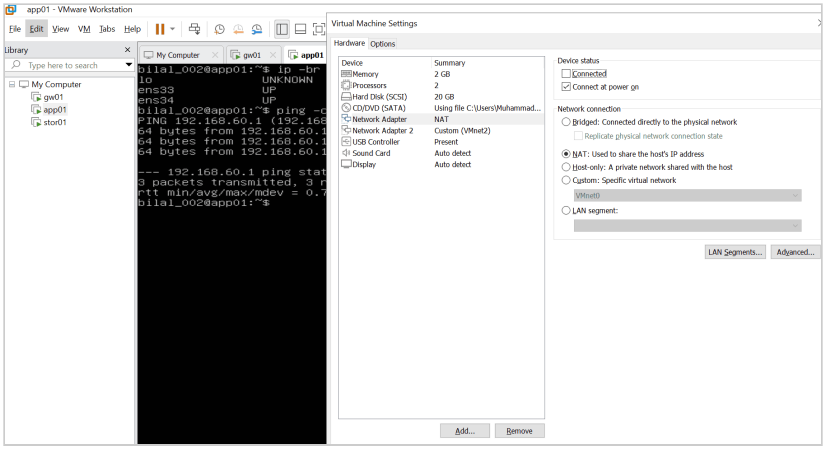


Figure 15 – Unplugging app01's internet adapter in VMware (Connected unticked).

```
bilal_002@app01:~$ sudo cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
[sudo] password for bilal_002:
network: {config: disabled}
bilal_002@app01:~$ sudo reboot_
```

Figure 16 – The cloud-init file is correct: "network: {config: disabled}", no "#".

```
bilal_002@app01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            DOWN
ens34            UP             192.168.60.10/24 fe80::20c:29ff:fee7:59ce/64
bilal_002@app01:~$ ip route
default via 192.168.60.1 dev ens34 proto static
192.168.60.0/24 dev ens34 proto kernel scope link src 192.168.60.10
bilal_002@app01:~$ sudo cat /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
network: {config: disabled}
```

Figure 17 – app01 after reboot: ens33 DOWN (unplugged), ens34 = 192.168.60.10, default route via gw01, cloud-init file still correct.

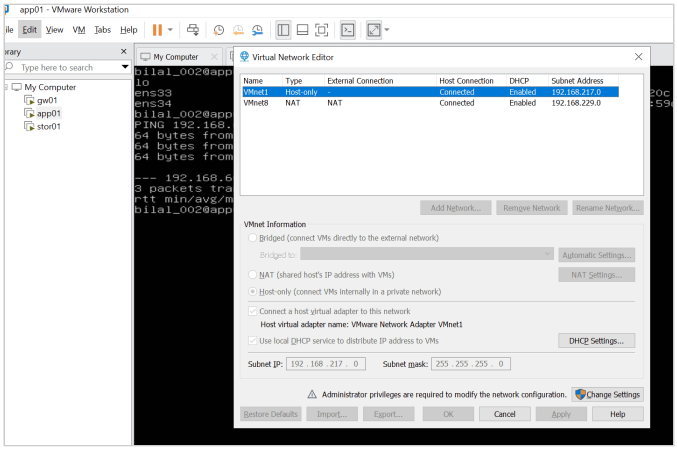


Figure 18 – VMware's Virtual Network Editor lists only VMnet1 and VMnet8 – VMnet2 is an isolated switch, as intended.

3.3 stor01 – the archive room, and the missing disks

```
bilal_003@stor01:~$ ls /etc/netplan
50-cloud-init.yaml
bilal_003@stor01:~$ sudo cp /etc/netplan/50-cloud-init.yaml ~/netplan-backup.yaml
bilal_003@stor01:~$ echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-
network-config.cfg
network: {config: disabled}
bilal_003@stor01:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:

  version: 2

  ethernets:

    ens34:

      addresses: [192.168.60.20/24]

      routes:

        - to: default

          via: 192.168.60.1

      nameservers:

        addresses: [1.1.1.1, 8.8.8.8]
bilal_003@stor01:~$ sudo chmod 600 /etc/netplan/50-cloud-init.yaml
bilal_003@stor01:~$
```

Figure 19 – stor01: file found, backed up, cloud-init disabled (correctly this time), settings with 192.168.60.20, permissions set.

```
bilal_003@stor01:~$ sudo netplan try
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
Do you want to keep these settings?


Press ENTER before the timeout to accept the new configuration


Changes will revert in 108 seconds
Configuration accepted.
bilal_003@stor01:~$ _
```

Figure 20 – stor01: netplan try accepted.

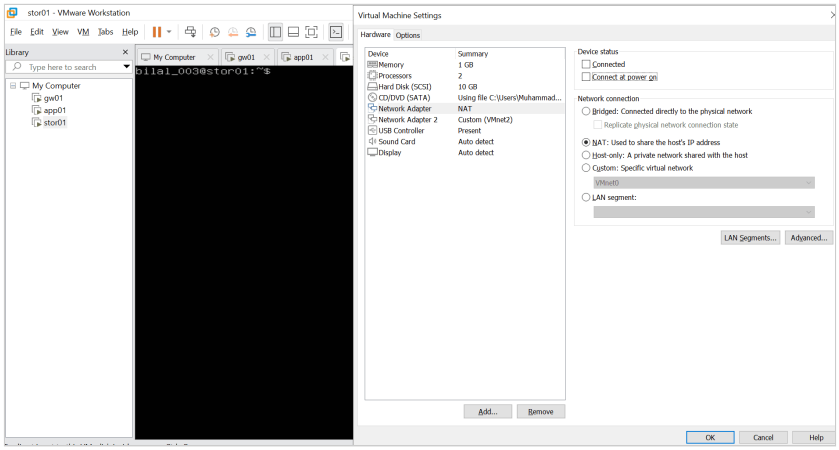


Figure 21 – stor01's internet adapter unplugged in VMware: both "Connected" and "Connect at power on" unticked.

```
bilal_003@stor01:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            DOWN
ens34            UP             192.168.60.20/24 fe80::20c:29ff:fe08:26d6/64
bilal_003@stor01:~$ ip route
default via 192.168.60.1 dev ens34 proto static
192.168.60.0/24 dev ens34 proto kernel scope link src 192.168.60.20
bilal_003@stor01:~$ ping -c 3 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=5.36 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=4.76 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=2.04 ms

--- 192.168.60.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.039/4.050/5.357/1.443 ms
bilal_003@stor01:~$ lsblk
NAME                      MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  63.9M  1 loop /snap/core20/2318
loop1                       7:1    0    74M  1 loop /snap/core22/2955
loop2                       7:2    0  63.8M  1 loop /snap/core20/2922
loop3                       7:3    0    87M  1 loop /snap/lxd/29351
loop4                       7:4    0  38.8M  1 loop /snap/snapd/21759
loop5                       7:5    0 115.2M  1 loop /snap/lxd/40911
loop6                       7:6    0  44.7M  1 loop /snap/snapd/28254
sda                         8:0    0    10G  0 disk
├─sda1                      8:1    0     1M  0 part
├─sda2                      8:2    0   1.8G  0 part /boot
└─sda3                      8:3    0   8.2G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   8.2G  0 lvm  /
sr0                        11:0    1  1024M  0 rom
bilal_003@stor01:~$
```

Figure 22 – stor01 after reboot: ens33 DOWN, 192.168.60.20, default via gw01, 3/3 pings to gw01 – but lsblk still shows only sda.

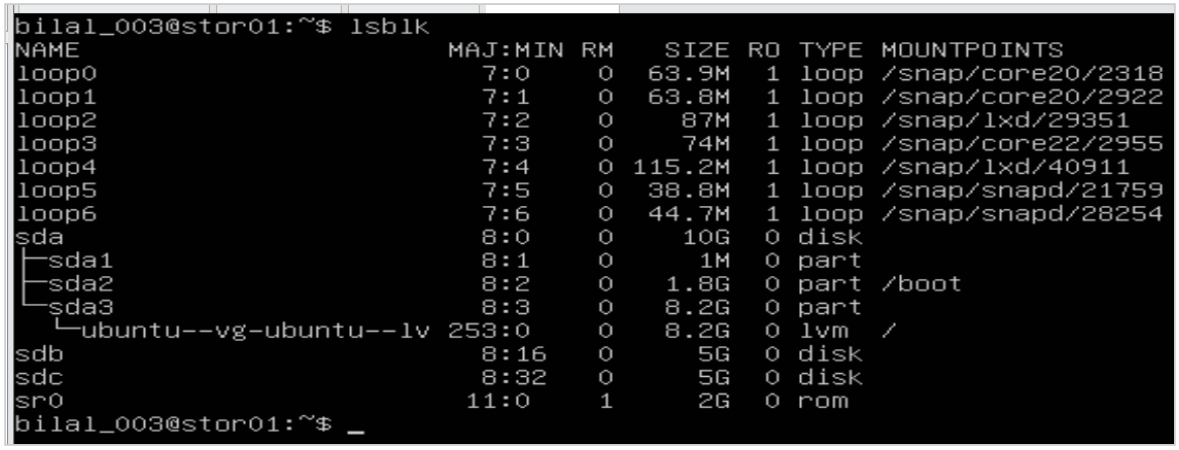


Figure 23 – After adding them: sdb and sdc, 5 GB each, empty and ready for Phase 2 (LVM).

Why do we go to the next step?

All three servers now have fixed addresses and can reach gw01. app01 and stor01 have **no internet line of their own** – so right now they have **no internet at all**. They know they should ask gw01, but gw01 does not pass anything on yet (`ip_forward = 0`, Figure 3). That is the next step.

Chapter 4 – gw01 becomes a router (Step 1.2)

What I did and why. By default, Linux refuses to pass on traffic that is not meant for itself – like a receptionist who only answers her own calls. Switching **IP forwarding** on turns gw01 into a switchboard between the lab and the internet.

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf  # permanent setting
sudo sysctl --system                                                   # load it now
sysctl net.ipv4.ip_forward                                             # check: must be 1
```

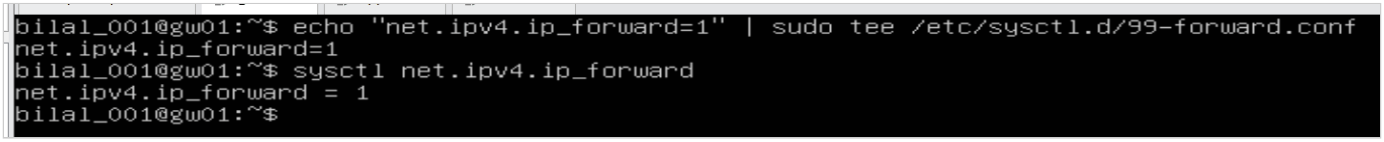


Figure 24 – ip_forward = 1. In the Phase 0 baseline (Figure 3) it was 0.

Why do we go to the next step?

gw01 now *passes traffic on*, but the internet still cannot answer app01. Addresses like 192.168.60.10 are **private**: they are not valid on the internet, so replies would have nowhere to go. gw01 must translate them – that is NAT.

Chapter 5 – Sharing the internet safely: NAT (Step 1.3)

What I did and why. NAT (Network Address Translation) works like a company mailroom. When app01 sends a letter out, gw01 crosses out the sender "Room 10" and writes the **company's street address** – its own. When the reply arrives, gw01 remembers who asked and delivers it to app01. The firewall rules decide **who may pass**: the lab may go out, and replies come back in. (Strictly blocking everything else needs a DROP default policy, which follows in Phase 6; today the lab is protected because its private addresses cannot be reached from the internet.)

```
# NAT: lab traffic leaving through ens33 gets gw01's address as sender
sudo iptables -t nat -A POSTROUTING -o ens33 -s 192.168.60.0/24 -j MASQUERADE
# allow: lab -> internet
sudo iptables -A FORWARD -i ens34 -o ens33 -j ACCEPT
# allow back in ONLY replies to conversations the lab started
sudo iptables -A FORWARD -i ens33 -o ens34 -m state --state ESTABLISHED,RELATED -j ACCEPT
# keep the rules after a reboot
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
```

The proof – from a server that has no internet card

```
ping -c 3 8.8.8.8      # can app01 reach a server on the internet?
curl -s ifconfig.me    # ask a website: which address do you see me coming from?
```

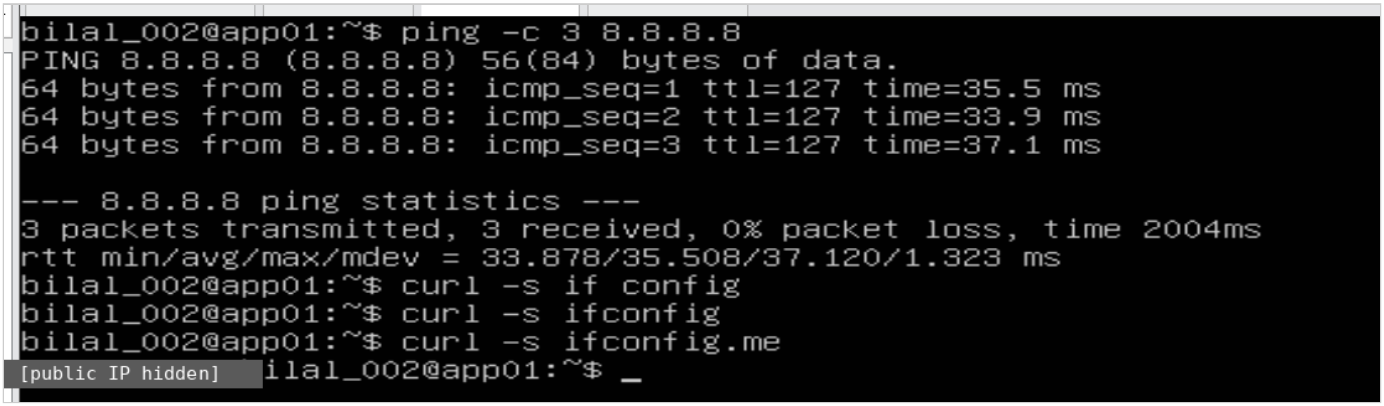


Figure 25 – app01 reaches the internet (3/3 replies from 8.8.8.8), and the website sees the home public address (hidden here), not 192.168.60.10. The two empty curl lines before were typos.

```
sudo iptables -t nat -L -n -v --line-numbers   # list NAT rules with packet counters
```

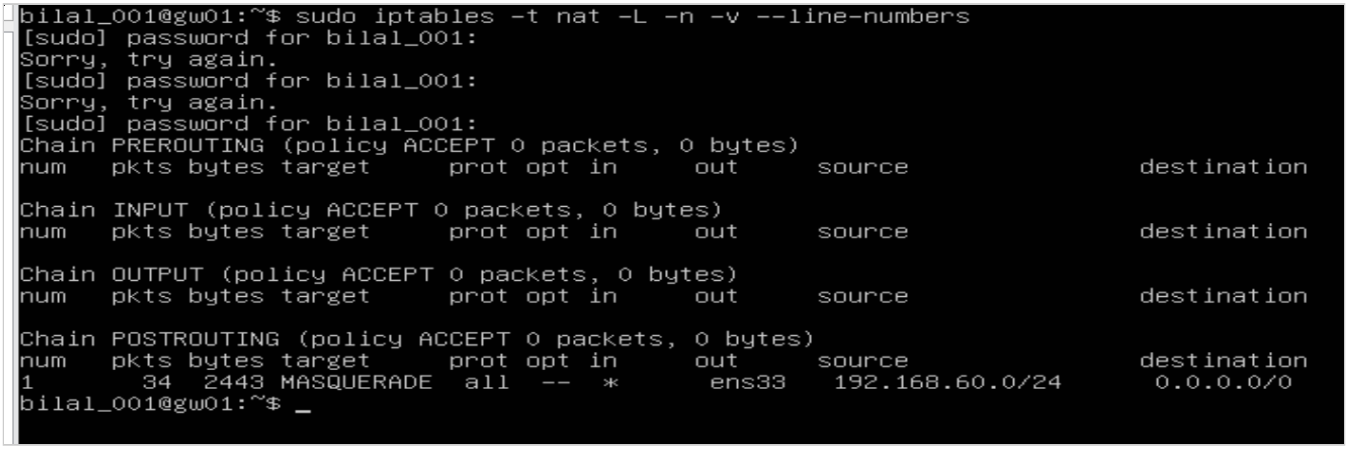


Figure 26 – On gw01 the MASQUERADE rule has translated 34 packets (2443 bytes) from 192.168.60.0/24 out through ens33. ("Sorry, try again" = mistyped sudo password.)

Why do we go to the next step?

The servers can now talk to each other and to the internet – but only by **number**. People and tools (SSH, file sharing, automation) use **names** like `stor01`. The next step teaches the servers to find each other by name.

Chapter 6 – Names instead of numbers (Step 1.4)

What I did and why. Computers connect only to numbers. Before connecting to a name, Linux looks it up – like a phone looking up a contact before dialing. It checks two places, in this order:

```
getent hosts stor01                                 # unknown -> prints nothing
echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts  # add stor01 to app01's contact list
getent hosts stor01                                 # now: 192.168.60.20
ping -c 1 stor01                                    # reach the server BY NAME
```

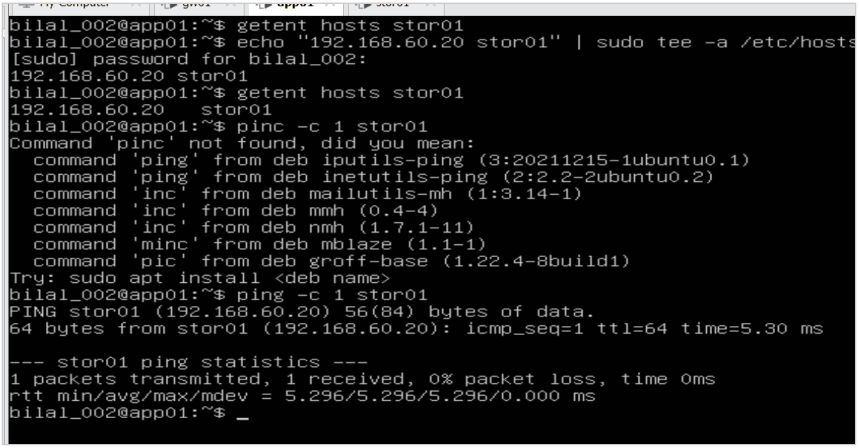


Figure 27 – On app01: the first lookup is empty; after adding the entry the name resolves and "ping stor01" works. ("pinc" was a typo.)

```
grep '^hosts' /etc/nsswitch.conf   # the lookup ORDER
dig +short google.com              # ask DNS directly
```

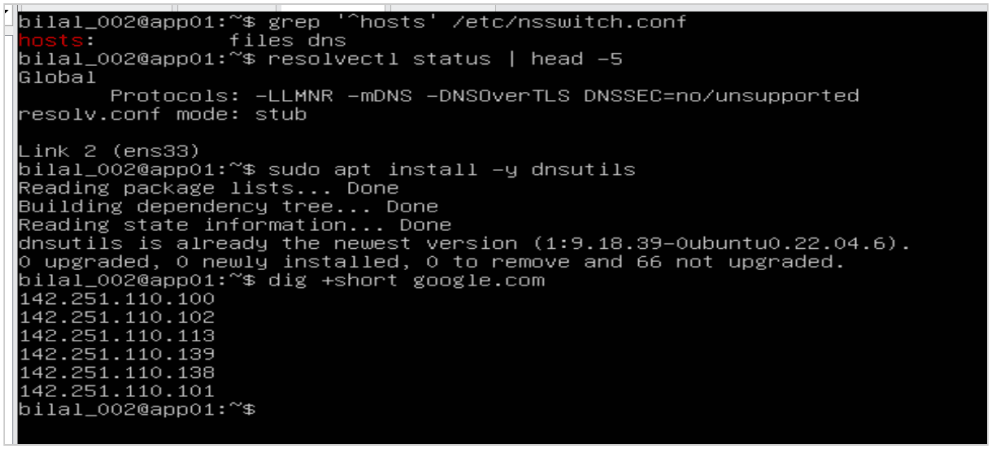


Figure 28 – "hosts: files dns" – the contact list is checked before DNS. dig returns Google's addresses, so DNS works through gw01.

Every server gets the same contact list

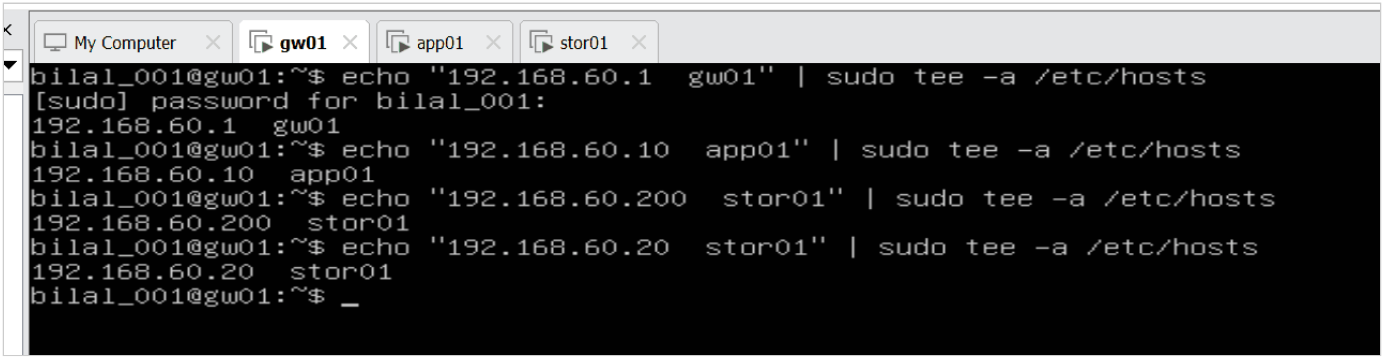


Figure 29 – The typo on gw01: a wrong 192.168.60.200 line for stor01 before the correct .20 line. The first match would have won.

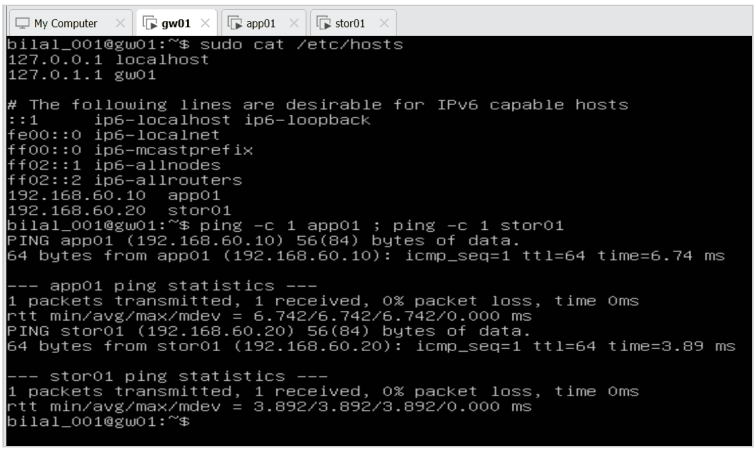


Figure 30 – gw01: correct contact list; ping app01 and ping stor01 by name, 1/1 received each.

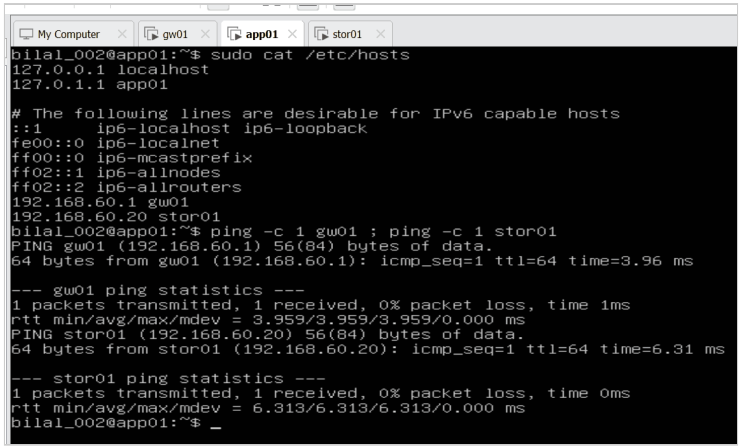


Figure 31 – app01: gw01 and stor01 by name, both reply.

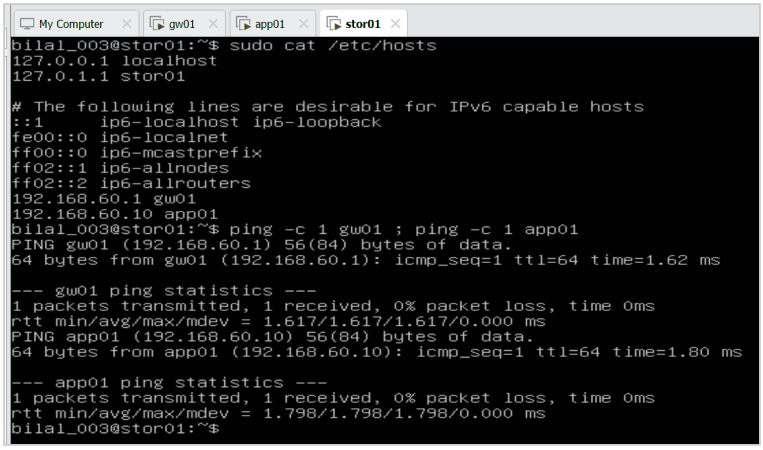


Figure 32 – stor01: gw01 and app01 by name, both reply.

Chapter 7 – Problems I met and how I solved them

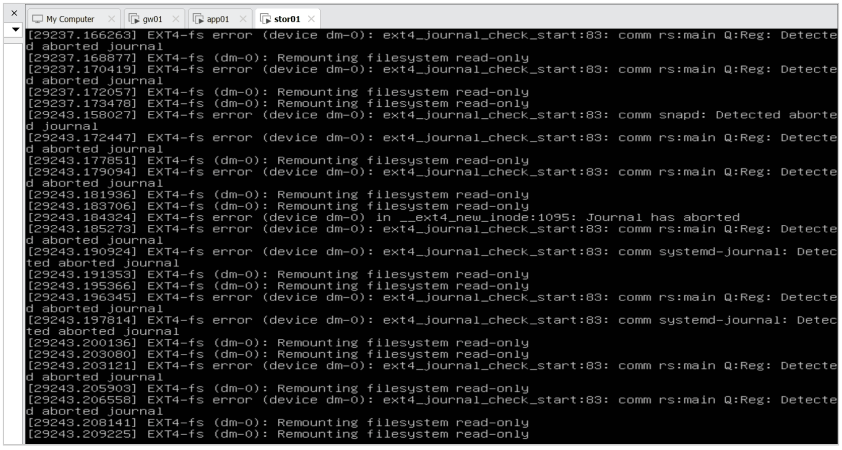


Figure 33 – Problem 12: stor01's system disk hit errors and Linux switched it to read-only to protect the data.

Chapter 8 – Where we stand at the end of Day 1

```
Morning:   3 separate machines, each with its own internet line, lab network unused
              |
Step 1.1:  fixed addresses  ->  gw01 .1   app01 .10   stor01 .20
           internet cables of app01 and stor01 unplugged
              |
Step 1.2:  gw01 becomes a router (ip_forward 0 -> 1)
              |
Step 1.3:  gw01 translates addresses (NAT) and lets only replies back in
              |
Step 1.4:  servers find each other by name
              |
Evening:   one controlled door to the internet - gw01
```

Chapter 9 – Key understanding (in my own words)

These are the conclusions I reached by questioning each step until it made sense. They are written so I can explain them in an interview.

1. The network file and cloud-init

- `/etc/netplan/50-cloud-init.yaml` is the main network settings file: IP address, gateway and DNS. It was written once by **cloud-init** when Ubuntu was installed. At every boot **netplan reads it** and configures the network cards.
- cloud-init is the **writer**, netplan is the **reader**. cloud-init could rewrite the file on a later boot and erase my settings.
- So I created `/etc/cloud/cloud.cfg.d/99-disable-network-config.cfg` with `network: {config: disabled}`. This only stops **cloud-init from managing the network** – the network itself and netplan keep working.
- Files in a `.d` folder are read in alphabetical order and **the last one wins**, so a `99-` file has the final word. The numbers only matter between files in the **same folder** – 50 (netplan) and 99 (cloud-init) are never compared.
- In Proxmox or the cloud, cloud-init is used the other way round: the admin enters IP, user and SSH key in the hypervisor, and cloud-init writes them into the server at first boot – ideal for cloned templates. In my VMware lab nothing feeds cloud-init, so I configure netplan by hand and keep cloud-init away from the network.

2. What the network file says

- `addresses: [192.168.60.10/24]` – **I am** 192.168.60.10; `/24` means 192.168.60.0–255 are my direct neighbours.
- `routes: - to: default via: 192.168.60.1` – "default" is not a machine, it means **everything I don't know**; `via` names who to hand it to: gw01.

- `nameservers: [1.1.1.1, 8.8.8.8]` – the internet's phone books. A name is first looked up in `/etc/hosts` (my own contact list – that is how `stor01` is found), and only then asked from DNS (that is how `google.com` is found).
- netplan only sets **addresses, gateway and DNS**. Being a router (ip_forward), NAT and firewall (iptables) and VPN (WireGuard) are set elsewhere.

3. The path to the internet

app01 is 192.168.60.10 and has no internet adapter. For anything outside the lab it goes via gw01 (192.168.60.1), its default gateway. gw01 translates app01's private address into its own (NAT) and passes the traffic on through VMware and the home router to the internet. stor01 works the same way with 192.168.60.20.

4. NAT, forwarding and conntrack

NAT recognises lab traffic by its IP range (`-s 192.168.60.0/24`) and changes the sender address as it leaves via `-o ens33`. Forwarding decides who may pass using the cards (`-i ens34 -o ens33`). Replies are let back in only if conntrack – gw01's memory of every conversation – confirms they are ESTABLISHED or RELATED to a conversation the lab started. conntrack also remembers which lab machine asked, so NAT can return each reply to the right one.

- **Two jobs, two tables:** the NAT table *changes the address* (`-j MASQUERADE`); the filter table's FORWARD chain *gives permission* (`-j ACCEPT`). A packet is checked by FORWARD first and rewritten in POSTROUTING just before it leaves – whatever order the commands were typed in.
- **`-s`/`-d` take IP addresses, `-i`/`-o` take card names.** `-s ens34` is always wrong. In POSTROUTING only the exit card is known, so `-i` is not allowed there – the lab must be identified by its IP range.
- **States:** NEW = someone starts a conversation; ESTABLISHED = a reply in a conversation the lab started; RELATED = a helper message (e.g. an error) belonging to it. `sudo conntrack -L` on gw01 shows the live list.
- **Honest limit today:** Ubuntu's default FORWARD policy is still ACCEPT, so the "only replies come back" rule is not yet enforced strictly. Right now the lab is protected because its private addresses sit behind two layers of NAT and cannot be reached from the internet. The strict version comes in Phase 6, when the default policy becomes DROP.
- `netfilter-persistent save` must run **last** – it saves only the rules that exist at that moment.

5. Names: /etc/hosts, DNS, getent and ping

getent ("get entries") is like searching the phone book. It looks up a name or IP first in `/etc/hosts`, then in **DNS** if one is configured. It works both ways: name → IP (forward lookup) and IP → name (reverse lookup, e.g. `getent hosts 192.168.60.20` → `stor01`). It only **reads** – it never contacts the machine.

ping is like calling. It uses the **same lookup** (`/etc/hosts`, then DNS) to find the number, then really contacts the machine with **ICMP** messages – "are you there?" (echo request) and "yes, I'm here" (echo reply). ICMP has **no ports**. So getent can succeed while ping fails – if the machine is off, the network is broken, or a **firewall blocks ICMP**.

In our lab there is no internal DNS server, so we put the names in `/etc/hosts`. The servers could already reach each other **by IP**; `/etc/hosts` lets them reach each other **by name**.

- **Phone picture:** the contacts app inside the phone = the system resolver (`/etc/hosts` → DNS); getent = opening the contacts and reading a number; ping = pressing *call* – it looks up the same contact, then dials; ICMP = the phone network that carries the call.
- **Troubleshooting rule:** getent fails → a **name** problem (missing/wrong entry or DNS). getent works but ping fails → a **network** problem (machine off, cabling, firewall).
- `/etc/hosts` is a file on **each** machine – fine for a small lab, but 100 servers means 100 files to edit. Companies use an **internal DNS server** for their own names (as the Windows Server does in Project 1) and **public DNS** (1.1.1.1, 8.8.8.8) for internet names.
- **The DNS helper and loopback:** on Ubuntu, programs do not ask 1.1.1.1 directly – they ask a small helper on the same machine, **systemd-resolved**, at **127.0.0.53**. Every **127.x.x.x** address is **loopback** – the machine talking to itself, like putting a note in your own pocket: the traffic never leaves the machine (`lo` in `ip -br a`). The helper answers from its **DNS cache** – short-term memory of name lookups for the whole machine – or asks the real DNS servers through gw01. The cache speeds things up, expires by itself (**TTL**), and can be cleared with `sudo resolvectl flush-caches`. `resolvectl status` shows which DNS servers are in use.
- **DNS troubleshooting in three commands:** `getent hosts <name>` (does the name resolve at all?) → `resolvectl status` (which DNS servers?) → `dig +short <name>` (does DNS itself answer? dig skips `/etc/hosts`).
- `hostname` shows the machine's own name; `getent hosts <name>` asks what address a name has; `tee -a /etc/hosts` appends a new entry.

Day 2 – 15 Oct: SSH keys, links, users, end of Phase 1

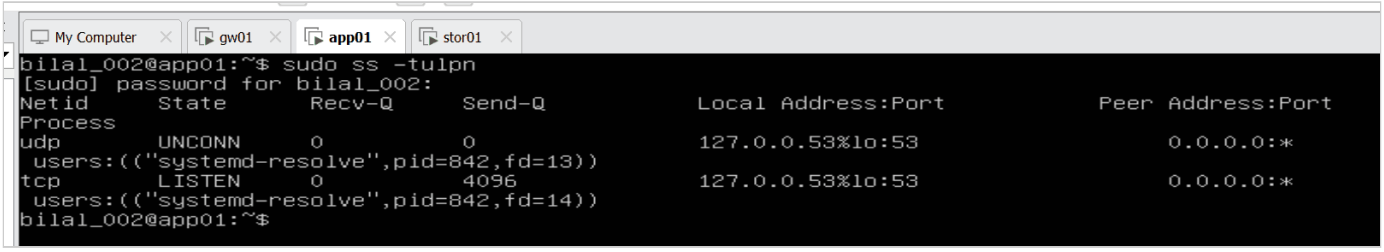
1. Objective

💡 **In plain words:** an IP address is the **address of a building** (which machine). A **port** is a **door number** inside that building (which program). Every service waits behind its own numbered door: SSH behind door 22, DNS behind door 53, a website behind door 80. Before opening new doors, I made an inventory of which doors app01 already has open – and to whom.

2. Procedure

```
sudo ss -tulpn   # ss = socket statistics
#  -t  TCP sockets        -u  UDP sockets
#  -l  only listening      -p  show the owning process
#  -n  numbers, not names  (memory aid: "tulpn" = tulip)
```

📸 **Evidence:**



ss -tulpn on app01

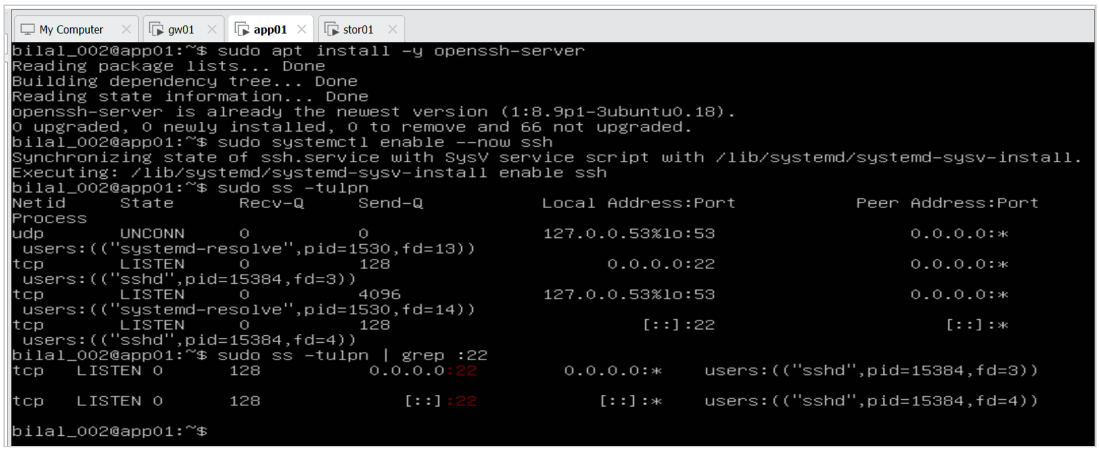
5. Final verdict

> **Before:** app01 exposed no network services at all – only the local DNS resolver on 127.0.0.53:53, reachable from inside the machine. The SSH package was installed but its service was not running, so it had to be enabled for remote administration.

6. Fix: enable SSH on app01

```
sudo apt install -y openssh-server   # make sure the SSH server package is installed
sudo systemctl enable --now ssh      # start it now (--now) AND at every boot (enable)
sudo ss -tulpn | grep :22            # verify: is anything listening on door 22?
```

📸 **Evidence:**



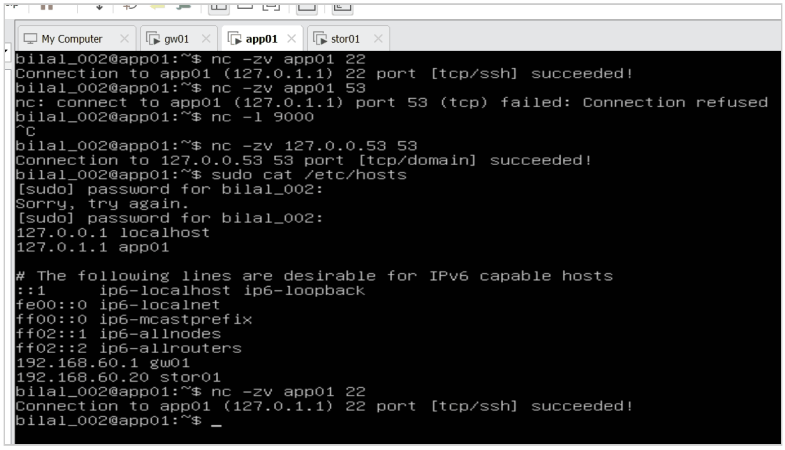
SSH enabled on app01, listening on port 22

7. Testing a port: `nc -zv` and the binding address

💡 **In plain words:** ping only proves a machine is alive. To prove a **service** answers, you knock on its **door** (port). `nc` (netcat) does exactly that – the admin's standard check when "ping works but the application doesn't".

```
nc -zv app01 22        # -z: only check the door, send no data · -v: print the result
nc -zv app01 53        # same for door 53
nc -zv 127.0.0.53 53   # door 53 at the DNS helper's own address
cat /etc/hosts         # where the name "app01" points to
```

📸 **Evidence:**



Port tests on app01 with nc -zv

8. Key understanding (in my own words)

Netid is the protocol, TCP or UDP. **TCP** starts every conversation with a 3-way handshake (SYN, SYN-ACK, ACK); after that the connection is **ESTAB**. **UDP** has no handshake – it just sends.

Every port leads directly to its own program. The port number decides which program receives the traffic: port **22** → **sshd** (SSH), port **53** → the **DNS helper**, port **80** → a web server. Traffic does **not** pass through the DNS helper first. A port with no program behind it is closed – nobody answers.

LISTEN = a TCP door is open and waiting. **UNCONN** = a UDP door is open. **Recv-Q** = data received but not read yet (0 = nothing waiting). **Send-Q** on a LISTEN socket = how many visitors may wait in the queue (4096 for the DNS helper's TCP socket). **Local address** = this machine's address and door, e.g. `127.0.0.53%lo:53` = the DNS helper on the loopback card, port 53, reachable only from inside app01. **Peer address** = who is visiting – `0.0.0.0:*` = nobody yet; in a real connection it shows the visitor's IP and its random port, e.g. `192.168.60.1:51234` = gw01.

A server can test itself by calling its own address – loopback (127.x.x.x). `localhost` (127.0.0.1) and `app01` (127.0.1.1) both mean "myself". SSH listens on **all** addresses (`0.0.0.0`), so `localhost:22` and `app01:22` both work. The DNS helper listens on **one fixed address only, 127.0.0.53**, so only `127.0.0.53:53` works – any other address gets "connection refused". **The binding address matters:** `0.0.0.0` = the door is open at every entrance of the building; `127.0.0.53` = the door exists at one entrance only.

A port is a door number; a socket is a program standing at that door on a specific machine (IP + port + protocol); a connection is two sockets talking to each other.

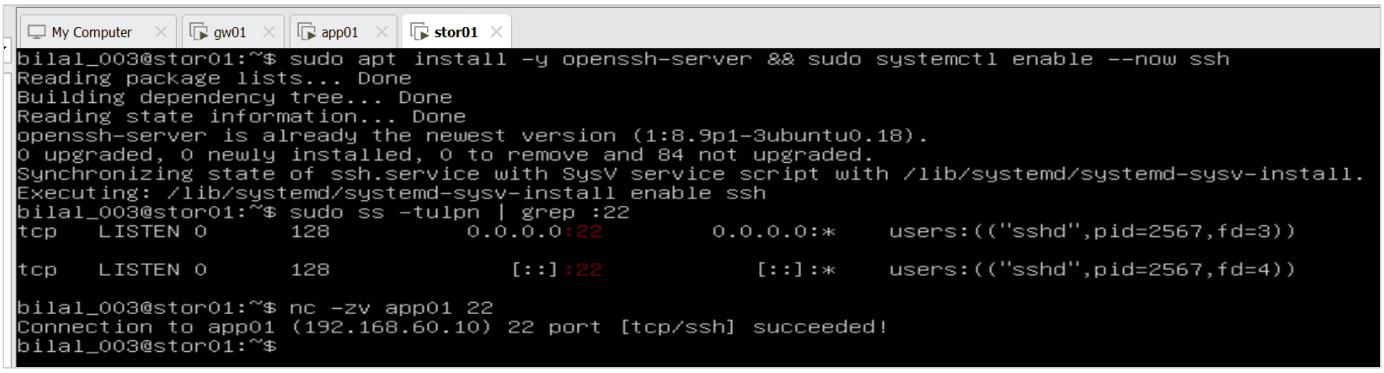
DNS has one job: translate a name into an IP address – the phone book, not the phone line. Lookup order on app01: **`/etc/hosts`** (own contact list) → **local DNS helper** `127.0.0.53` (answers from its cache) → **outside DNS server** `1.1.1.1` / `8.8.8.8` through gw01. Once the IP is known, the real traffic goes directly to that IP and port. In a company, an **internal DNS server** answers internal names; in our lab `/etc/hosts` does that job.

Before the fix, the only open port on app01 was 53, used by the local DNS helper, and only from inside app01 – it accepted **no connections from the network**. Next: enable SSH so port 22 opens.

9. Step 1.5 finished – SSH reachable across the lab network

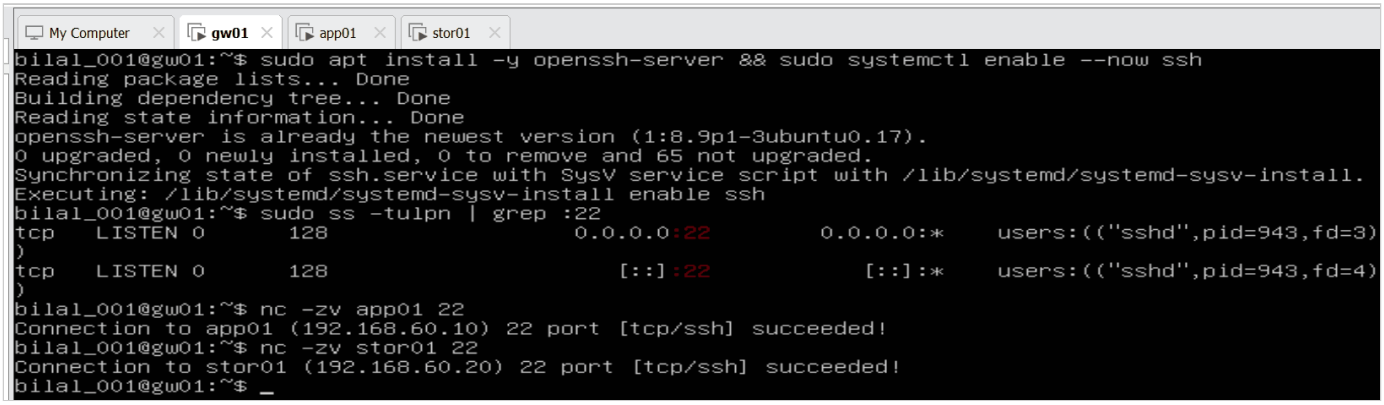
```
sudo apt install -y openssh-server && sudo systemctl enable --now ssh
sudo ss -tulpn | grep :22     # LISTEN 0.0.0.0:22 = door 22 open to the network
nc -zv app01 22               # knock on app01's SSH door over the lab network
```

📸 **Evidence – stor01:** SSH listening, and app01 reached at **192.168.60.10** (not loopback):



stor01: SSH enabled, nc to app01 succeeded

📸 **Evidence – gw01:** SSH listening, both servers reachable on port 22:



gw01: SSH listening, nc to app01 and stor01 succeeded

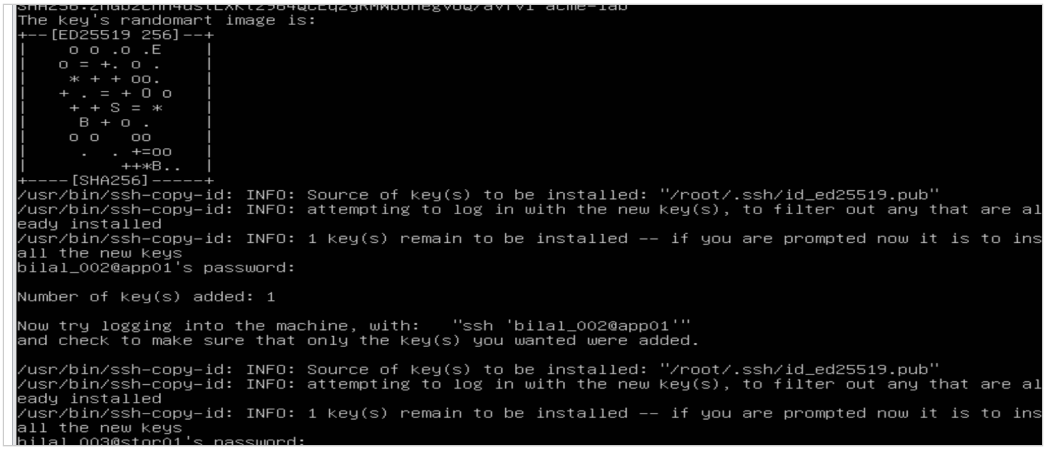
10. Step 1.6 – SSH keys: log in without a password

💡 **In plain words:** a password is something you **type**; an SSH key is a **pair of files** – a private key that never leaves gw01 (like a house key in your pocket) and a public key that is copied to each server (like a lock fitted to the door). If the key fits the lock, the door opens – no password. gw01 needs this because it becomes the **Ansible control node** in Phase 7A and must reach every server automatically.

```
ssh-keygen -t ed25519 -C "acme-lab"   # create the key pair (Enter 3×: default path, no passphrase for the lab)
ssh-copy-id bilal_002@app01           # copy the PUBLIC key to app01 (asks the password once)
ssh-copy-id bilal_003@stor01          # same for stor01
```

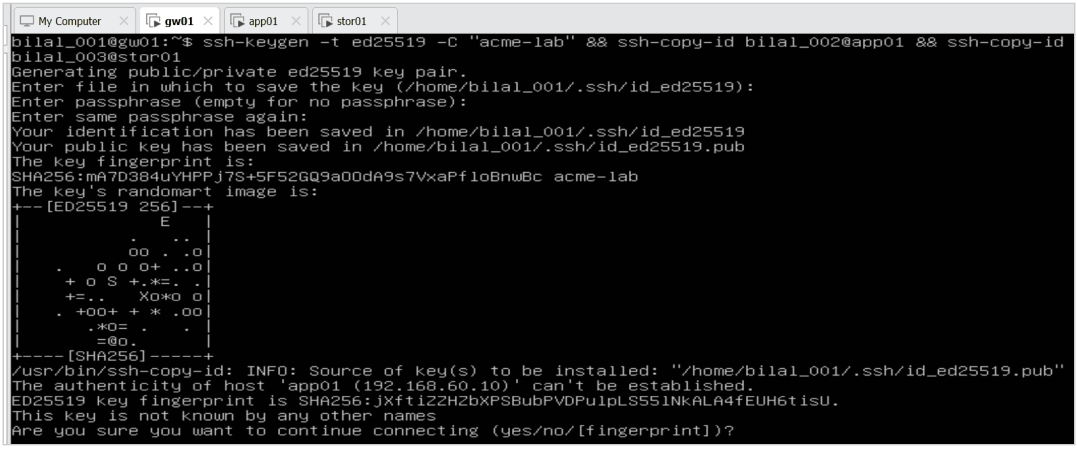
Issue: key first created for root

📸 The first attempt ran as root – the key landed in `/root/.ssh/`, so password-free login would only work with `sudo ssh …`:



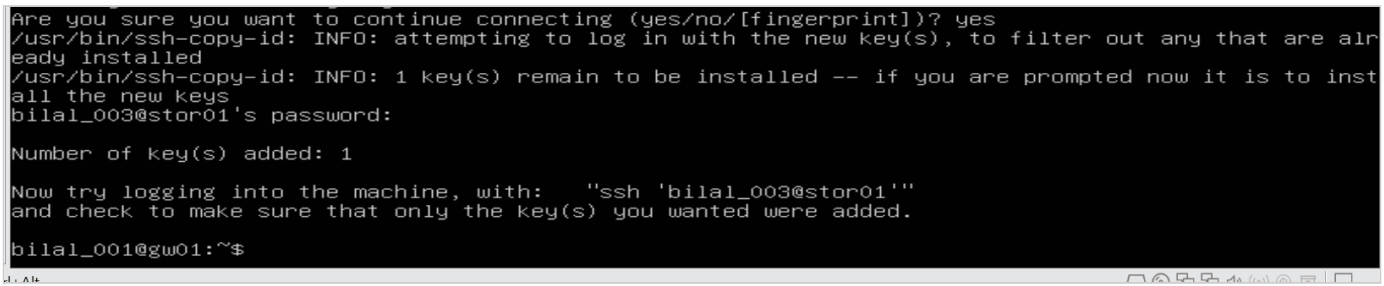
ssh-copy-id using /root/.ssh/id_ed25519.pub

📸 **Evidence – correct key for bilal_001:**



ssh-keygen as bilal_001, key in /home/bilal_001/.ssh

📸 **Evidence – key added to stor01 (“Number of key(s) added: 1”):**

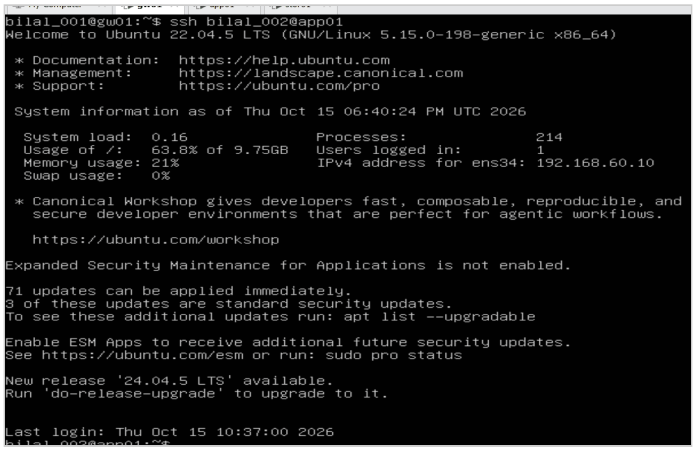


ssh-copy-id to stor01

The proof: login without a password

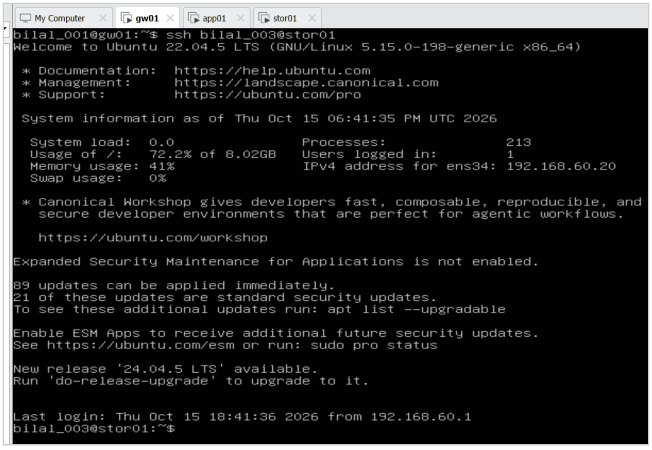
```
ssh bilal_002@app01     # log in to app01 — no password prompt
exit                    # close the session, back to gw01
ssh bilal_003@stor01    # log in to stor01 — no password prompt
exit                    # back to gw01
```

📸 **app01** – logged in with no password; the welcome screen confirms `IPv4 address for ens34: 192.168.60.10`:



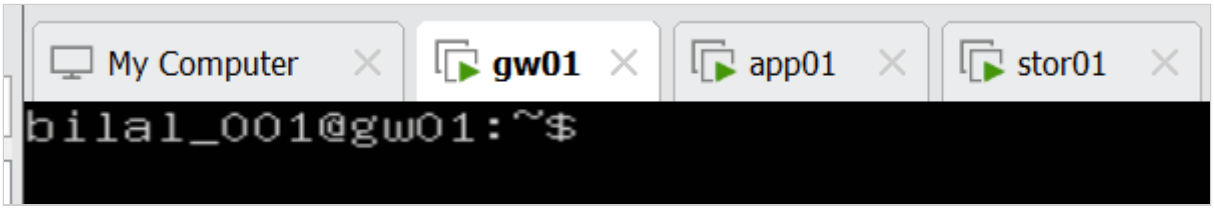
SSH login to app01 without password

📸 **stor01** – same; note `Last login … from 192.168.60.1` – Linux records who logged in from where (the start of an audit trail):



SSH login to stor01 without password

📸 **exit** closes the remote session – the prompt `bilal_001@gw01:~$` shows I am back on gw01:



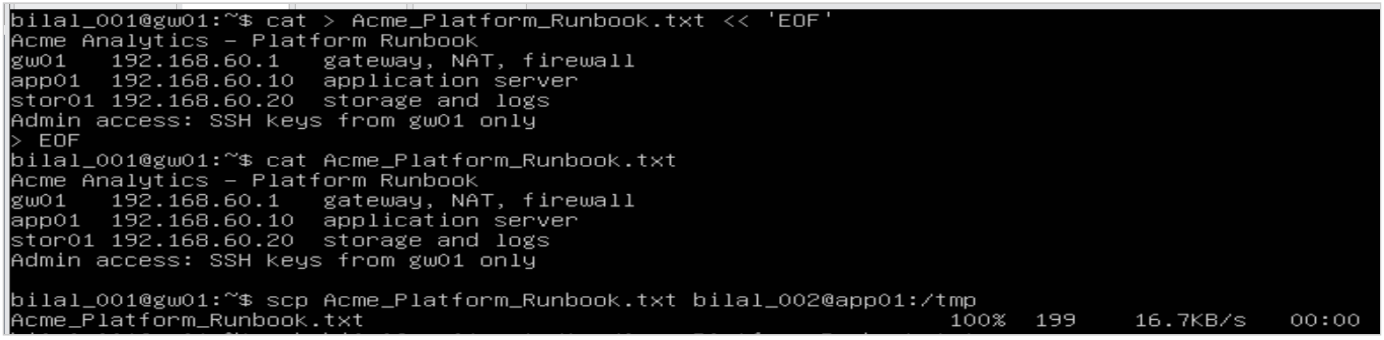
Back on gw01 after exit

Copying a file over SSH: `scp`

💡 **In plain words:** `scp` (secure copy) sends a file to another server through the same encrypted SSH door – and because the key is installed, no password is needed. Instead of a throw-away test file I created a short **platform runbook** – the kind of handover note admins keep.

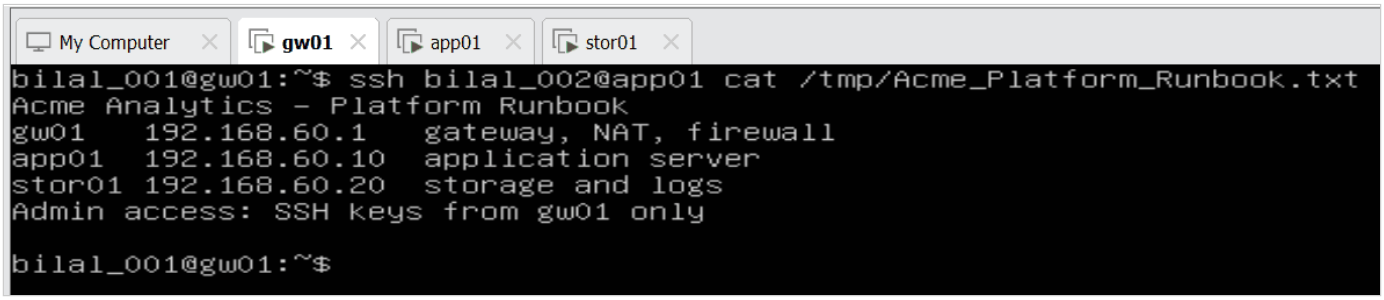
```
cat > Acme_Platform_Runbook.txt << 'EOF'      # write the lines below into the file …
Acme Analytics - Platform Runbook
gw01   192.168.60.1   gateway, NAT, firewall
app01  192.168.60.10  application server
stor01 192.168.60.20  storage and logs
Admin access: SSH keys from gw01 only
EOF                                           # … until the end marker EOF ("End Of File")
scp Acme_Platform_Runbook.txt bilal_002@app01:/tmp/        # copy it to app01's /tmp folder
ssh bilal_002@app01 cat /tmp/Acme_Platform_Runbook.txt     # run ONE command on app01 and show the result
```

📸 **Evidence – file created and copied (`100% 199 bytes`, no password):**



Runbook created with a heredoc and copied with scp

📸 **Evidence – read back on app01 with a one-shot SSH command:**



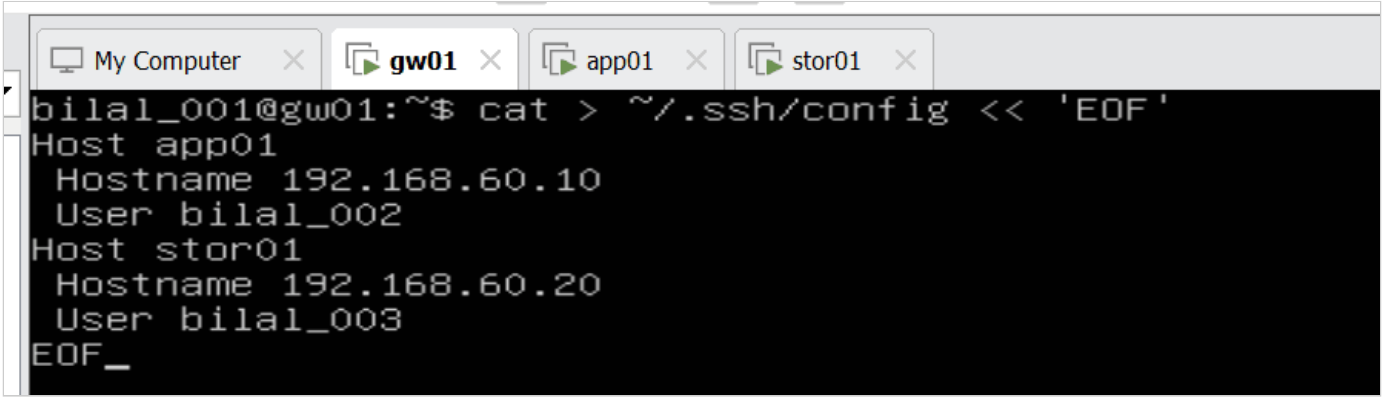
Runbook read on app01 via ssh

SSH shortcuts: `~/.ssh/config`

💡 **In plain words:** a personal address book for SSH – type `ssh app01` instead of `ssh bilal_002@192.168.60.10`.

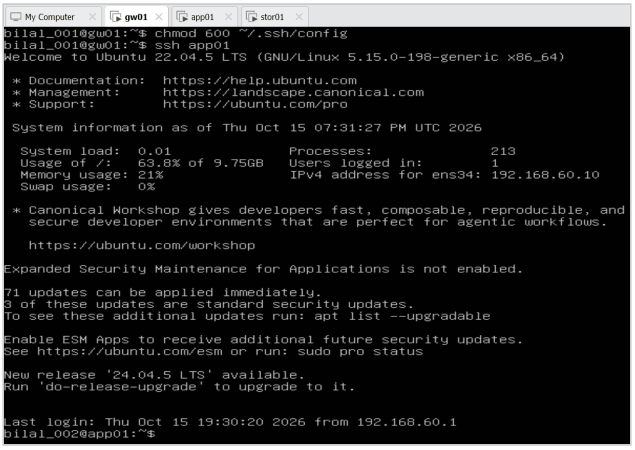
```
cat > ~/.ssh/config << 'EOF'
Host app01
    HostName 192.168.60.10
    User bilal_002
Host stor01
    HostName 192.168.60.20
    User bilal_003
EOF
chmod 600 ~/.ssh/config   # only the owner may read it — SSH refuses a config others can read
ssh app01                 # log in by short name — no user, no password
exit
ssh stor01
exit
```

📸 **Evidence – the shortcut file:**



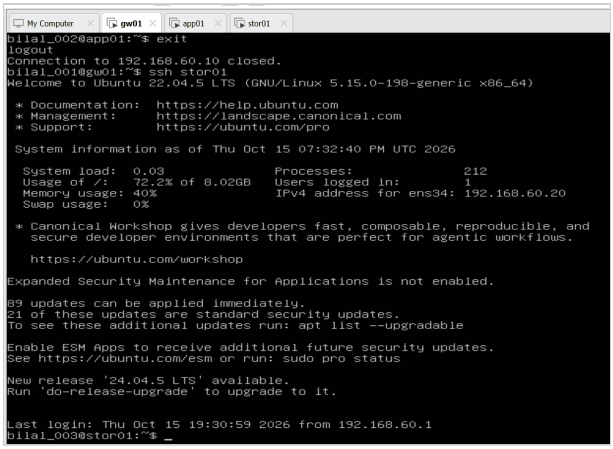
~/.ssh/config with Host entries for app01 and stor01

📸 **Evidence – `ssh app01` logs in as bilal_002 without a password:**



ssh app01 by alias

📸 Evidence – `exit` ("Connection to 192.168.60.10 closed"), then `ssh stor01`:

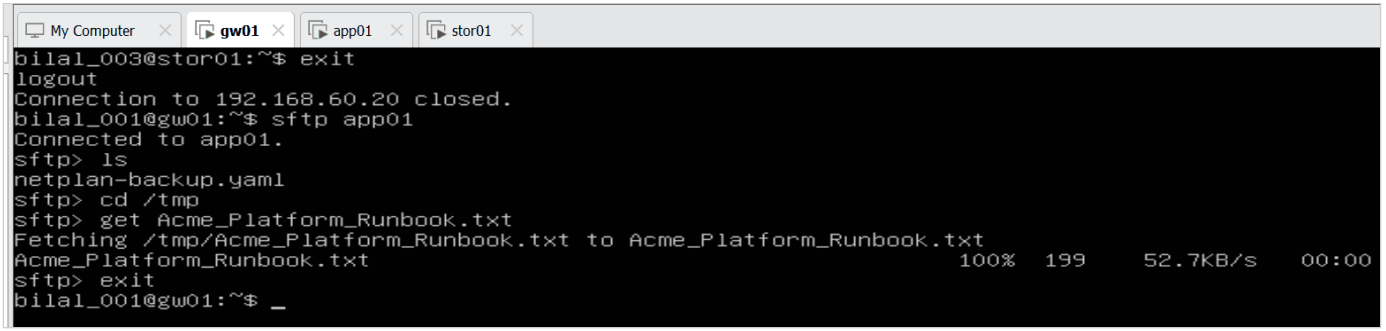


exit and ssh stor01 by alias

Interactive transfer: `sftp`

```
sftp app01                          # open a file session over SSH
sftp> ls                            # list files in the home folder on app01
sftp> cd /tmp                       # change folder on app01
sftp> get Acme_Platform_Runbook.txt # download to gw01
sftp> exit
```

📸 Evidence – runbook downloaded from app01 to gw01 (100 %):



sftp session: ls, cd, get

Why not FTP?

> **FTP sends usernames, passwords and files in clear text** – anyone listening on the network can read them. **SCP and SFTP run inside SSH**, so everything is **encrypted**, and with keys no password crosses the network at all. Companies do not use plain FTP for administration.

11. Step 1.7 – Hard links, soft links and inodes

💡 **In plain words:** every file has an **ID card** (the *inode*) holding its owner, permissions, size, dates and where the data sits on disk. The **file name is only a label** in a folder pointing to that ID card.

Procedure (on app01, as bilal_002)

```
mkdir -p ~/acme && cd ~/acme
echo "APP_PORT=8080" > app.conf          # an Acme config file
ln app.conf app.conf.backup              # hard link: second name, same inode
ln -s app.conf current.conf              # soft link: signpost holding the path "app.conf"
ls -li                                   # first column = inode number, third = link count
echo "LOG_LEVEL=info" >> app.conf.backup # write through the hard link
cat app.conf                             # the change appears here too — same file
stat app.conf | head -3                  # Inode and Links: 2
```

📸 Evidence – same inode 276722 and link count 2 for both names; the soft link has its own inode 276727; an edit via the hard link shows in the original:

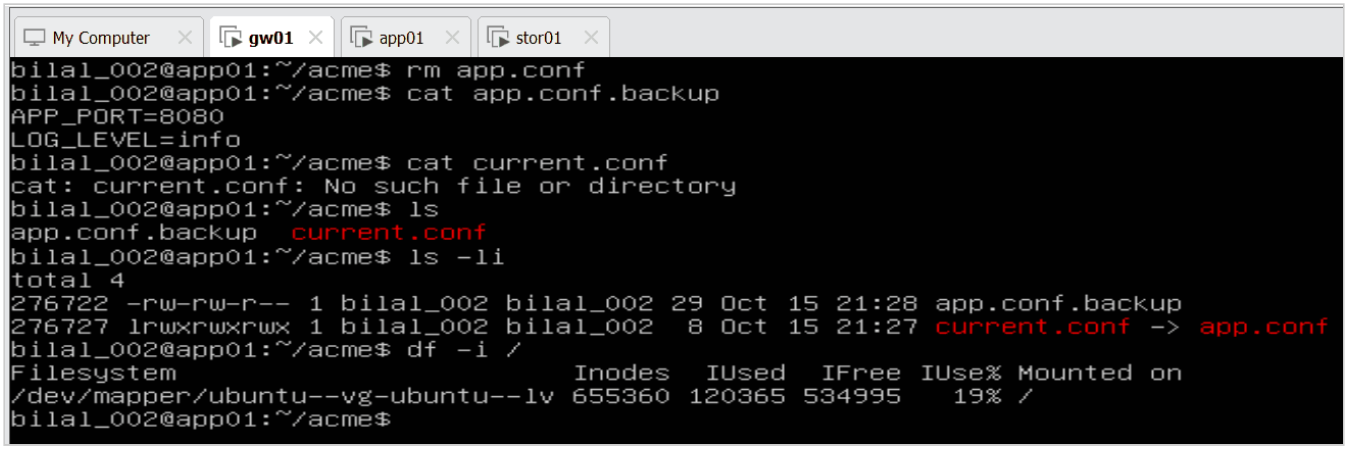
```
bilal_002@app01:~/acme$ ls -li
total 8
276722 -rw-rw-r-- 2 bilal_002 bilal_002 14 Oct 15 21:26 app.conf
276722 -rw-rw-r-- 2 bilal_002 bilal_002 14 Oct 15 21:26 app.conf.backup
276727 lrwxrwxrwx 1 bilal_002 bilal_002  8 Oct 15 21:27 current.conf -> app.conf
bilal_002@app01:~/acme$ echo "LOG_LEVEL=info" >> app.conf.backup
bilal_002@app01:~/acme$ cat app.conf
APP_PORT=8080
LOG_LEVEL=info
bilal_002@app01:~/acme$ stat app.conf | head -3
  File: app.conf
  Size: 29              Blocks: 8          IO Block: 4096   regular file
Device: fd00h/64768d    Inode: 276722      Links: 2
bilal_002@app01:~/acme$
```

ls -li, write through the hard link, stat

The “accidental delete” test

```
rm app.conf            # simulate a colleague deleting the file
cat app.conf.backup    # data is still there
cat current.conf       # "No such file or directory" — the signpost is broken
ls -li                 # link count of the survivor is now 1; current.conf shown in red
df -i /                # inode capacity of the root file system
```

📸 Evidence – hard link survives, soft link broken, inode usage 19 %:



rm original, hard link survives, soft link broken, df -i

Key understanding (in my own words)

> Linux works with inode numbers; the name I type is only a label in a folder. A **hard link** is a second label on the same inode – deleting one name loses nothing, but it does **not** protect against bad edits, because it is the same file. A **soft link** has its own inode and only holds a path – deleting it never harms the original, but deleting the original breaks it. Hard links protect important files against accidental deletion; real protection against bad changes still needs backups (Phase 2).

Step 1.7 complete. `~/acme` stays on app01 as sample data.

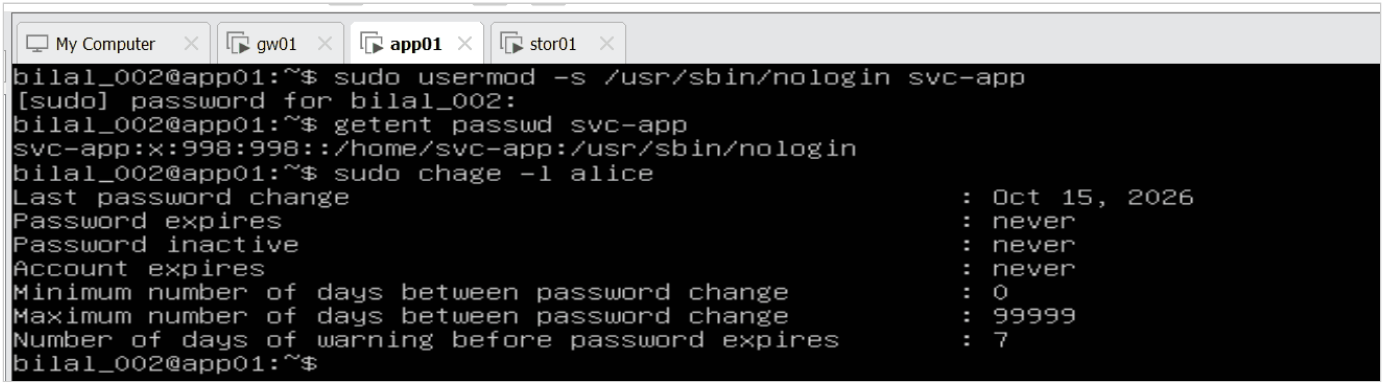
12. Step 1.8 – Users, groups and a service account (continued 2026-10-16)

💡 **In plain words:** every person gets their own account, people who do the same job share a **group**, and every **program** gets its own low-rights account that nobody can log in to. When someone leaves, their account is **locked**, not shared or forgotten.

Procedure (on app01)

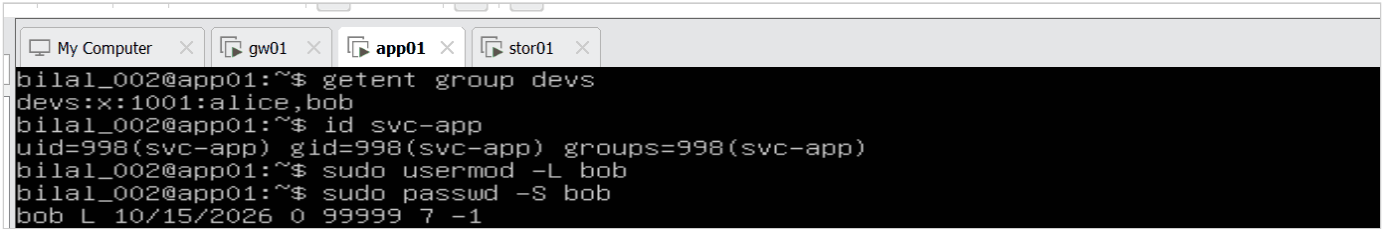
```
sudo groupadd devs                                  # team group (it already existed)
sudo useradd -m -s /bin/bash -G devs alice          # -m home folder, -s shell, -G extra group
sudo useradd -m -s /bin/bash -G devs bob
sudo useradd -r -s /usr/sbin/nologin svc-app        # -r system account, nobody can log in
sudo passwd alice                                   # give alice a password
id alice                                            # uid, primary group, secondary groups
getent group devs                                   # members of devs
sudo chage -l alice                                 # password aging ("change age")
sudo usermod -L bob                                 # lock (leaver)
sudo passwd -S bob                                  # status: L = locked, P = password, NP = none
sudo usermod -U bob                                 # unlock (returner)
```

📸 Evidence – svc-app has the shell `/usr/sbin/nologin` and system uid 998; alice's password aging:



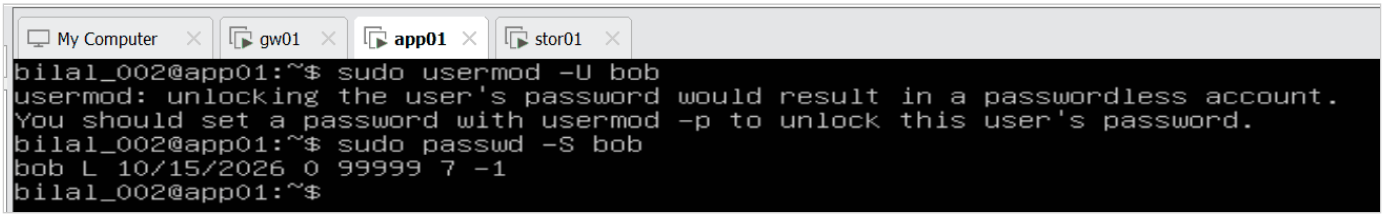
getent passwd svc-app and chage -l alice

📸 Evidence – devs = alice, bob; svc-app in its own group only; bob locked (L):



getent group devs, id svc-app, lock bob

📸 Evidence – unlock refused, because bob has no password yet:



usermod -U refused: passwordless account

Key understanding – final summary

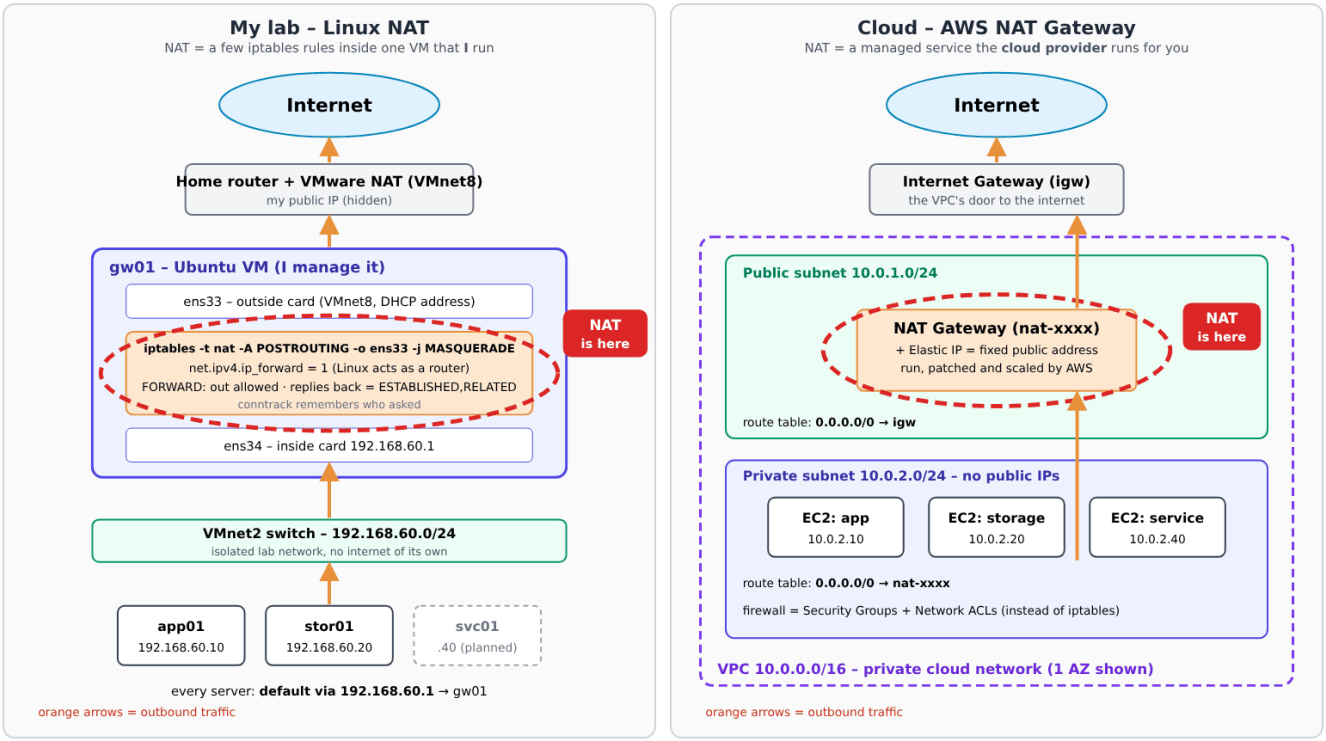
On app01, the server where the Acme application runs, we created named accounts for each developer – alice and bob – each with their own home folder (`-m`) and login shell (`-s`), and put both in a shared team group `devs`, so rights are given to the team instead of to each person; each user has a primary group (their own) and a secondary group (devs). We then created `svc-app` as a **system account** (`-r`, uid 998, below 1000) with the shell `nologin` and no password: no human can log in as it, it exists only so a program can run under it. Every service runs as some user – the main sshd as root with its pre-login part as `sshd`, nginx as `www-data` – and giving each service its own low-rights account means a hacked service reaches only that account's own files, not root. `chage` shows and sets the password-aging rules, and `usermod -L` / `-U` lock and unlock an account – the leaver/returner process. Linux refused to unlock bob while he had no password, because a human account must have a password or key before it is opened. This step answers the Acme customer's security question "who can log in, and with which rights?", and it is the base for later phases: in Phase 3 the app runs as `svc-app`, in Phase 6 groups control `sudo` and Docker access, and in Phase 7A Ansible automates joiners and leavers.

Step 1.8 complete.

13. Step 1.9 – Linux NAT vs Cloud NAT

Where does NAT live? Linux NAT (my lab) vs Cloud NAT (AWS)

Same job in both: private servers reach the internet, but the internet cannot start a connection to them. The red circle marks the NAT.



Same job: swap the private sender address for a public one on the way out, and let only the replies back in. **Difference:** in the lab I build, patch and keep the NAT alive; in the cloud the provider does, and charges per hour and per GB.

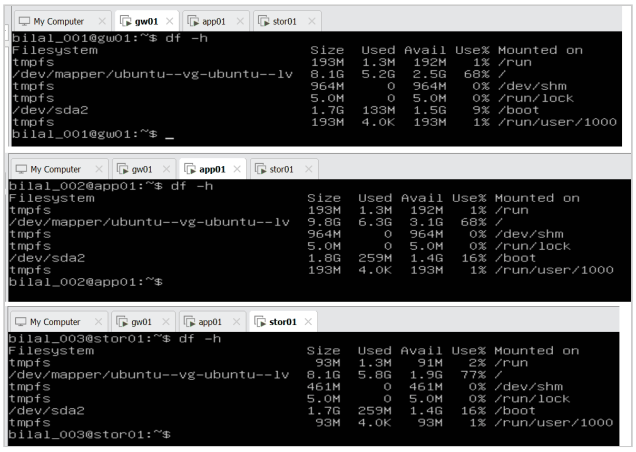
My lab piece	Cloud equivalent (AWS)	What it does
gw01 + iptables MASQUERADE	NAT Gateway	Rewrites the source address of outgoing traffic
ens33 / VMware NAT address	Elastic IP	The public address the internet sees
Home router / VMnet8	Internet Gateway	The door between the network and the internet
VMnet2 192.168.60.0/24	Private subnet	Servers with private addresses only
default via 192.168.60.1	Route table 0.0.0.0/0 → nat-xxxx	Sends unknown destinations to the NAT
iptables FORWARD rules	Security Groups + NACLs	Decides which traffic may pass

Muhammad Bilal Javed · Acme Analytics Linux Platform · Phase 1, Step 1.9

Linux NAT vs Cloud NAT

14. End of Phase 1 – updates and snapshot

📸 Evidence – disk usage on gw01, app01, stor01 before the snapshot:



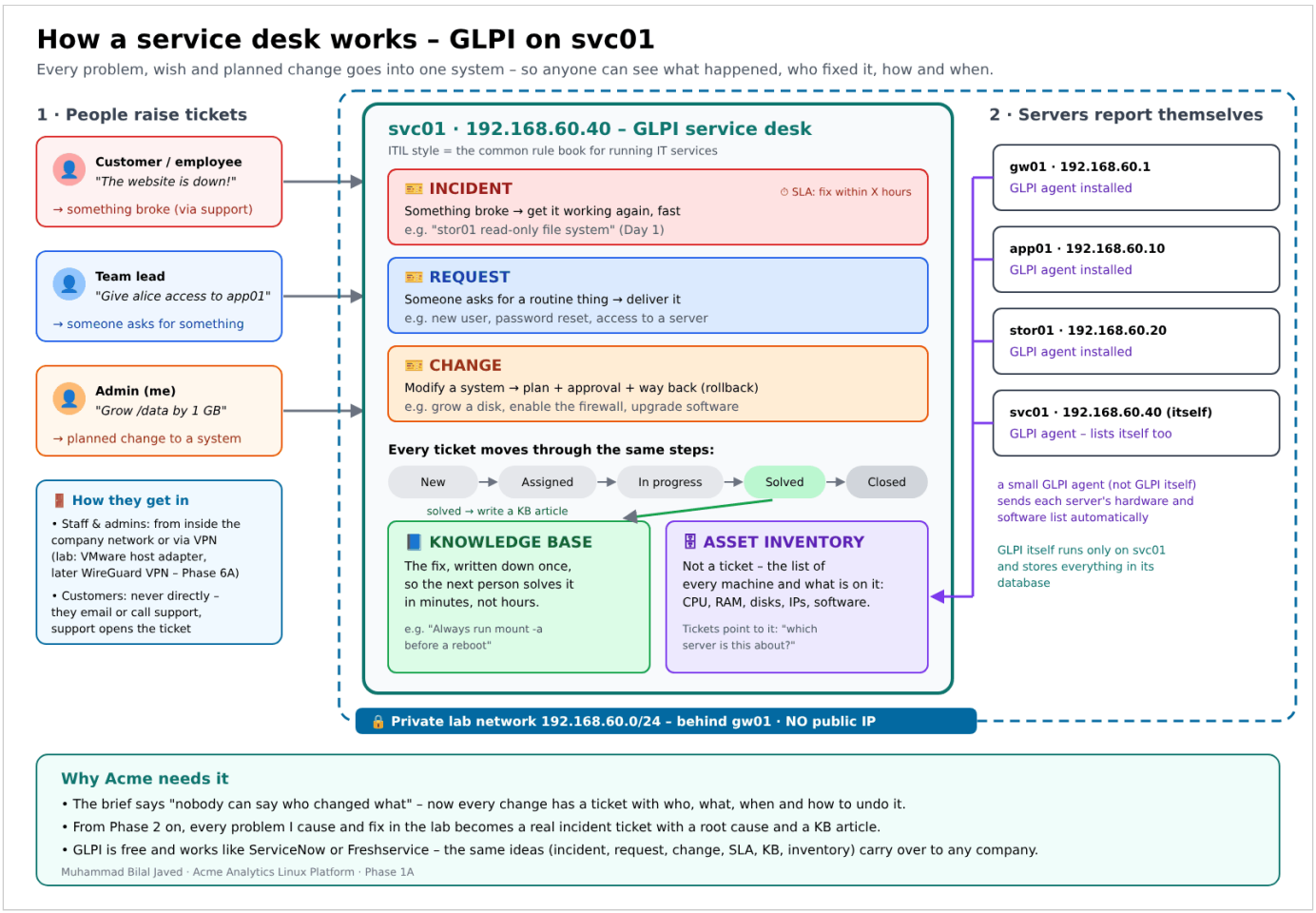
df -h on all three VMs

💡 **Why:** a snapshot taken while the VM is **off** is clean and small (no memory saved). If Phase 2 (storage) breaks something, I return to this point in one click instead of rebuilding the network. stor01's root disk is the fullest, which is fine: backups will go to its separate data disks (`sdb`, `sdc`) with LVM in Phase 2.

Day 3 – 16–17 Oct: Phase 1A – svc01, GLPI, inventory

1. The story – why a service desk, and why now

💡 **In plain words:** until now I fixed problems and wrote them down in these reports. A real IT team does not work like that: every problem, every change and every “please give me access” goes into a **ticket system**, so anyone can see *what happened, who fixed it, how, and when*. Phase 1A gives Acme that system.

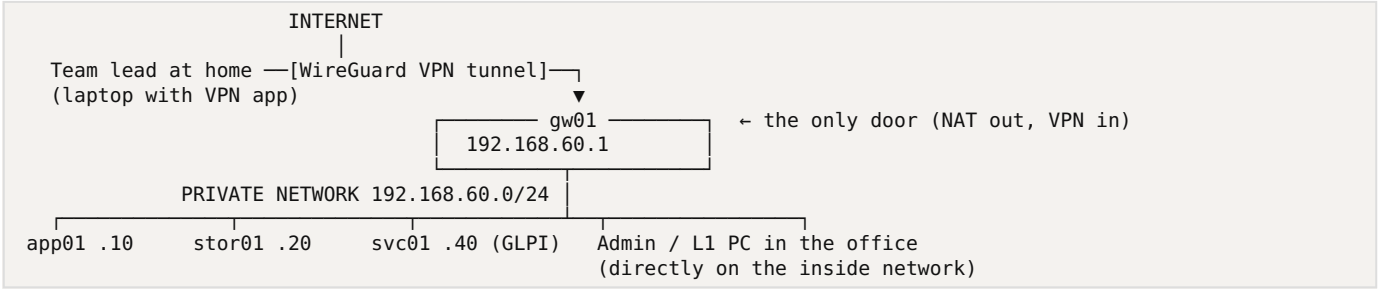


How a service desk works – GLPI on svc01

In the IT industry a problem is an **incident**, and handling them is **incident management**. When someone asks for a routine thing – a new user, a password reset, access to a server – it is a **request**. When I modify a system itself – grow a disk, enable a firewall, upgrade software – it is a **change**, which needs a plan, approval and a way back. All three are **tickets** in GLPI, and every ticket moves through the same steps: new → assigned → in progress → solved → closed; a solved fix is written down in the **knowledge base**. The **asset inventory** is not a ticket: it is the list of every machine and what is on it, filled automatically by the GLPI agent on each server, so a ticket can say which server it is about. GLPI follows **ITIL**, the common rule book for running IT services, and is used by many European companies and public bodies.

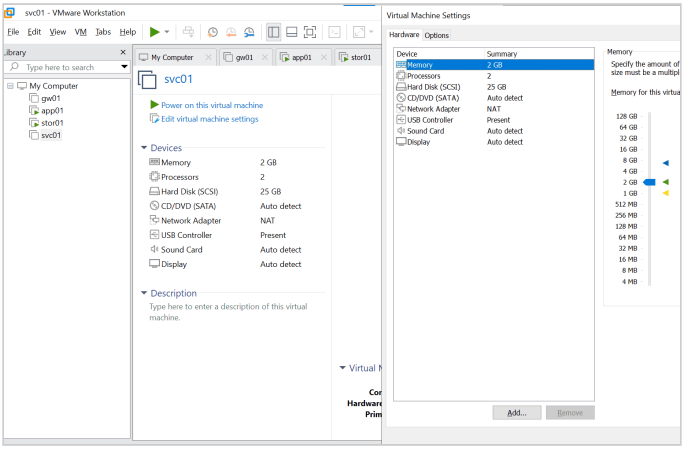
2a. How people reach svc01 – and where gw01 comes in

💡 **In plain words:** gw01 is the **only door** between the private network and the internet. People already **inside** (in the office) go straight to svc01. People **outside** (at home) must come in through the door with a **VPN** – like a company badge for remote workers. Strangers and customers never get in.



3. VM settings

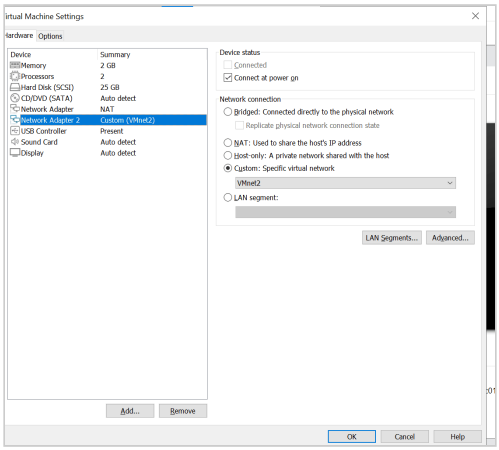
📸 **Evidence – VMware’s default settings: 2 GB, 2 CPUs, 25 GB, but the network adapter is on NAT:**



svc01 default settings with NAT

Issue: network adapter on NAT

📸 Evidence – Network Adapter 2 on Custom (VMnet2), connect at power on:

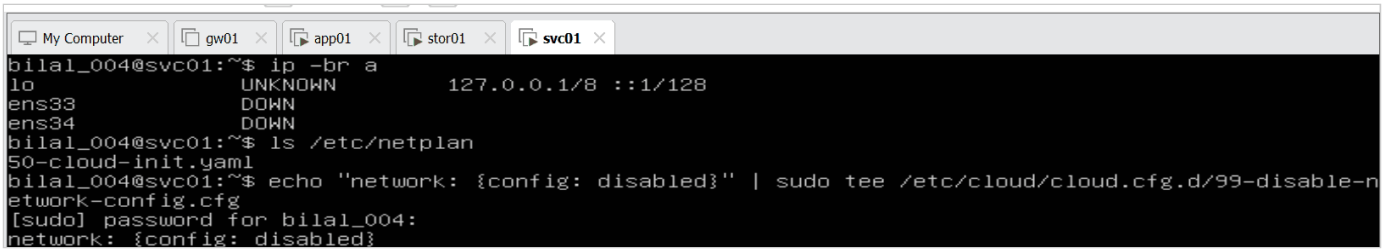


adapter on VMnet2

4. Installation and first boot

4.1 No network after the install

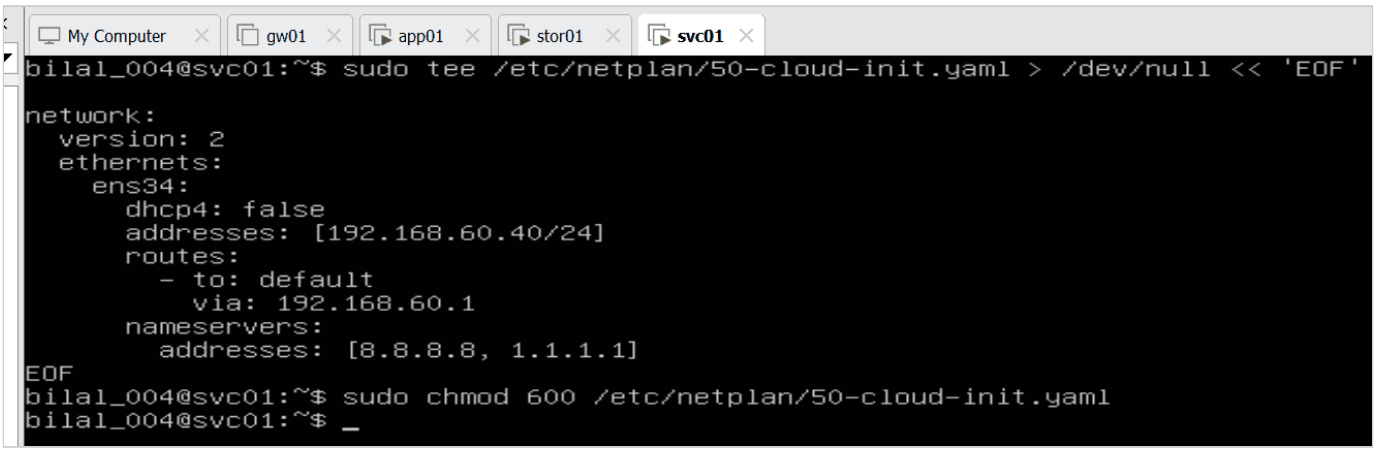
📸 Evidence – two cards, both DOWN, no IP address; only one netplan file:



ens33 and ens34 down

```
        echo "network: {config: disabled}" | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
        sudo tee /etc/netplan/50-cloud-init.yaml > /dev/null << 'EOF'
        network:
          version: 2
          ethernets:
            ens34:
              dhcp4: false
              addresses: [192.168.60.40/24]
              routes:
                - to: default
                  via: 192.168.60.1
              nameservers:
                addresses: [8.8.8.8, 1.1.1.1]
        EOF
        sudo chmod 600 /etc/netplan/50-cloud-init.yaml   # only root may read it
        sudo netplan try                                 # applies it; reverts after 120 s unless I press Enter
```

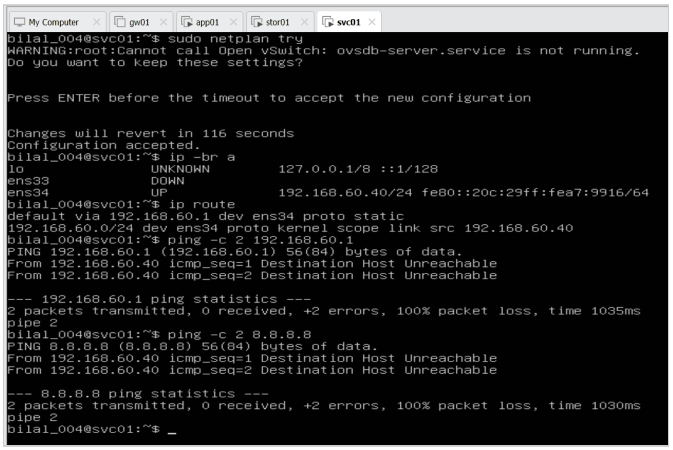
📸 Evidence – the netplan file:



netplan file for ens34

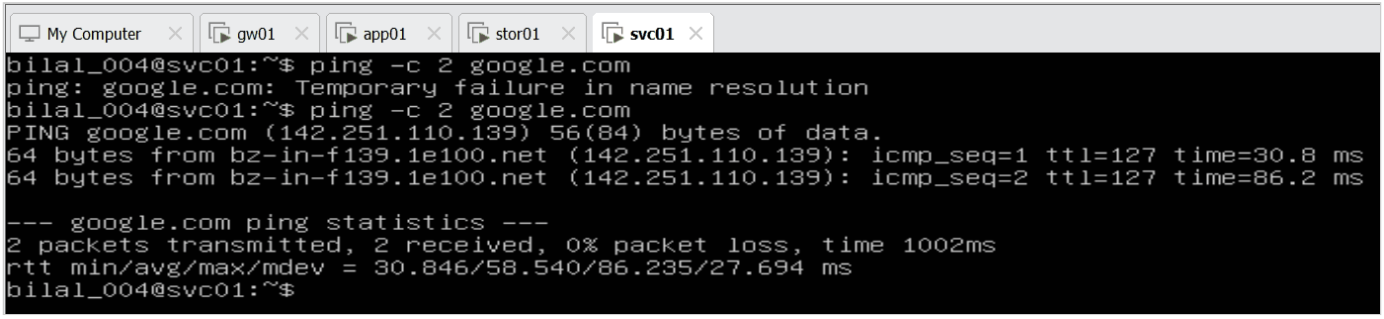
4.2 Address works – but gw01 was off

📸 Evidence – `ens34 UP 192.168.60.40/24`, default route via .1, but "Destination Host Unreachable":



netplan try, ping unreachable

📸 Evidence – after starting gw01: google.com resolves and answers:

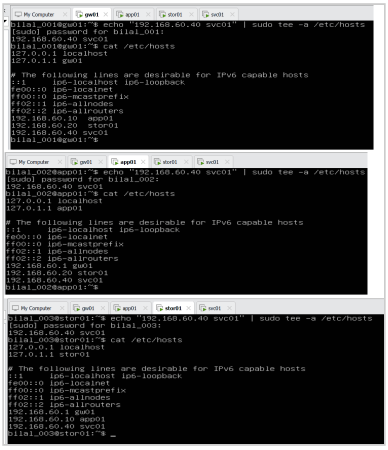


DNS and internet through gw01

4.3 Names: `/etc/hosts` on all four servers

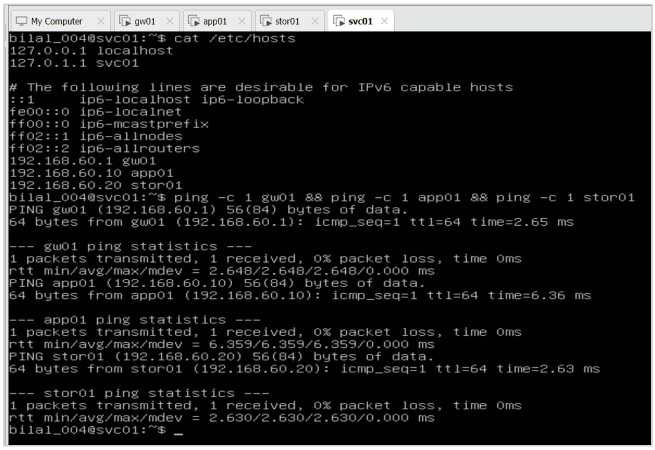
```
        echo "192.168.60.40 svc01" | sudo tee -a /etc/hosts      # on gw01, app01, stor01
        echo "192.168.60.1 gw01"    | sudo tee -a /etc/hosts     # on svc01 (one line per echo)
        echo "192.168.60.10 app01"  | sudo tee -a /etc/hosts
        echo "192.168.60.20 stor01" | sudo tee -a /etc/hosts
```

📸 Evidence – svc01 added on gw01, app01 and stor01:



hosts on the other three servers

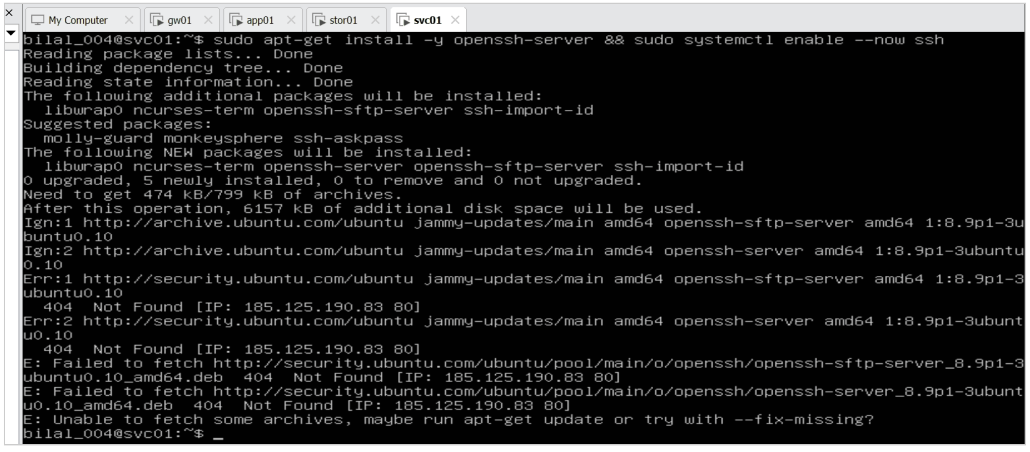
📸 Evidence – svc01's list and ping by name to all three (`ttl=64` = same network, no router in between; `&&` = run the next command only if this one worked):



svc01 hosts and ping by name

4.4 SSH: package list out of date

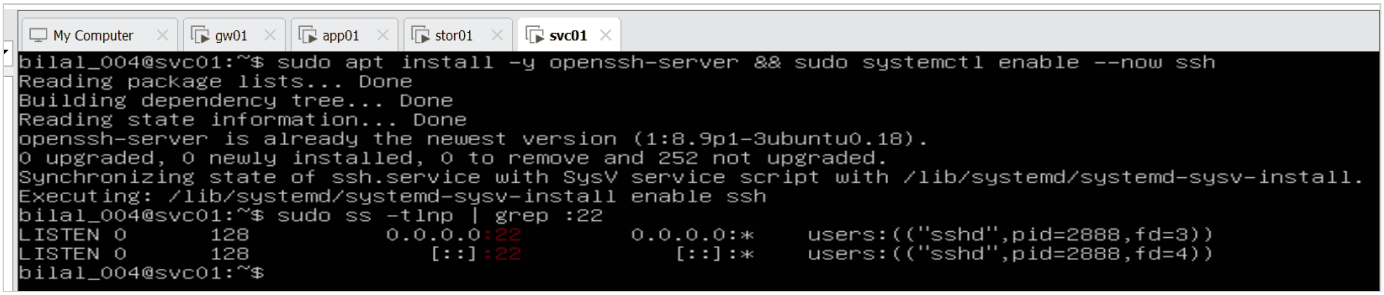
📸 Evidence – the 404 (note `jammy` = Ubuntu 22.04):



apt 404

```
        sudo apt update
        sudo apt install -y openssh-server
        sudo systemctl enable --now ssh
        sudo ss -tlnp | grep :22
```

📸 Evidence – sshd listening on 0.0.0.0:22 and [::]:22:

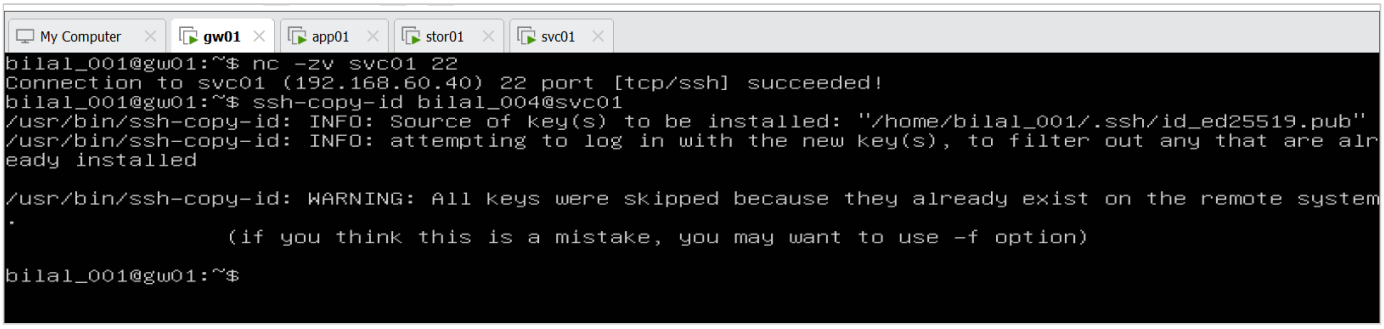


sshd listening

4.5 Key login and shortcut from gw01

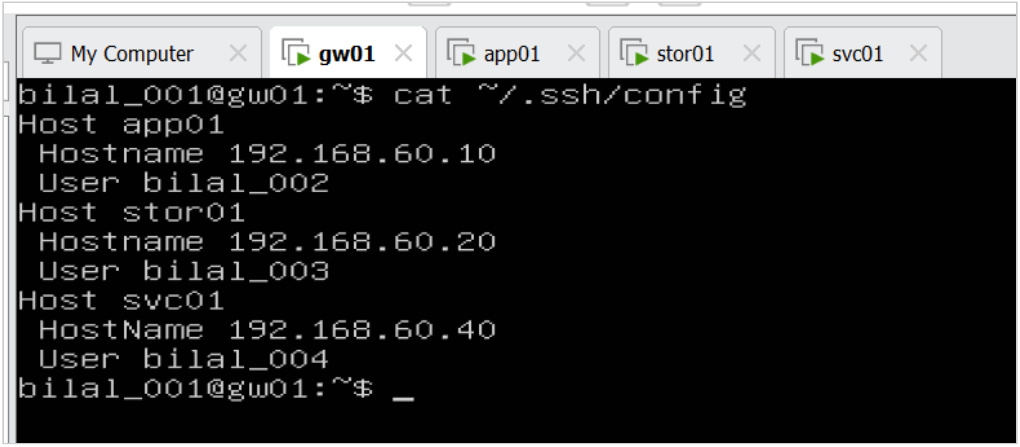
```
nc -zv svc01 22                 # door open?
ssh-copy-id bilal_004@svc01     # copy bilal_001's public key
cat >> ~/.ssh/config << 'EOF'
Host svc01
    HostName 192.168.60.40
    User bilal_004
EOF
ssh svc01
```

📸 **Evidence – port 22 open, key already present:**



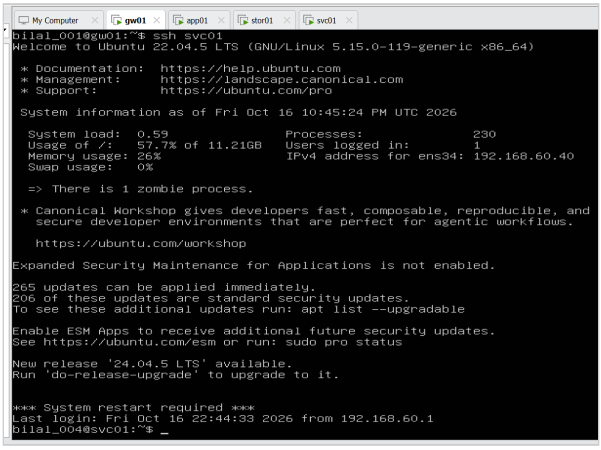
nc and ssh-copy-id

📸 **Evidence – gw01's SSH address book now has app01, stor01 and svc01 (keywords are not case-sensitive: `Hostname` = `HostName`):**



ssh config

📸 **Evidence – `ssh svc01` logs in as bilal_004 without a password:**



ssh svc01

Key understanding (in my own words)

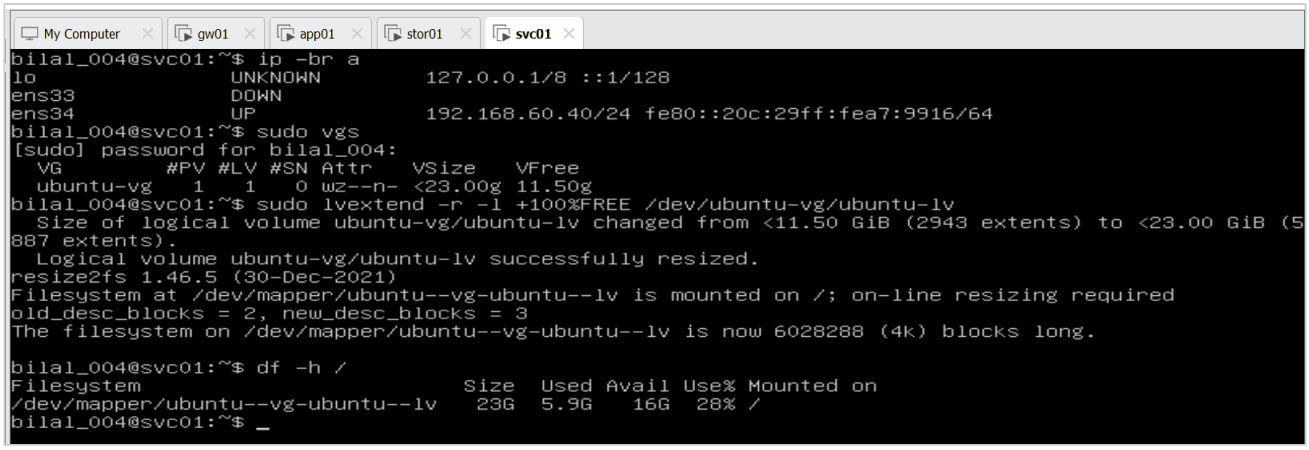
> svc01 joined the lab like app01 and stor01: one card on VMnet2, a fixed address set in netplan, and the default route pointing to gw01. Because gw01 is the only door, svc01 has no internet and no DNS while gw01 is off. Every machine has its own `/etc/hosts`, so a new server must be added to the others' lists and learn theirs. On a fresh server, `apt update` comes before `apt install`. With the key and the SSH shortcut, gw01 now manages all three servers by name, without passwords.

5. Growing the disk before Docker (LVM) – 2026-10-17

💡 **In plain words:** the welcome screen showed the system using only **11.5 GB of the 25 GB disk**. GLPI is about to move in – Docker images, a MySQL database with every ticket, user and inventory record, and ticket attachments – and all of it grows every day. A full disk makes MySQL crash, which would take the service desk itself down: exactly the Acme problem *"disks fill up without warning"*. So the disk was grown **before** installing anything. The system was also fully patched first, because GLPI will open a web service on port 80.

```
        ip -br a                                                # ens34 192.168.60.40 (ens33 = old NAT adapter, to be
removed)
        sudo vgs                                                # the pool: VSize 23 GB, VFree 11.5 GB unused
        sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv  # give the volume all free space; -r grows the file
system too
        df -h /                                                 # 23 GB, 28 % used
        sudo apt update && sudo apt upgrade -y                  # patch before opening a service
```

📸 **Evidence – `vgs` shows 11.5 GB free in the pool; `lvextend` grows the volume from 11.5 to 23 GB while running; `df` shows 23 GB:**



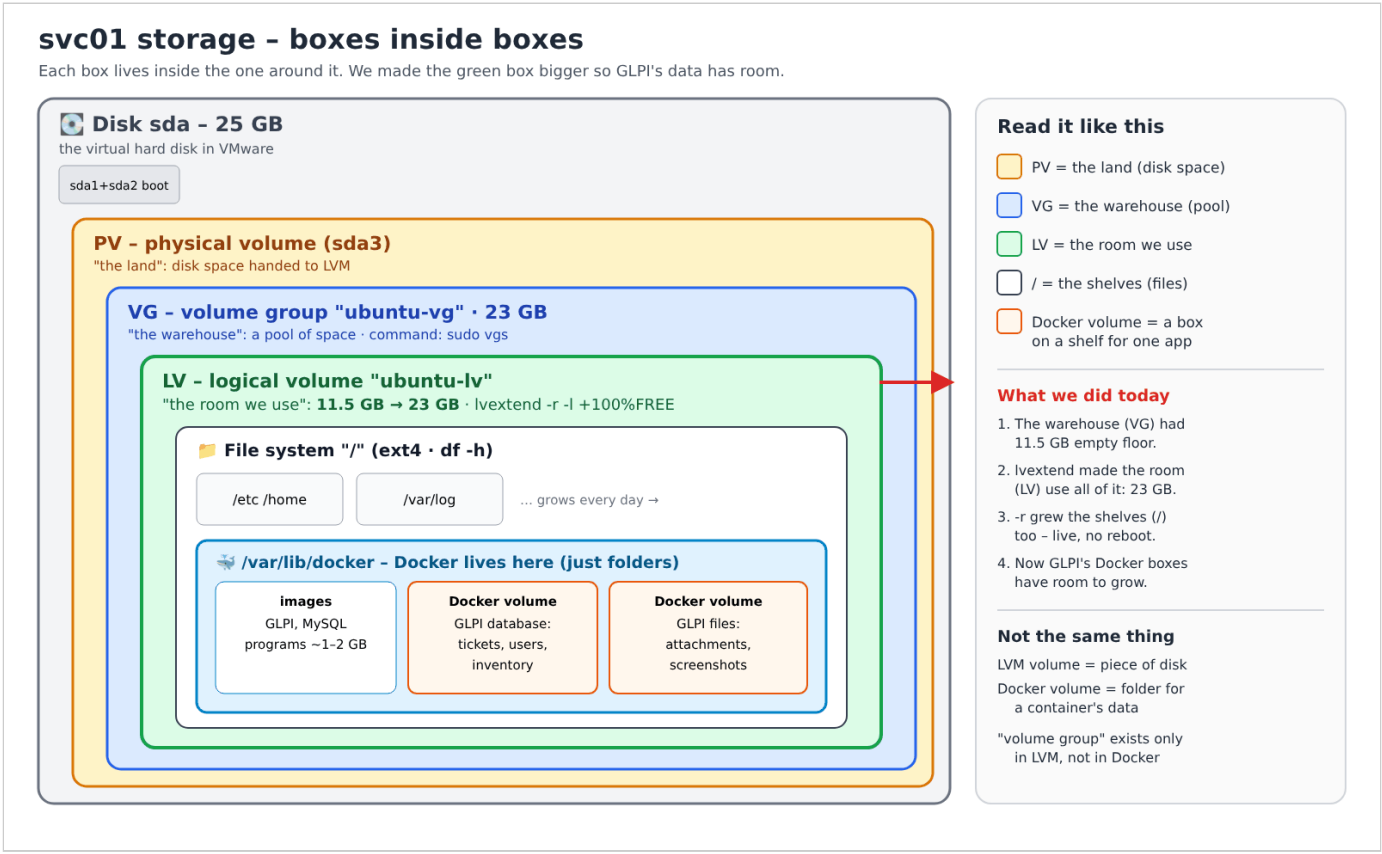
vgs, lvextend, df

📸 **Evidence – all packages up to date:**

```
bilal_004@svc01:~$ df -h /
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   23G  5.9G   16G  28% /
bilal_004@svc01:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,907 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,288 kB]
Fetched 5,324 kB in 3s (1,947 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0
  libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager tcl tcl8.6 usb-modeswitch
  usb-modeswitch-data
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
bilal_004@svc01:~$ _
```

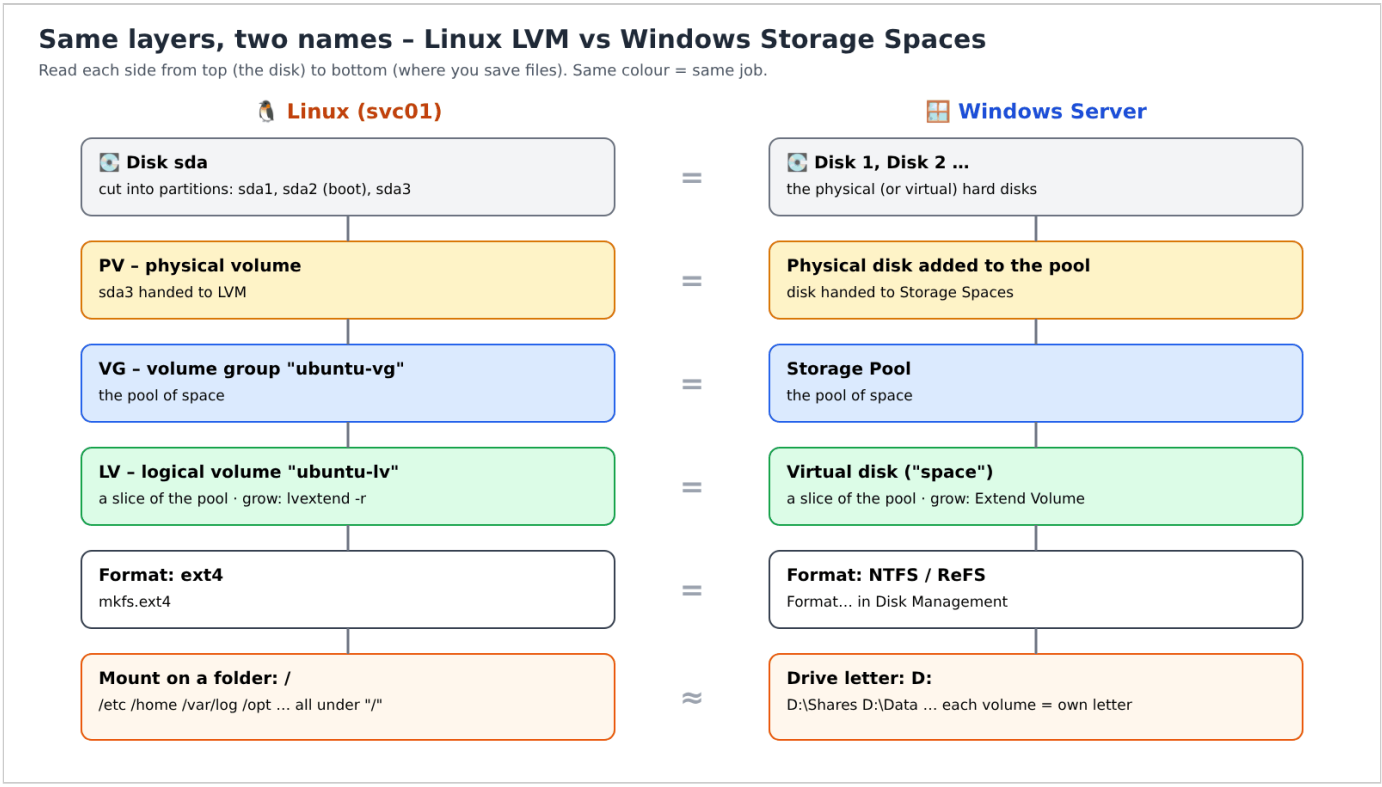
apt upgrade

The picture – boxes inside boxes



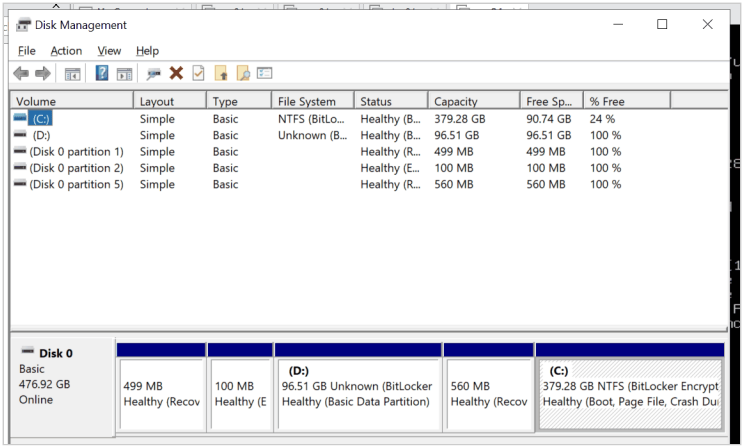
svc01 storage layers

Same layers in Windows



Linux LVM vs Windows Storage Spaces

📸 On my own PC (Disk Management, `diskmgmt.msc`): a "Basic" disk – Disk 0 cut into partitions with drive letters (C:, D:), no pool layer:



Windows Disk Management

Key understanding (in my own words)

When I created svc01, I gave it one 25 GB virtual disk, **sda**. A disk can be divided into **partitions**, like C: and D: in Windows. Here, sda1 and sda2 are for booting, and **sda3** (23 GB) is used as a **physical volume (PV)**. On its own, a PV is just raw space. It is never divided: once it is added to a **volume group (VG)**, a pool of space, it belongs to that VG only. One VG can be built from one or several PVs. Here there is one VG, `ubuntu-vg` (23 GB), made from one PV. For a second VG, I would first create another partition on the disk, which becomes its own PV.

Inside a VG I can create many **logical volumes (LVs)**. Here there was one, `ubuntu-lv`, with 11.5 GB, and the other 11.5 GB were free. I could have used the free space for more LVs; separate LVs give **isolation of space**: if one fills up (for example `/var/log` with logs), the others are not affected, and each LV can be grown, backed up or snapshotted on its own. This is about disk space, not security; data is protected by permissions, firewalls and encryption. On svc01 I chose to grow the single LV to the full 23 GB with `lvextend -r -l +100%FREE` while the system was running, and `-r` grew the file system too.

An LV is only usable once it is **formatted** (ext4) and **mounted**; this one is mounted as `/`, the umbrella for every folder (`/etc`, `/home`, `/var/log`, `/var/lib/docker`, `/opt`).

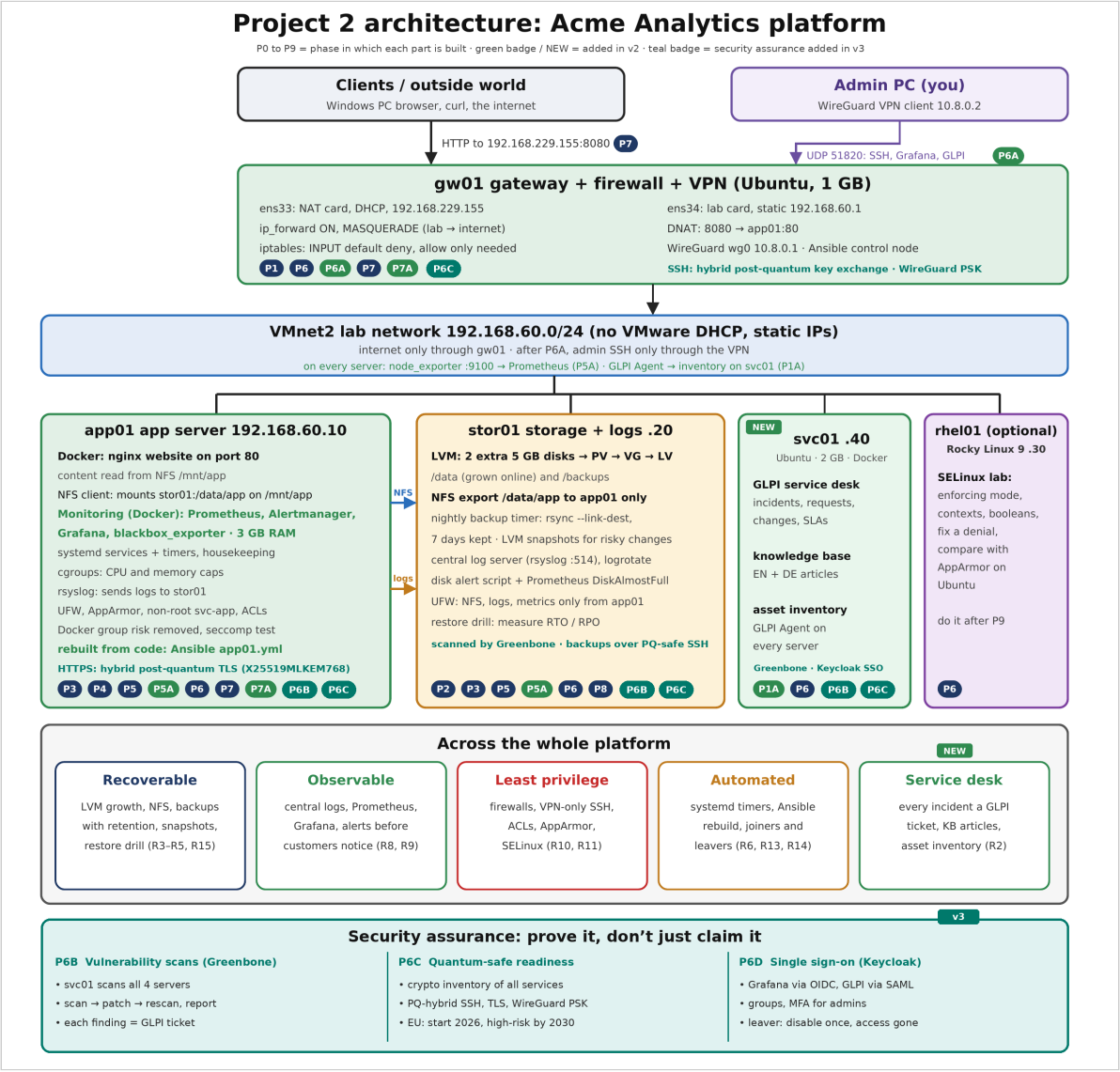
The same idea in Windows: physical disks (≈ PVs) are added to a **Storage Pool** (≈ VG) in Storage Spaces. The pool is cut into **virtual disks** (≈ LVs), which are formatted with NTFS or ReFS (≈ ext4) and given a drive letter such as D: (≈ mounting). Extend Volume in Disk Management grows a volume live, like

`lvextend -r`. Linux uses one folder tree under `/`, while Windows uses drive letters, but the layers are the same: disk → pool → volume → file system → mount point. On a normal PC like mine the disk is “Basic”: only partitions with drive letters, no pool – storage pools are used on servers with several disks.

For the service desk, Docker takes **images** (the installers for GLPI and MySQL) and runs them as **containers**. Their data is saved in **mounted folders** on svc01, for example `/opt/helpdesk/storage/mysql`, which appears inside MySQL as `/var/lib/mysql`. So tickets, users and inventory stay safe on `/` even if a container is deleted or upgraded, and that is why I grew the disk before installing GLPI.

6. The network story – from the first picture to today

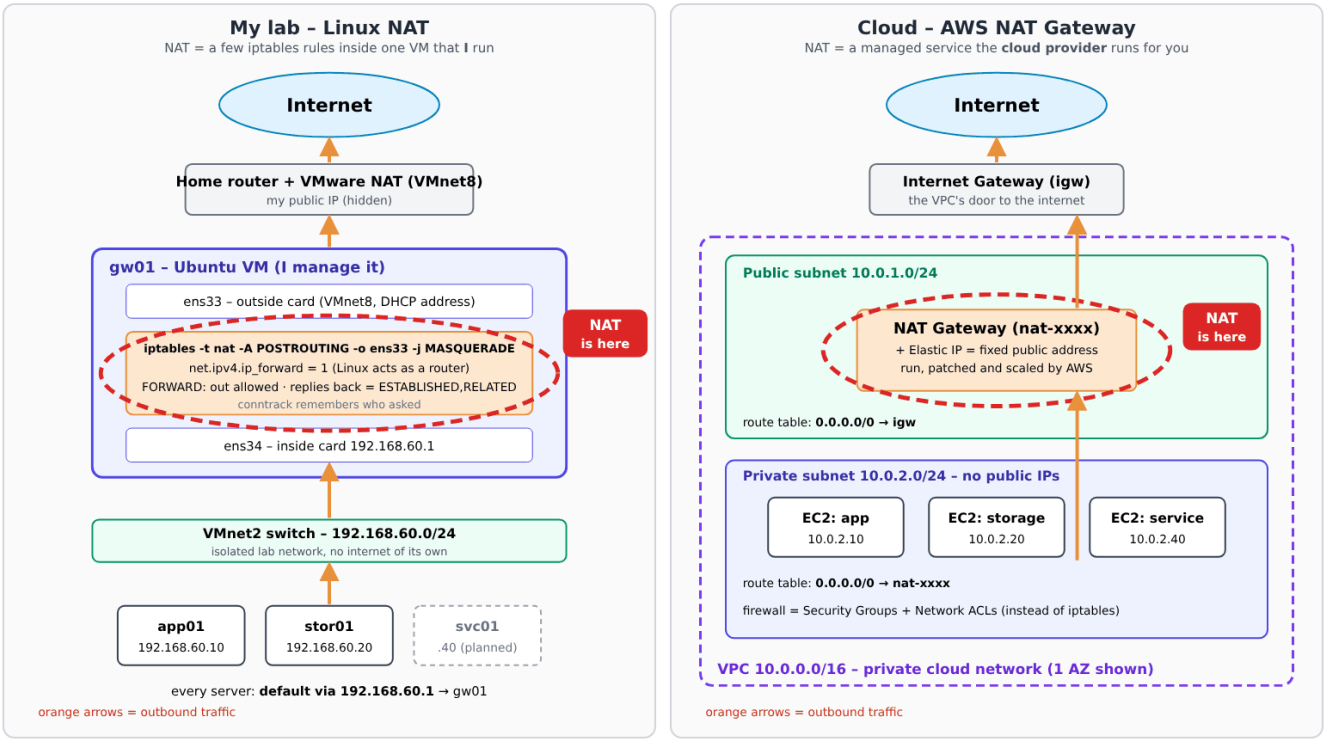
💡 **Read this section alone and you have the whole network in your head – no need to go back to Day 1.**



Project architecture

Where does NAT live? Linux NAT (my lab) vs Cloud NAT (AWS)

Same job in both: private servers reach the internet, but the internet cannot start a connection to them. The red circle marks the NAT.

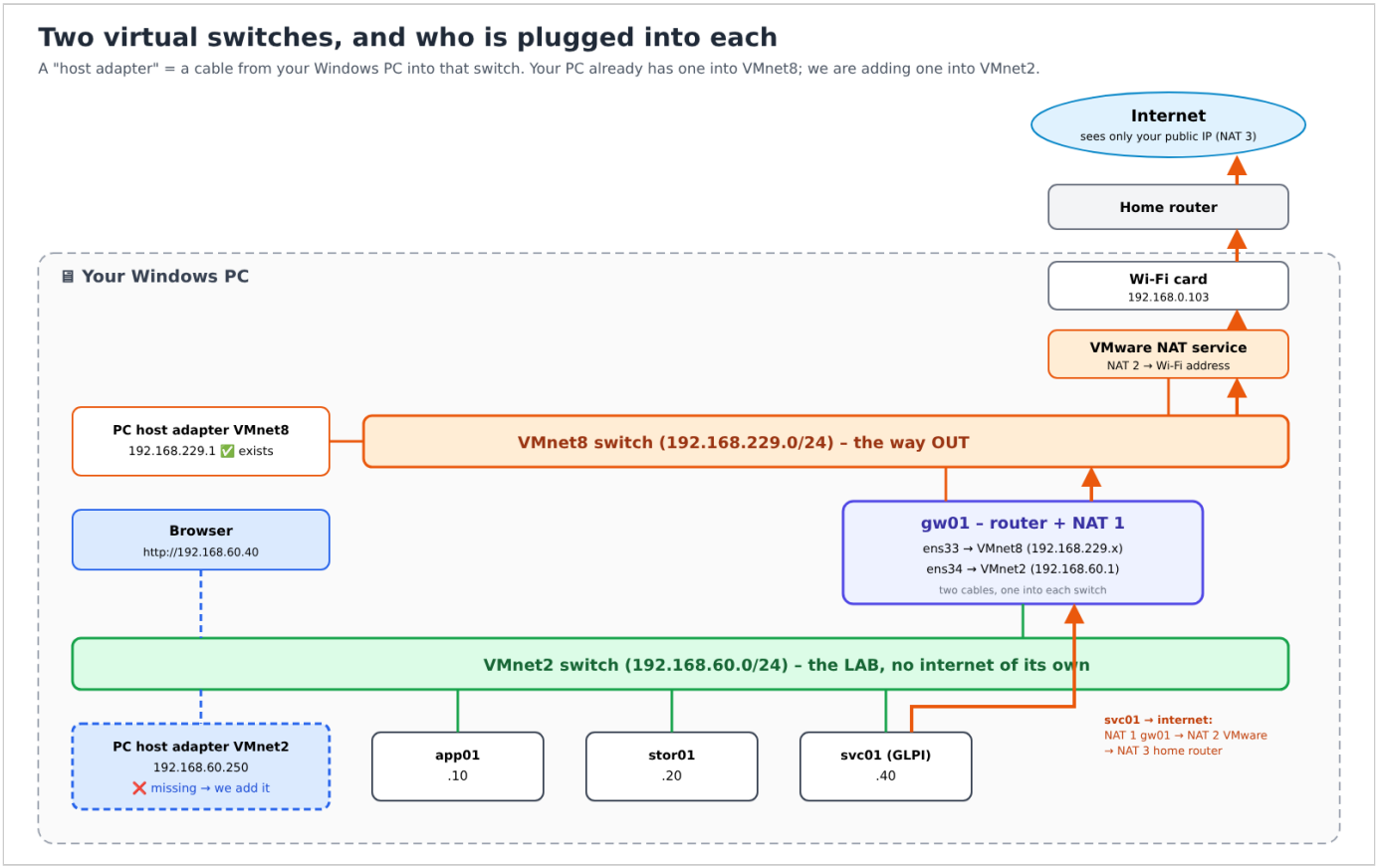


Same job: swap the private sender address for a public one on the way out, and let only the replies back in. **Difference:** in the lab I build, patch and keep the NAT alive; in the cloud the provider does, and charges per hour and per GB.

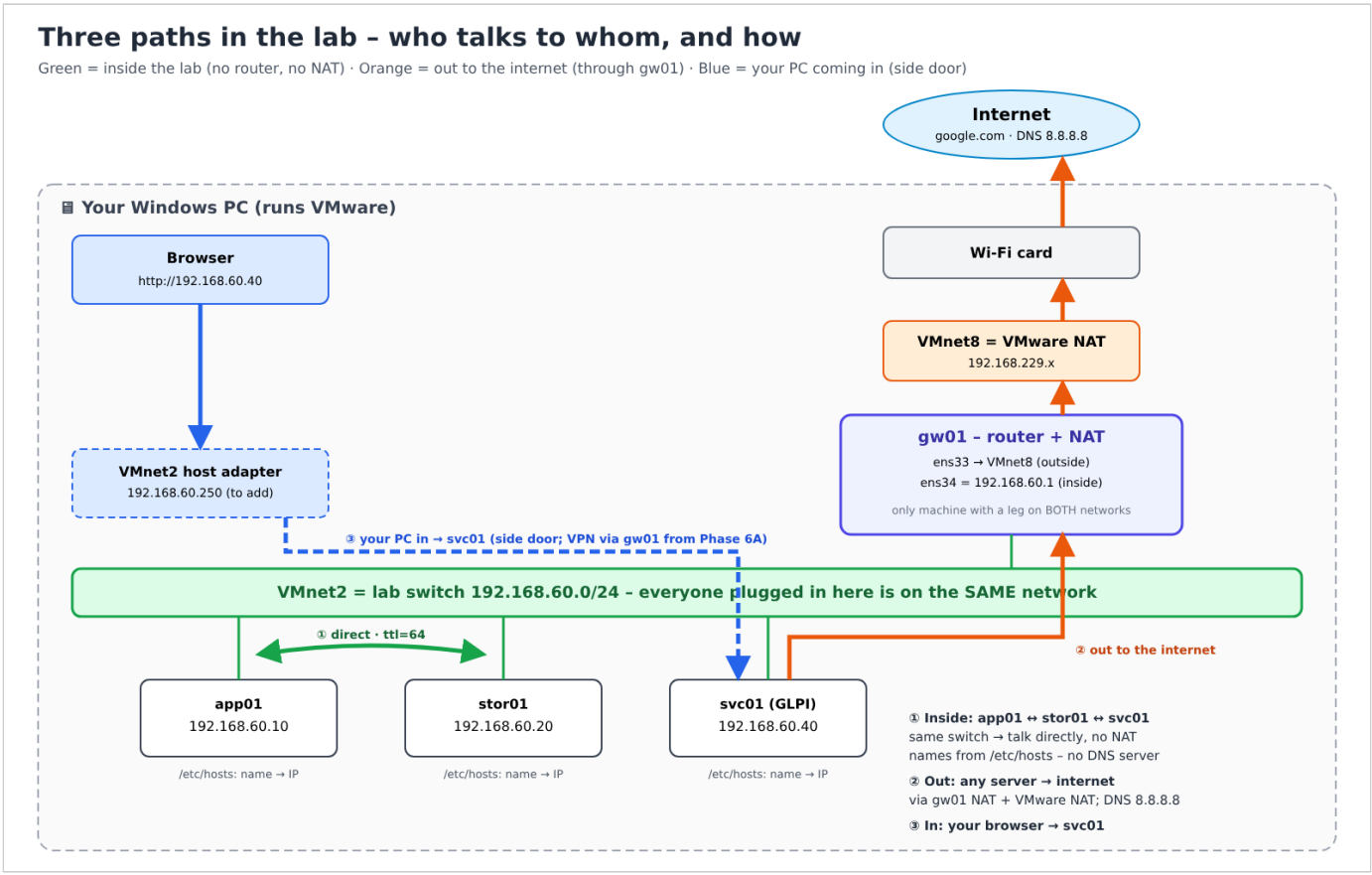
My lab piece	Cloud equivalent (AWS)	What it does
gw01 + iptables MASQUERADE	NAT Gateway	Rewrites the source address of outgoing traffic
ens33 / VMware NAT address	Elastic IP	The public address the internet sees
Home router / VMnet8	Internet Gateway	The door between the network and the internet
VMnet2 192.168.60.0/24	Private subnet	Servers with private addresses only
default via 192.168.60.1	Route table 0.0.0.0/0 → nat-xxxx	Sends unknown destinations to the NAT
iptables FORWARD rules	Security Groups + NACLs	Decides which traffic may pass

Muhammad Bilal Javed · Acme Analytics Linux Platform · Phase 1, Step 1.9

Linux NAT vs cloud NAT



Two switches and the host adapters



Three paths in the lab

Key understanding (in my own words). Every server has a network card. VMware calls it a “Network Adapter”. Inside Linux the card is called ens34 or ens33.

A virtual switch is like an extension board. VMnet2 is the lab board. VMnet8 is the board for the way out.

app01, stor01 and svc01 are plugged into VMnet2. gw01 is also plugged into VMnet2 with its inside card. All of them have an address in the same network. So they talk to each other directly by IP. This worked from the first day.

To use names, Linux looks in two places. First it checks `/etc/hosts`. Then it asks DNS. Our lab names are written in `/etc/hosts`, so they are found in the first place. Internet names like google.com are not in `/etc/hosts`. So Linux asks DNS. The helper 127.0.0.53 forwards the question to 8.8.8.8 and keeps the answer in a cache.

For the internet, the servers send their traffic to gw01. gw01 has a second card on VMnet8. The address is changed three times on the way out. First gw01 changes it to its own address on VMnet8. Then VMware's NAT changes it to my Wi-Fi card's address. Then my home router changes it to my public IP. The DNS question to 8.8.8.8 also takes this way. So nothing on the internet works while gw01 is off.

My Windows PC had a card on VMnet8 but no card on VMnet2. So my browser could not reach svc01. Now I give my PC a card on VMnet2 with the tick "Connect a host virtual adapter". The card gets the address 192.168.60.250. My browser can now open GLPI on svc01 directly. Nothing changes for the servers.

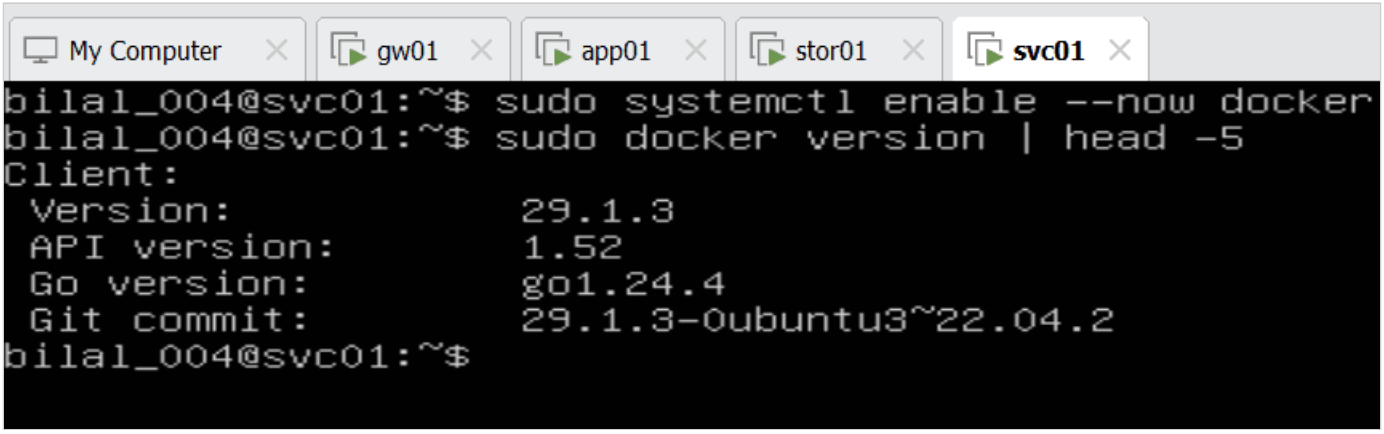
7. Step 1A.2 – Docker and GLPI running, opened in the browser

💡 **In plain words:** GLPI and its MySQL database run as two **containers** on svc01, described in one small file (`compose.yaml`). Their data lives in folders on svc01, so it survives updates. Then my Windows PC got a plug on the lab switch so the browser can open GLPI.

7.1 Install Docker

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2   # Docker engine + "docker compose"
sudo systemctl enable --now docker                                   # start now and at every boot
sudo docker version | head -5
```

📸 **Evidence – Docker 29.1.3 installed:**



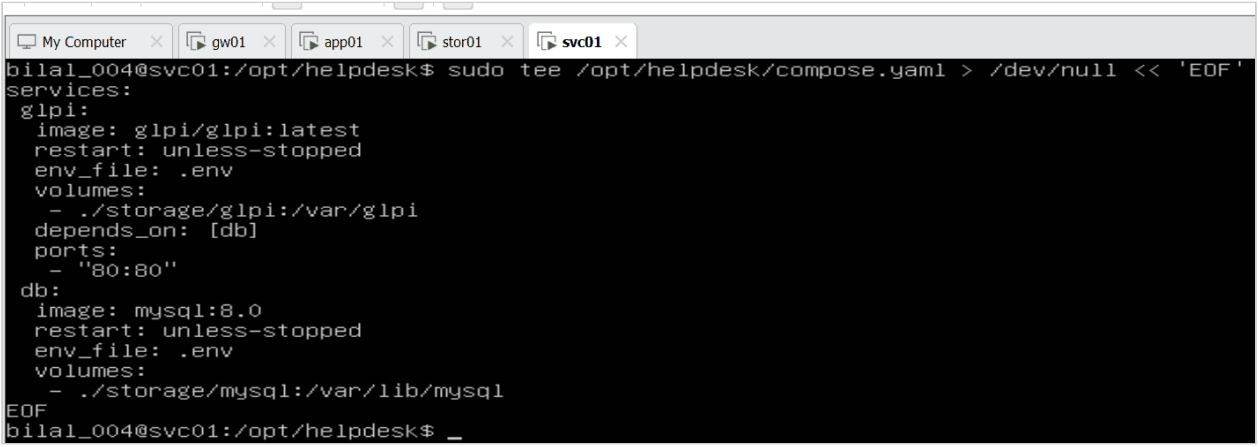
docker version

7.2 Settings file and compose file

```
sudo mkdir -p /opt/helpdesk && cd /opt/helpdesk
sudo tee /opt/helpdesk/.env > /dev/null << 'EOF'
GLPI_DB_HOST=db
GLPI_DB_PORT=3306
GLPI_DB_NAME=glpi
GLPI_DB_USER=glpi
GLPI_DB_PASSWORD=<lab-db-password>
MYSQL_RANDOM_ROOT_PASSWORD=yes
MYSQL_DATABASE=glpi
MYSQL_USER=glpi
MYSQL_PASSWORD=<lab-db-password>
EOF
sudo chmod 600 /opt/helpdesk/.env       # passwords inside – only root may read it
```

```
        # /opt/helpdesk/compose.yaml – which containers to run
        services:
          glpi:
            image: glpi/glpi:latest          # the GLPI installer (image)
            restart: unless-stopped          # start again after a crash or reboot
            env_file: .env
            volumes:
              - ./storage/glpi:/var/glpi     # svc01 folder shown inside the container
            depends_on: [db]                 # start the database first
            ports:
              - "80:80"                      # svc01 port 80 → container port 80
          db:
            image: mysql:8.0
            restart: unless-stopped
            env_file: .env
            volumes:
              - ./storage/mysql:/var/lib/mysql
```

📸 **Evidence – the compose file:**



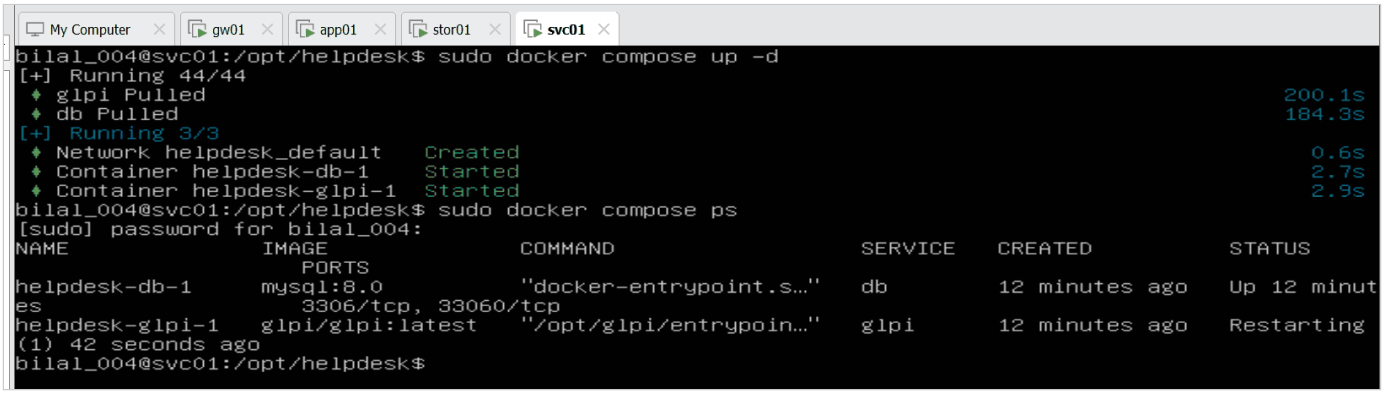
compose.yaml

Lesson – YAML is strict. My first attempts failed: `Services:` instead of `services:` (keywords are lowercase), `glpi:Latest` instead of `latest` (the tag is case-sensitive), `mysql: 8.0` with a space (YAML reads it as a new key → *"mapping values are not allowed"*), and `var/glpi` without the leading `/`. Check a file before starting it: `sudo docker compose config --quiet && echo "YAML OK".`

7.3 Start – and the permission problem

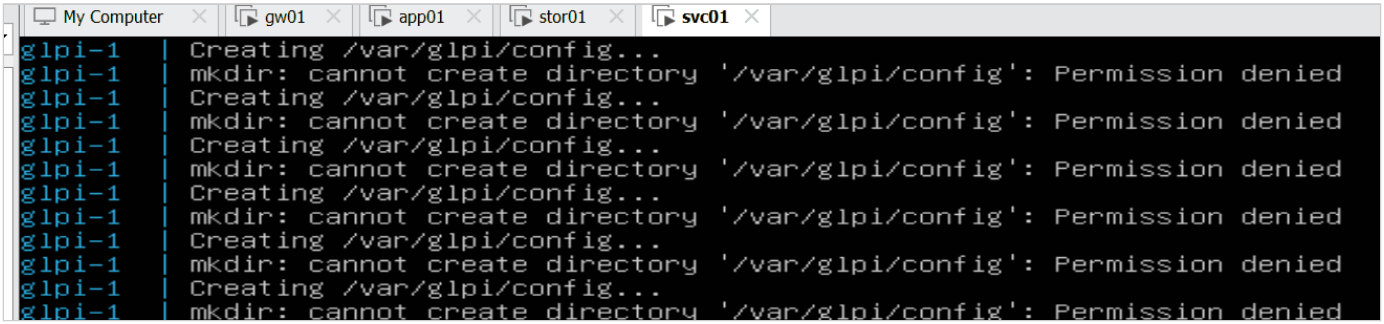
```
        sudo docker compose up -d      # download images, create network, start both containers
        sudo docker compose ps
```

📸 **Evidence – images pulled (~3 min), db "Up", but glpi "Restarting (1)":**



compose up, glpi restarting

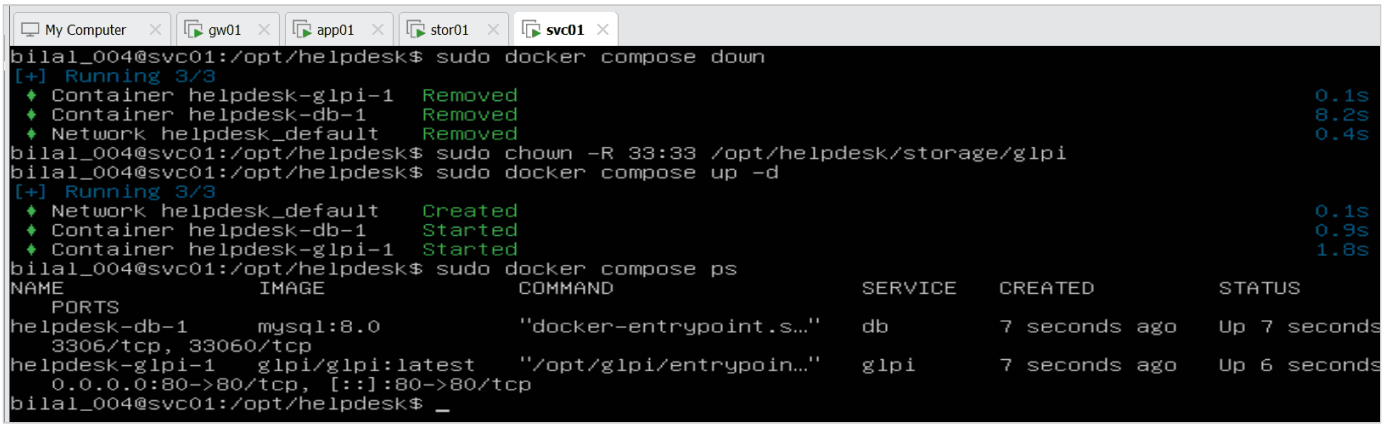
📸 **Evidence – the log:**



permission denied

```
        sudo docker compose down
        sudo chown -R 33:33 /opt/helpdesk/storage/glpi    # 33 = www-data inside the container
        sudo docker compose up -d
        sudo docker compose ps
```

📸 Evidence – both containers "Up"; port 0.0.0.0:80 → 80:

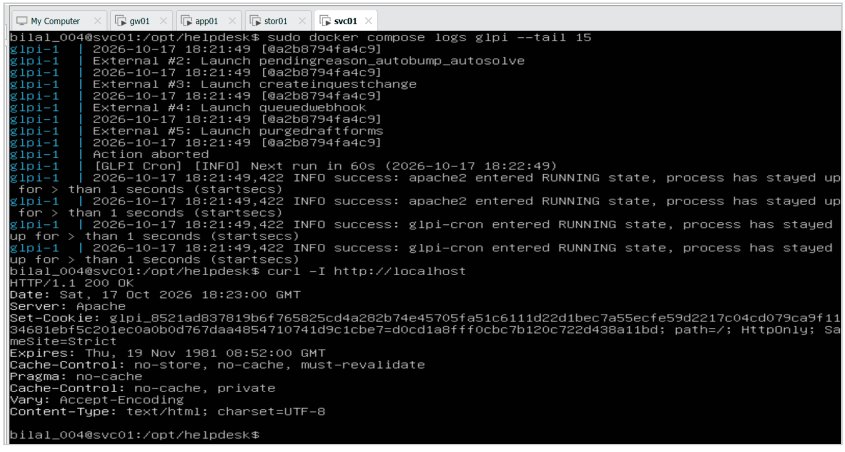


glpi up

7.4 Test from svc01 and from gw01

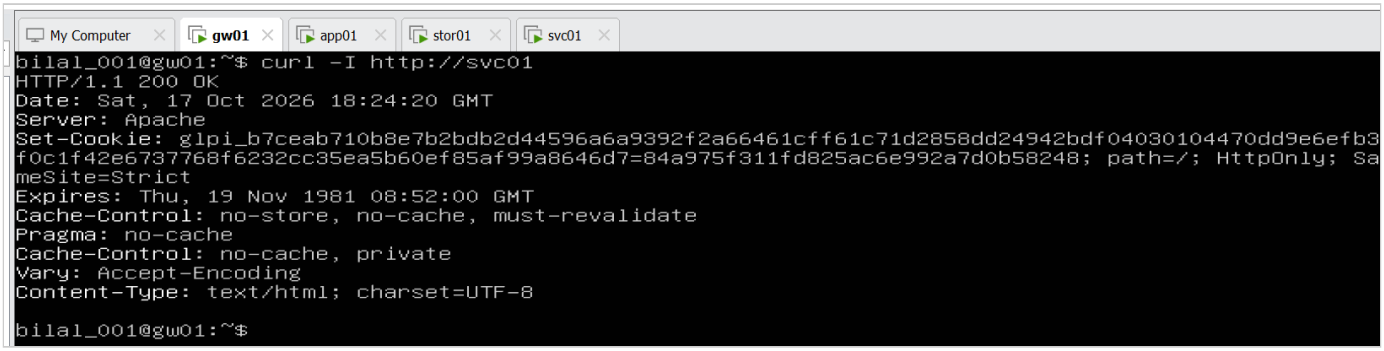
```
        sudo docker compose logs glpi --tail 15   # apache2 and glpi-cron "RUNNING"
        curl -I http://localhost                   # on svc01
        curl -I http://svc01                       # on gw01 — name from /etc/hosts
```

📸 Evidence – Apache and GLPI cron running; `HTTP/1.1 200 OK` on svc01:



glpi running, 200

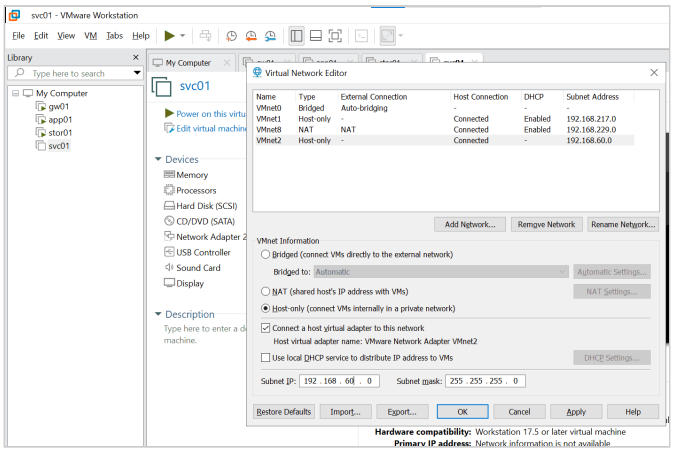
📸 Evidence – `200 OK` from gw01 over the lab network:



gw01 curl 200

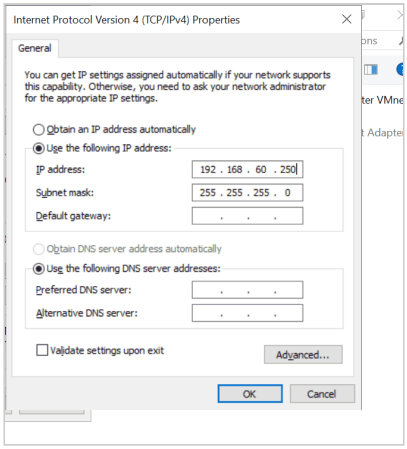
7.5 My PC on the lab switch – the VMnet2 host adapter

📸 Evidence – VMnet2 in the Virtual Network Editor:



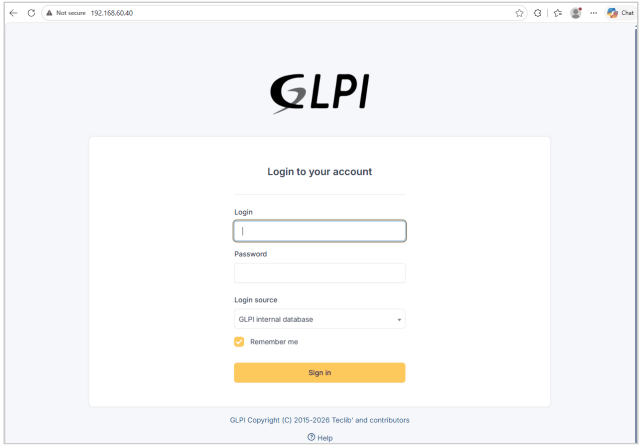
VMnet2 host adapter

📸 Evidence – Windows adapter VMnet2 set to 192.168.60.250:



ncpa VMnet2 IPv4

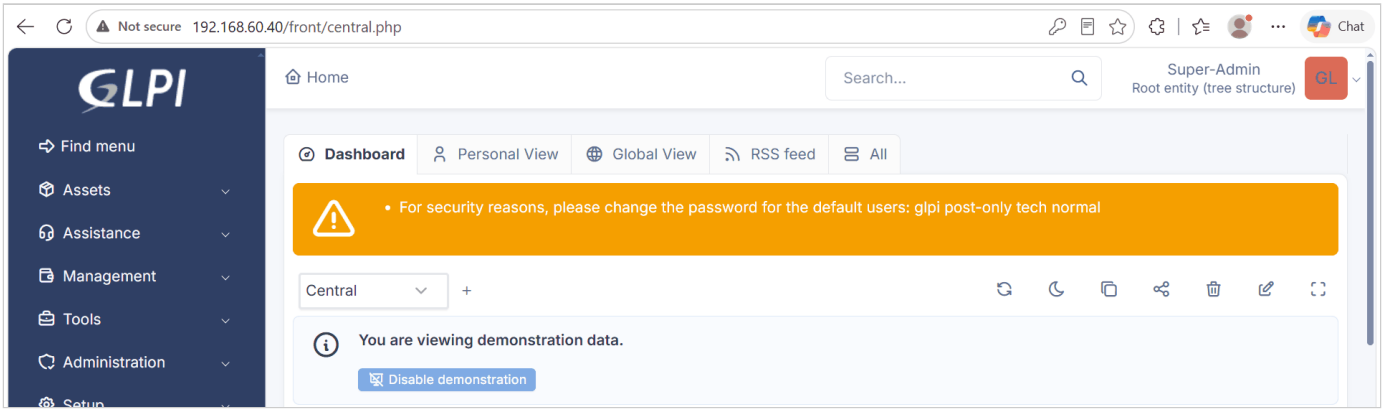
📸 Evidence – GLPI login page in the browser at http://192.168.60.40 (“Not secure” = plain HTTP, fine inside the lab):



GLPI login

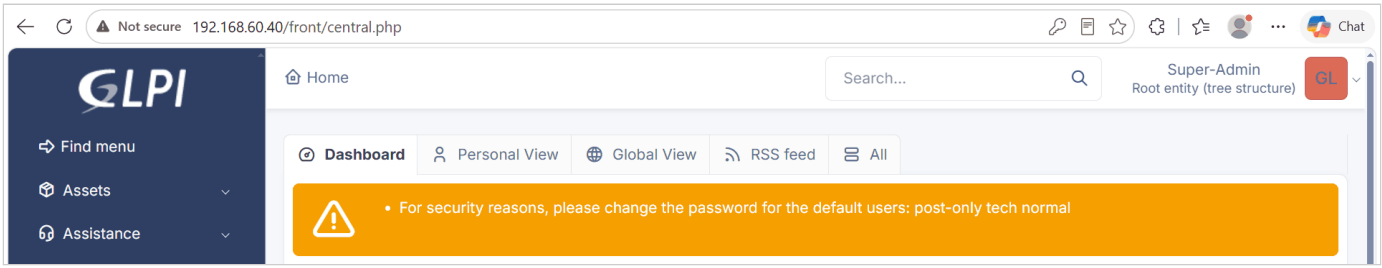
7.6 First login and default passwords

📸 Evidence – first login as Super-Admin with the warning (the numbers are GLPI demonstration data, not mine – "Disable demonstration" removes them):



first login

📸 Evidence – `glpi` no longer in the warning; `tech`, `normal`, `post-only` to be disabled next:



glpi password changed

Key understanding (in my own words)

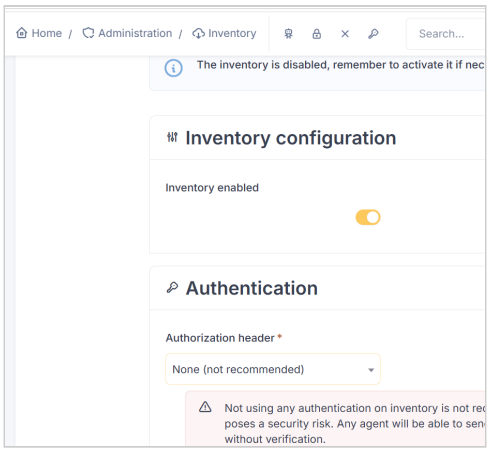
> GLPI runs on svc01 as two containers – the GLPI web application and its MySQL database – described in one `compose.yaml`. Their data is kept in folders on svc01, so it survives updates. The first start failed because GLPI runs inside the container as user 33 and could not write to a folder owned by root; the container log showed it, and `chown 33:33` fixed it. GLPI then answered `200 OK` on svc01 and from gw01. My Windows PC had no plug on the lab switch, so I added a VMnet2 host adapter with the free address 192.168.60.250 and opened GLPI in the browser. The default passwords are being changed first, because everybody knows them.

8. Step 1A.4 – GLPI agent: automatic asset inventory – 2026-10-17

💡 **In plain words:** a small program, the **GLPI agent**, now runs on every server. Once a day it collects what the machine is made of – hardware, operating system, serial number, installed software, network cards – and sends it to GLPI on svc01. So the asset list fills itself and stays up to date; nobody types it in. The agent is not a person and creates no tickets. (The guide puts Step 1A.4 after 1A.3; I did the inventory first because it only needs GLPI running.)

8.1 Switch the inventory on in GLPI

📸 Evidence – inventory enabled, no authentication (GLPI shows the warning itself):

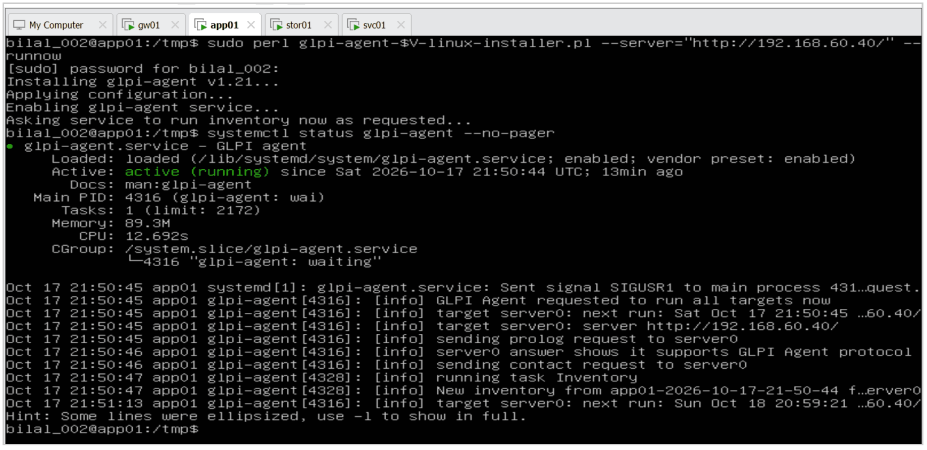


inventory enabled

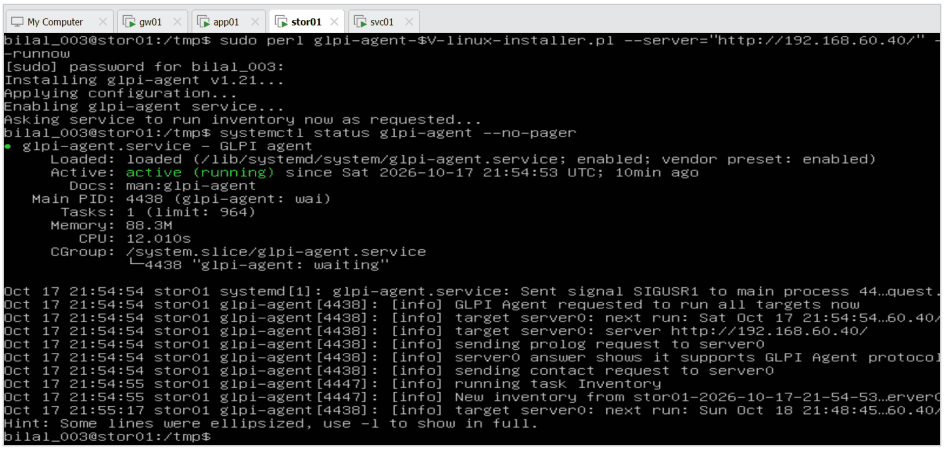
8.2 Install the agent on every server

```
cd /tmp && V=1.21                                          # agent version
wget https://github.com/glpi-project/glpi-agent/releases/download/$V/glpi-agent-$V-linux-installer.pl
sudo perl glpi-agent-$V-linux-installer.pl --server="http://192.168.60.40/" --runnow
#   --server = where GLPI lives (svc01)   --runnow = send the first inventory immediately
systemctl status glpi-agent --no-pager                     # service running, enabled at boot
```

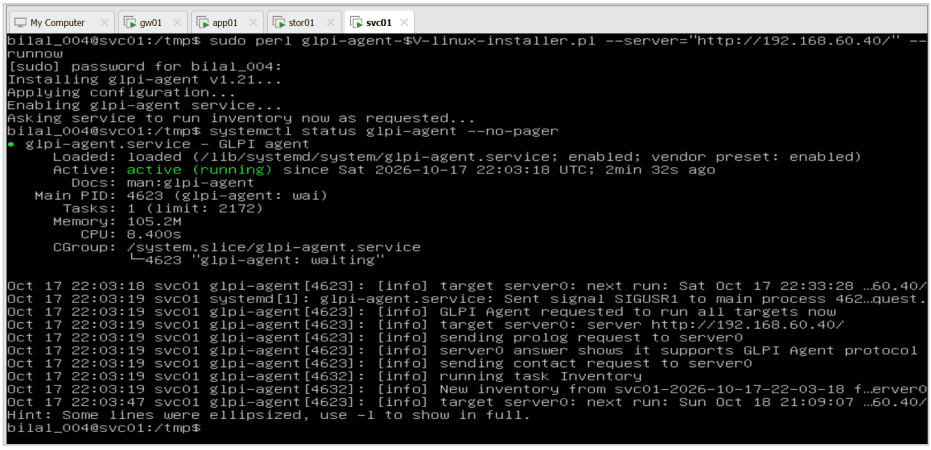
📸 Evidence – app01, stor01 and svc01 sent their first inventory:



app01 agent



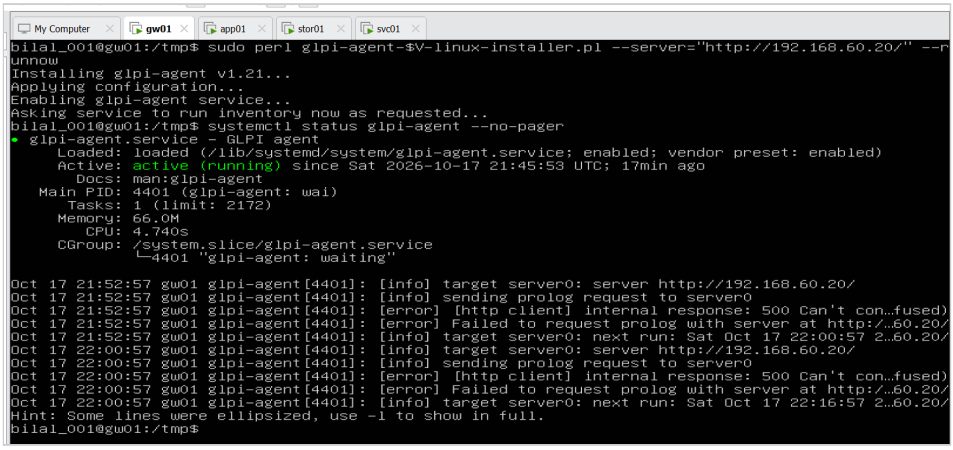
stor01 agent



svc01 agent

8.3 Issue – gw01 sent to the wrong address

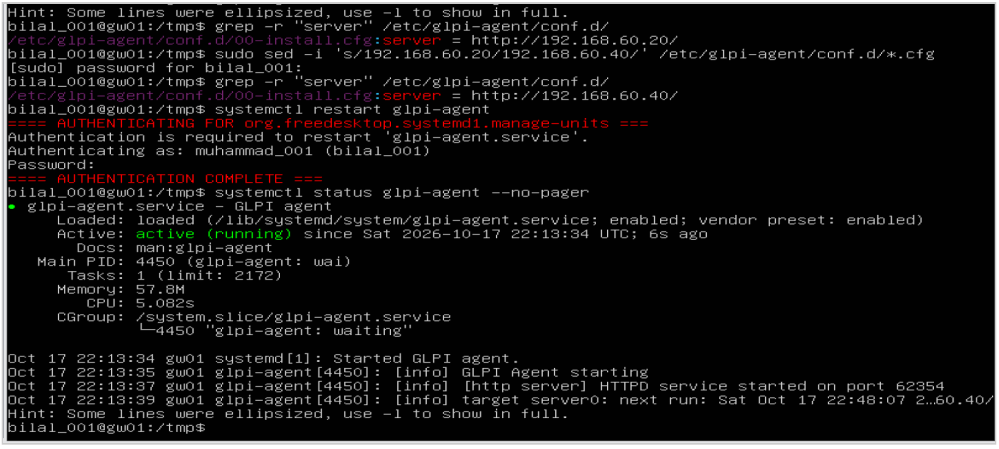
📸 Evidence – the error:



gw01 wrong server

```
        grep -r "server" /etc/glpi-agent/conf.d/                                    # shows server = http://
192.168.60.20/
        sudo sed -i 's/192.168.60.20/192.168.60.40/' /etc/glpi-agent/conf.d/*.cfg   # .20 → .40
        grep -r "server" /etc/glpi-agent/conf.d/                                    # check: now .40
        sudo systemctl restart glpi-agent                                           # load the new setting
        sudo glpi-agent --force                                                     # send the inventory now, don't wait
```

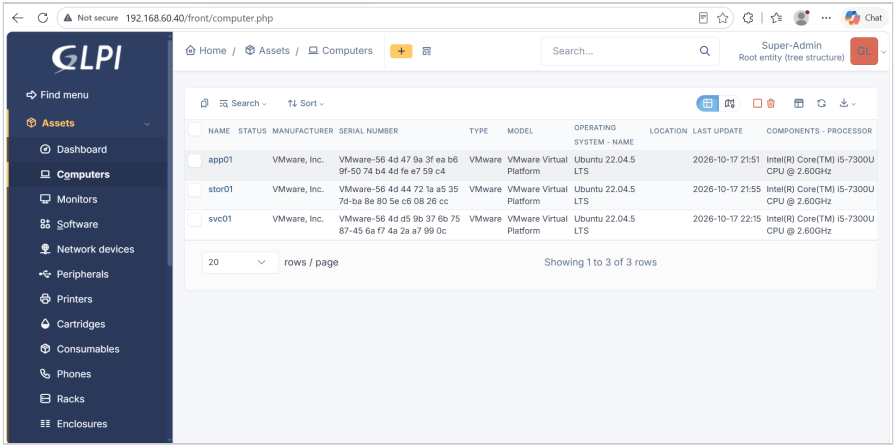
📸 Evidence – setting changed to .40, agent restarted:



gw01 fixed

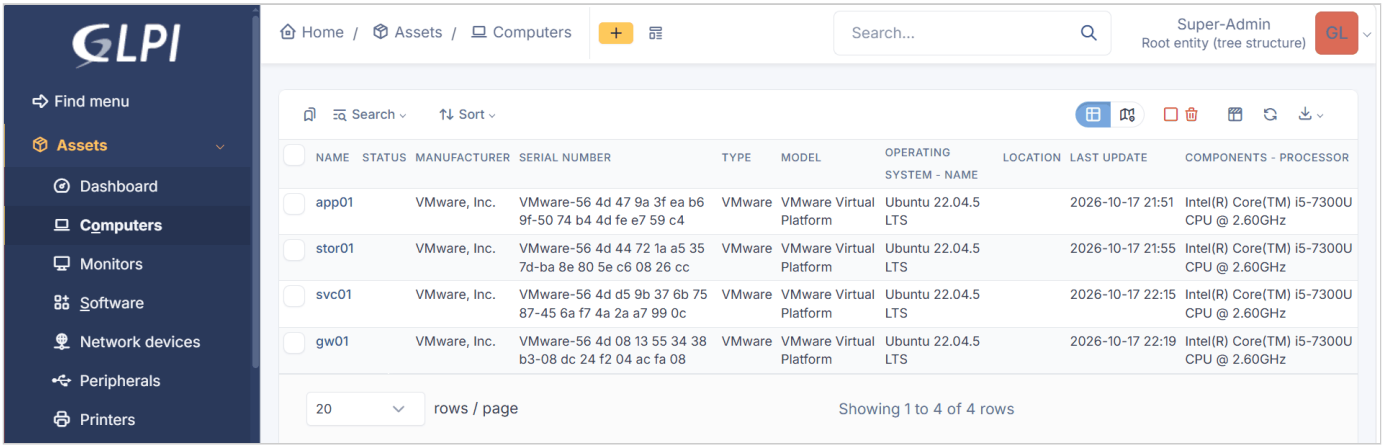
8.4 The result – Assets → Computers

📸 Evidence – first 3 of 4 (gw01 still missing):



3 computers

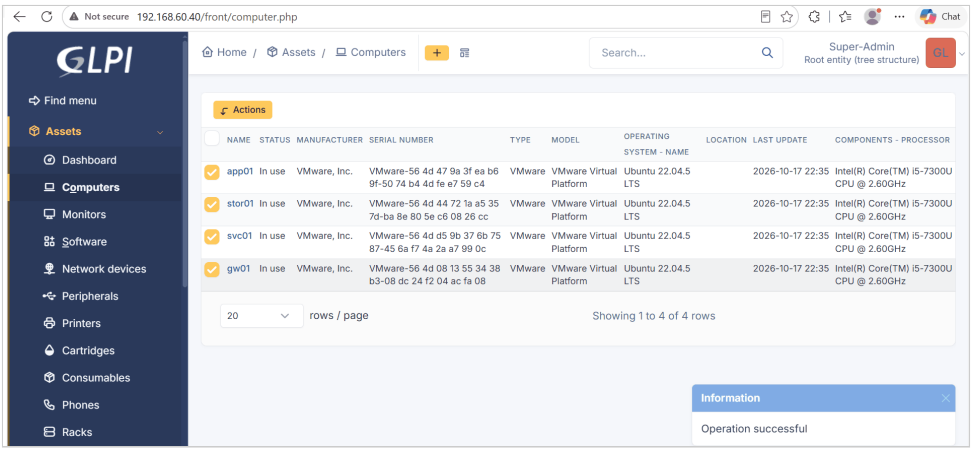
📸 Evidence – all 4 servers, after `glpi-agent --force` on gw01:



4 computers

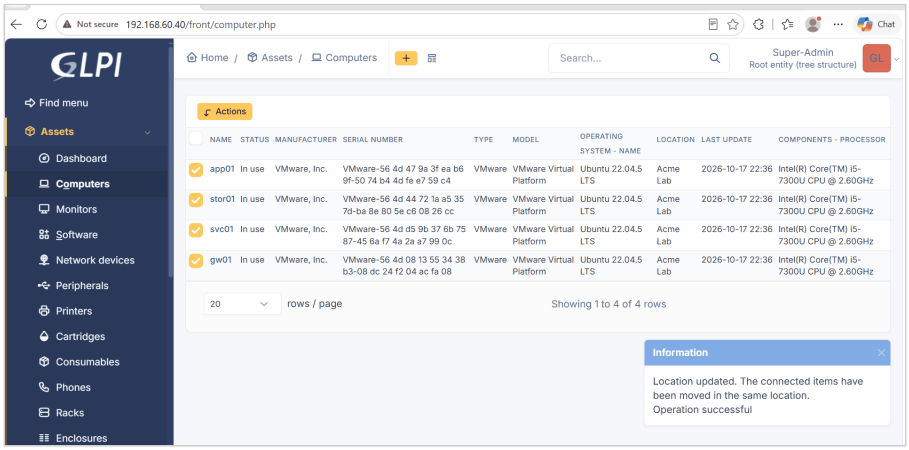
8.5 Status and location for all 4 at once

📸 Evidence – Status “In use” on all 4 (“Operation successful”):



status in use

📸 Evidence – Location “Acme Lab” on all 4:



location acme lab

Key understanding (in my own words)

The GLPI agent is a small program on each server. It reads the machine’s hardware, operating system, serial number and software and sends this to GLPI on svc01 at 192.168.60.40, about once a day. So the asset list fills itself and stays correct. I switched the inventory on in GLPI without authentication, which is fine inside the private lab; in Phase 6 I will add Basic Authentication. Three servers reported at once. gw01 failed with “connection refused” because I had typed .20 – that is stor01, which has no web server. The log showed the address, I corrected it in the agent’s settings file, and gw01 reported too. Now all four servers are listed. The agent fills the technical facts; I added the business facts – Status “In use” and Location “Acme Lab” – for all four at once with Actions → Update.

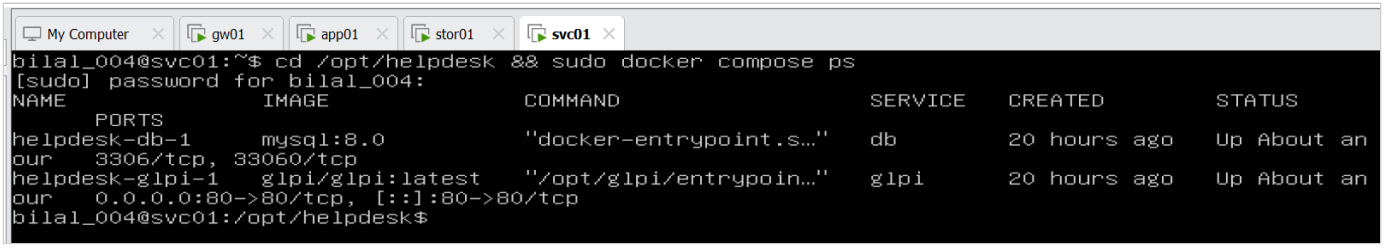
Day 4 – 18 Oct: GLPI wrap-up, LVM, NFS, backups, restore

1. Start of the day – does GLPI survive a full power-off?

💡 **In plain words:** last night all four VMs were shut down. Today they were started again, gw01 first (it is the router). GLPI came back by itself – nobody had to start it.

```
cd /opt/helpdesk && sudo docker compose ps     # both containers "Up"
```

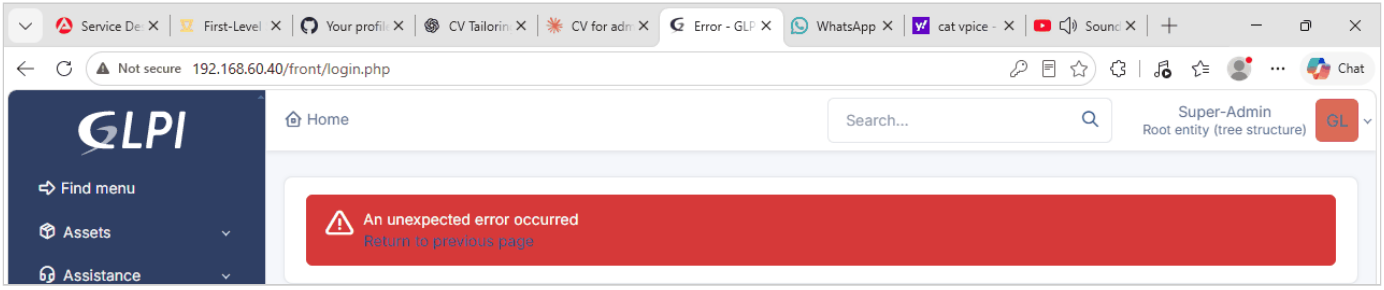
📸 **Evidence – containers created 20 hours ago, "Up about an hour" (since svc01 booted):**



containers back after reboot

Issue – "An unexpected error occurred"

📸 **Evidence:**

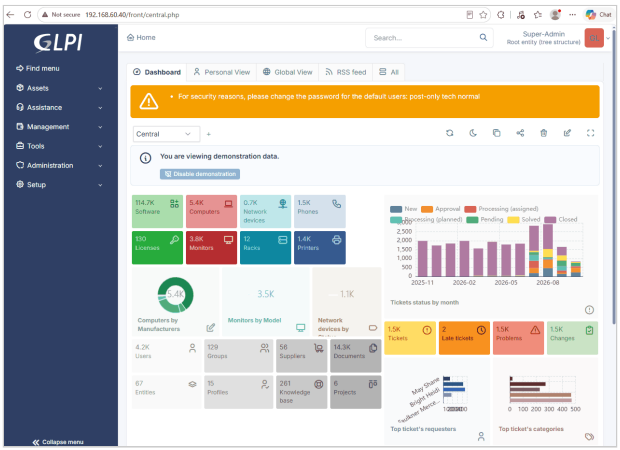


stale login tab

2. Clean-up – default accounts and demo data

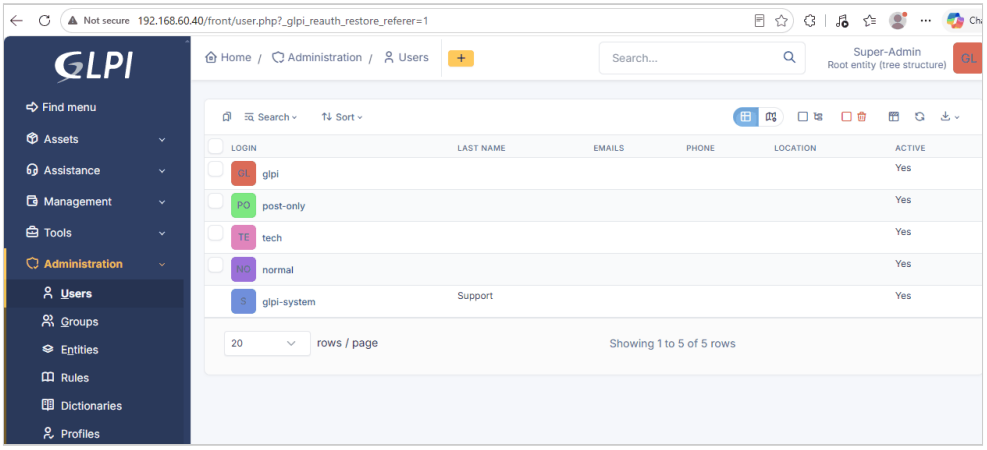
💡 **In plain words:** GLPI ships with four accounts whose passwords are printed in every guide on the internet. The `glpi` password was changed on Day 3; the other three are not needed, so they are switched off. The dashboard was also full of invented sample numbers.

📸 **Evidence – before: GLPI warns about the default passwords; "You are viewing demonstration data" (5.4K computers, 1.5K tickets – none of it real):**



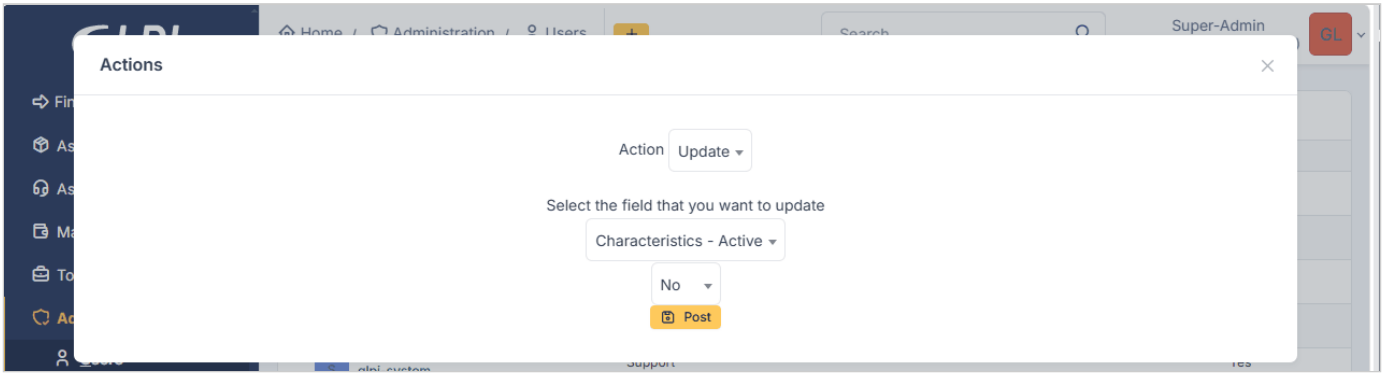
dashboard before

📸 **Evidence – before: all five accounts Active:**



users before

📸 **Evidence – the mass update:**

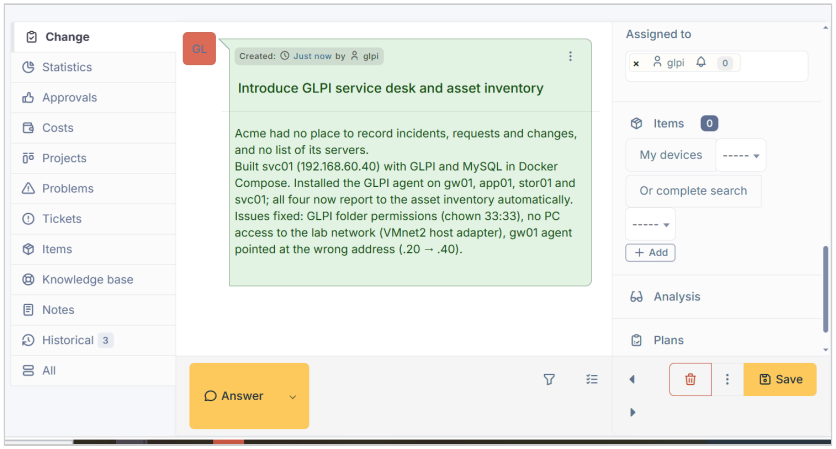


active no

3. The first ticket – Change #1

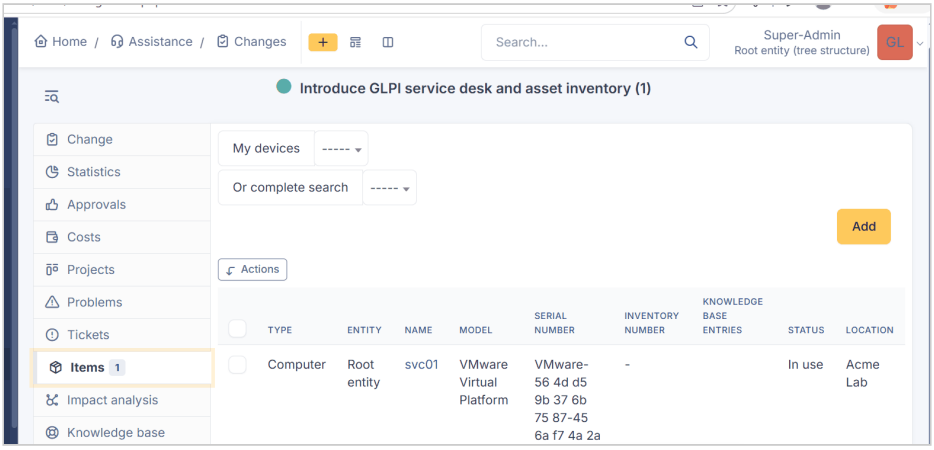
💡 **In plain words:** from now on, every piece of work is recorded in GLPI. The first record describes the service desk itself, written as a **Change**, because it was planned work with a way back.

📸 **Evidence – Change #1 created:**



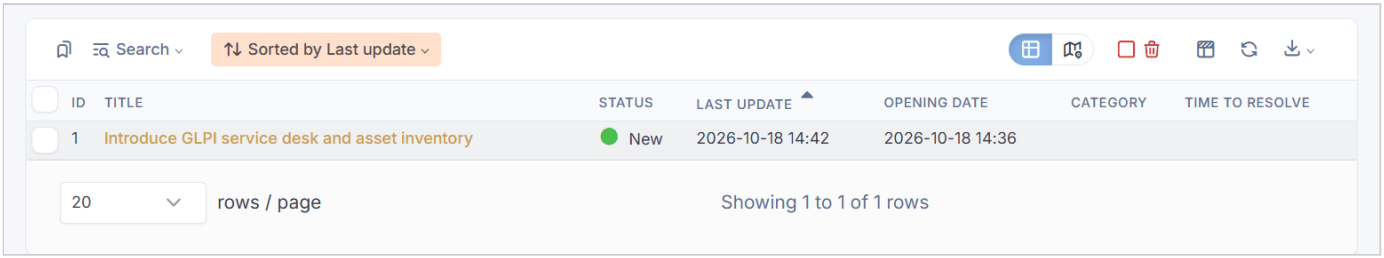
change created

📸 **Evidence – svc01 linked to the change (status In use, location Acme Lab – from the inventory):**



change item

📸 Evidence – Changes list, ID 1:



changes list

4. ITIL basics – three ticket types and the SLA

Key understanding (in my own words)

In GLPI every piece of work is recorded as one of three types: an **Incident** when something is broken, a **Request** when someone asks for something, and a **Change** for planned work. A change carries extra boxes – impact, deployment plan and backout plan – so another engineer can see what was changed and how to undo it. My first change records the service desk itself, linked to svc01.

An **SLA** (Service Level Agreement) is a written promise of how fast IT will react to a problem and fix it, depending on how serious it is. Every ticket gets a priority. For a high-priority problem, like the app being down, Acme promises that a technician takes the ticket within 1 hour (time to own) and fixes it within 8 hours (time to resolve); a medium problem gets 4 hours and 3 days. GLPI starts both clocks when the ticket is created and shows it as late when a promise is missed. So technicians know what to work on first and when to escalate, and managers can report how many tickets were solved on time. “Work according to SLAs” in a job ad means picking up tickets on time, handling the most urgent ones first, and escalating early before a deadline is missed.

5. Phase 2 – storage on stor01: LVM from empty disks

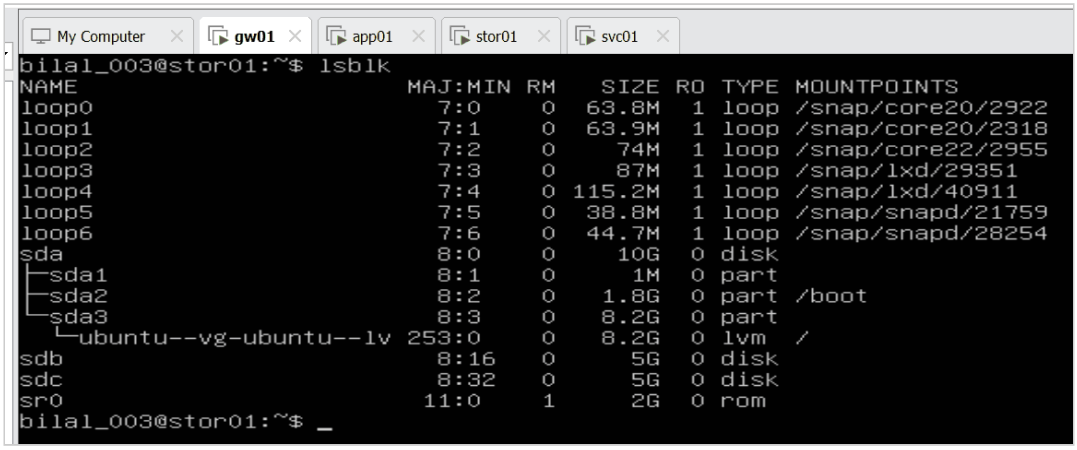
💡 **In plain words:** Acme’s application runs on **app01**, its data and backups live on **stor01**. stor01 needs storage that can **grow** when it fills up – without stopping anything. LVM gives exactly that: disks go into a pool, and volumes are cut from the pool and grown later.

```
sdb + sdc  →  pvcreate  →  PV, PV          (disks handed to LVM)
           →  vgcreate  →  VG "vgdata"     (one 10 GB pool)
           →  lvcreate  →  LV, LV          (slices: lvdata, lvbackups)
           →  mkfs.ext4 →  file system     (format)
           →  mount     →  /data, /backups (usable folders)
```

5.1 The disks

```
lsblk
```

📸 Evidence – sda = system disk (sda1/sda2 boot, sda3 = the system’s own LVM); sdb and sdc = two empty 5 GB disks; loop0–6 are snap images, sr0 the CD drive:

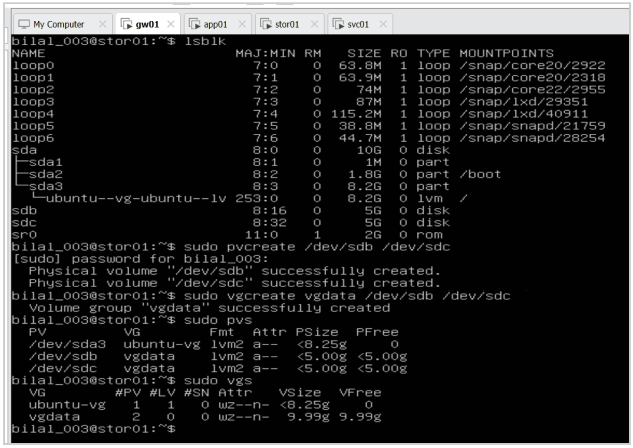


lsblk

5.2 Disks → physical volumes → volume group (the pool)

```
sudo pvcreate /dev/sdb /dev/sdc            # hand both disks to LVM (wipes them — check lsblk first)
sudo vgcreate vgdata /dev/sdb /dev/sdc     # one pool from both
sudo pvs                                   # the PVs
sudo vgs                                   # vgdata: 2 PVs, 0 LVs, 9.99 GB, all free
```

📸 Evidence:

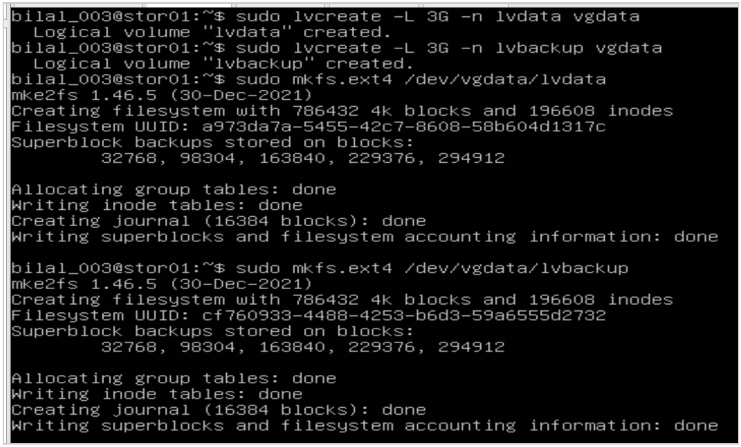


pvcreate vgcreate

5.3 Cut, format, mount

```
sudo lvcreate -L 3G -n lvdata vgdata       # 3 GB for app data
sudo lvcreate -L 3G -n lvbackup vgdata     # 3 GB for backups — about 4 GB stay free
sudo mkfs.ext4 /dev/vgdata/lvdata          # format
sudo mkfs.ext4 /dev/vgdata/lvbackup
```

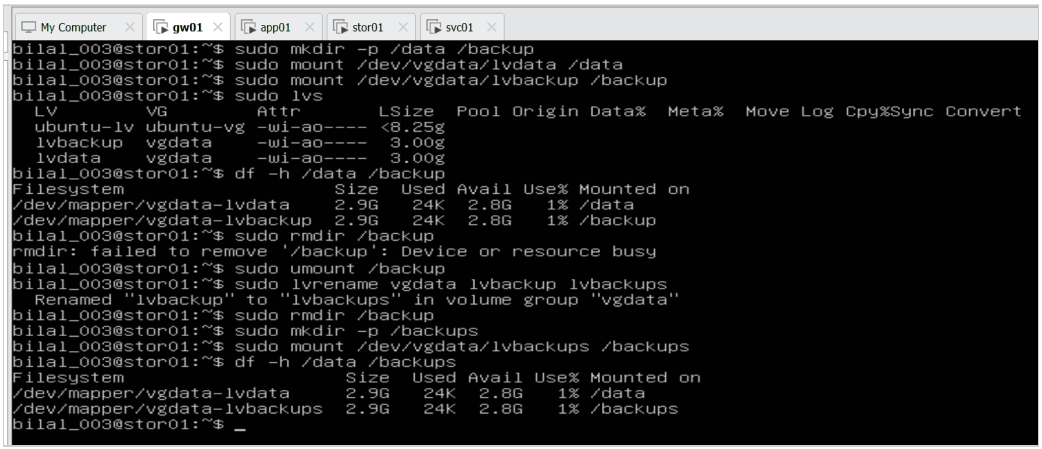
📸 Evidence – 786,432 blocks × 4 KB = 3 GB; 196,608 inodes = the most files this volume can hold; the UUID is its permanent ID:



lvcreate mkfs

```
        sudo umount /backup                             # rmdir before this failed: "Device or resource busy"
        sudo lvrename vgdata lvbackup lvbackups         # renaming does not change the UUID
        sudo rmdir /backup && sudo mkdir -p /backups
        sudo mount /dev/vgdata/lvbackups /backups
        df -h /data /backups
# 2.9 GB each (ext4 keeps room for its journal, inode tables and 5 % for root)
```

📸 **Evidence:**



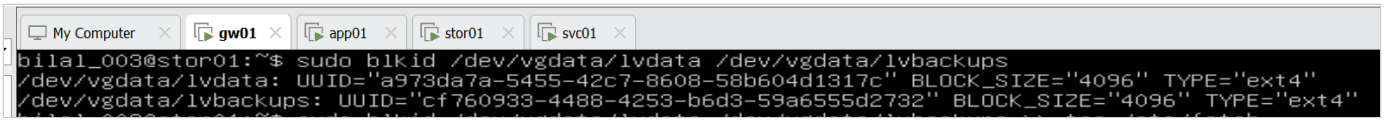
rename and mount

Lesson: a folder with a volume mounted on it is “busy” – `umount` first, then remove it. Separate volumes give isolation of space: full backups cannot fill the app’s `/data`.

5.4 Make the mounts permanent – `/etc/fstab`

```
        sudo blkid /dev/vgdata/lvdata /dev/vgdata/lvbackups      # the UUIDs
        sudo cp /etc/fstab /etc/fstab.bak                        # backup = way back
        echo 'UUID=a973da7a-5455-42c7-8608-58b604d1317c  /data     ext4  defaults  0 2' | sudo tee -a /etc/fstab
        echo 'UUID=cf760933-4488-4253-b6d3-59a6555d2732  /backups  ext4  defaults  0 2' | sudo tee -a /etc/fstab
```

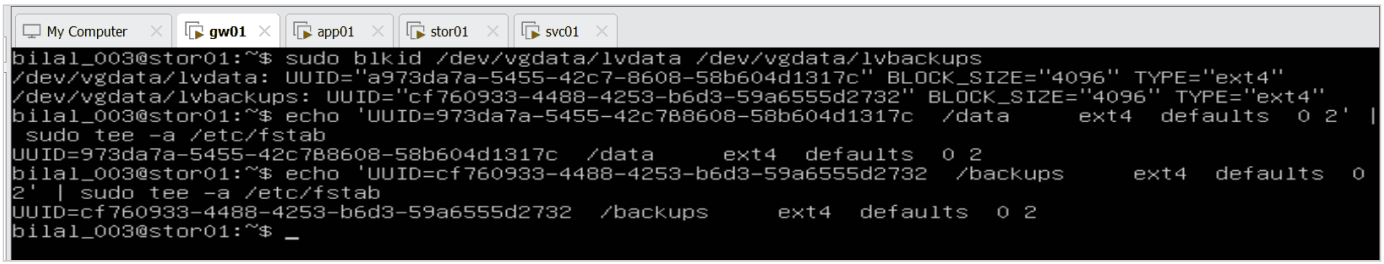
📸 **Evidence – the UUIDs:**



blkid

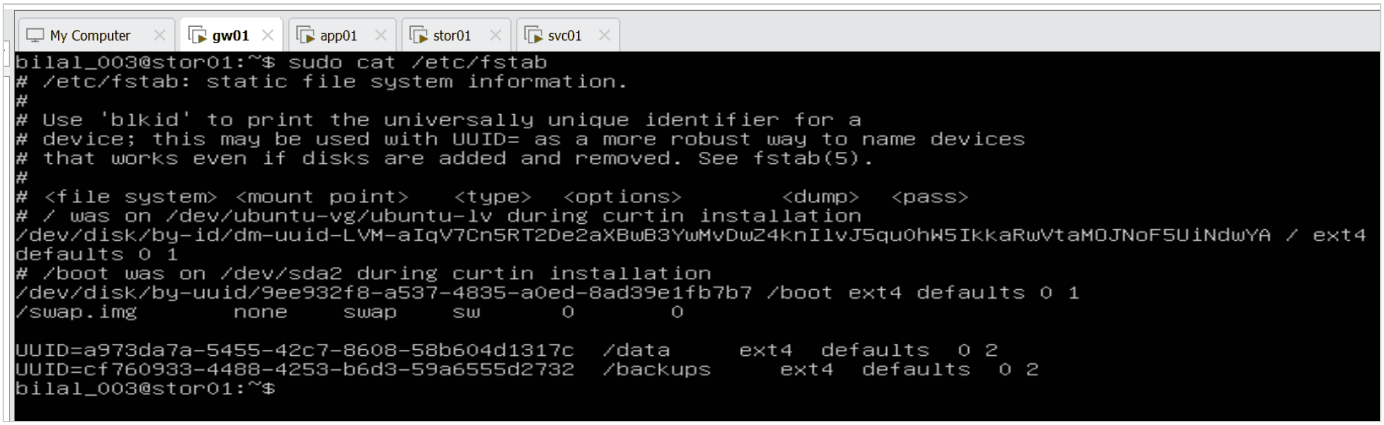
Issue – one wrong character in a UUID

📸 **Evidence – the wrong line:**



uuid typo

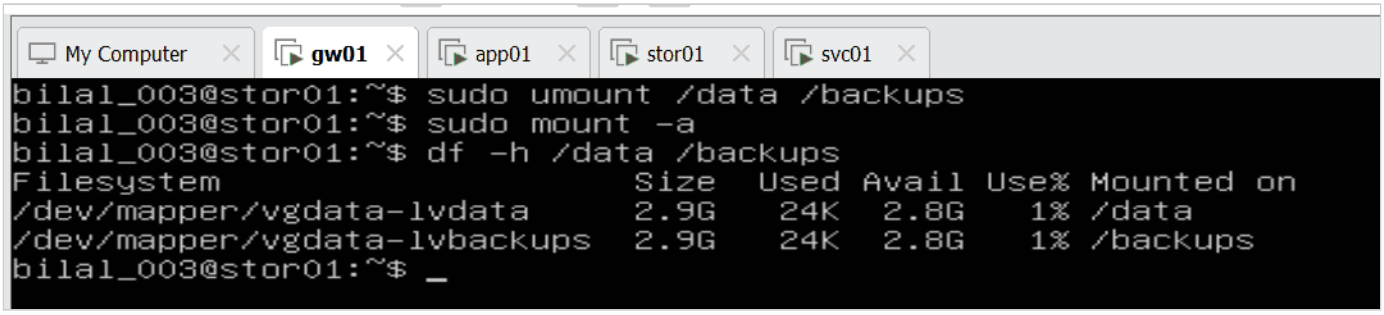
📸 **Evidence – fixed; both UUIDs match `blkid`:**



fstab fixed

```
        sudo umount /data /backups
        sudo mount -a              # no output = fstab is correct
        df -h /data /backups
```

📸 **Evidence – mounted again from fstab alone:**



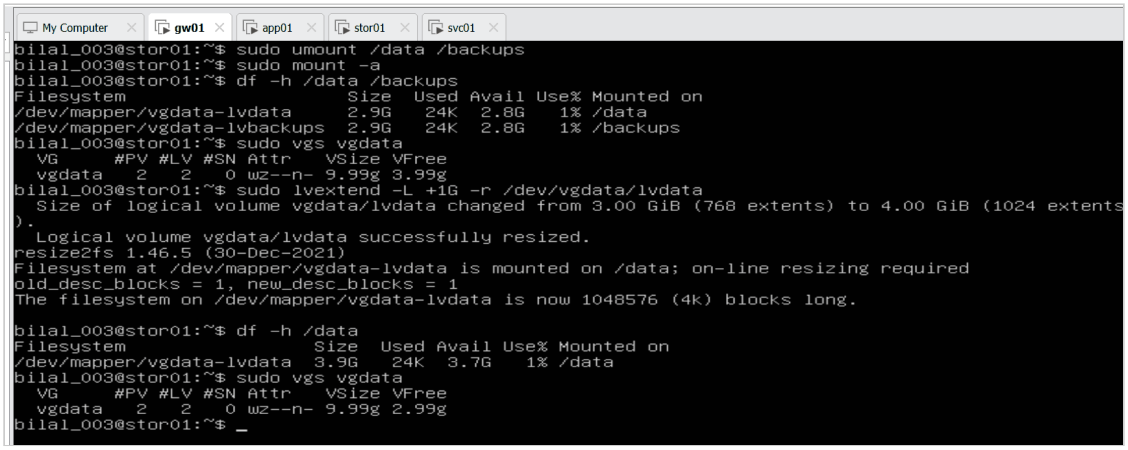
mount -a

5.5 Grow /data while it is running

💡 **The story:** a month later Acme's data has grown and `/data` is almost full. Without LVM: stop the app, add a disk, copy data. With LVM: take free space from the pool, live.

```
        sudo vgs vgdata                                 # VFree 3.99 GB
        sudo lvextend -L +1G -r /dev/vgdata/lvdata      # +1 GB; -r grows ext4 too
        df -h /data                                     # 3.9 GB, no unmount, no downtime
        sudo vgs vgdata                                 # VFree 2.99 GB
```

📸 **Evidence – 3.00 GiB (768 extents) → 4.00 GiB (1024 extents); "on-line resizing"; the pool's free space drops by 1 GB:**



lvextend

6. Phase 2 – NFS: app01 uses a folder on stor01

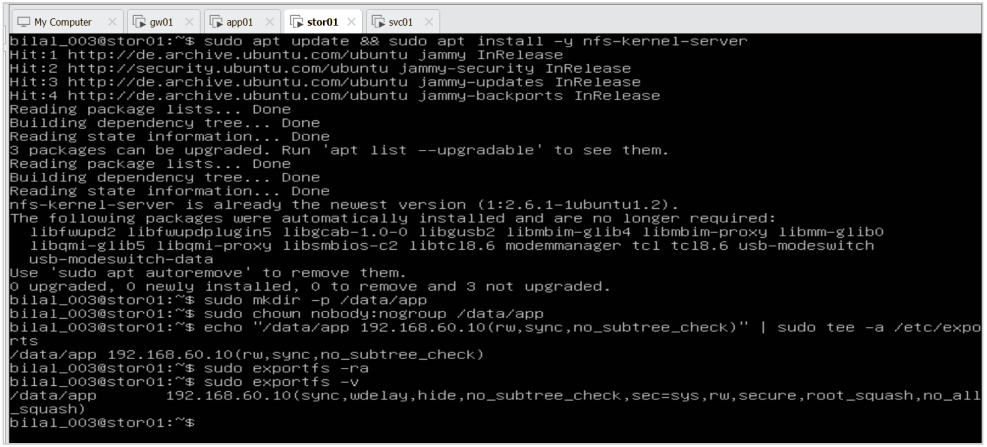
💡 **In plain words:** the app runs on app01, but its files should live on stor01, where the storage and backups are. **NFS** (Network File System) lets app01 use a stor01 folder over the network as if it were its own disk.

```
stor01 /data/app  ──NFS over the lab network──►  app01 /mnt/app
   (server: shares the folder)                  (client: mounts it)
```

6.1 stor01 – the server

```
sudo apt install -y nfs-kernel-server                          # already installed — nothing to download
sudo mkdir -p /data/app
sudo chown nobody:nogroup /data/app                            # neutral owner (see root_squash)
echo "/data/app 192.168.60.10(rw,sync,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra                                              # apply the share list
sudo exportfs -v                                               # what is shared, to whom
```

📸 Evidence – shared with app01 only:

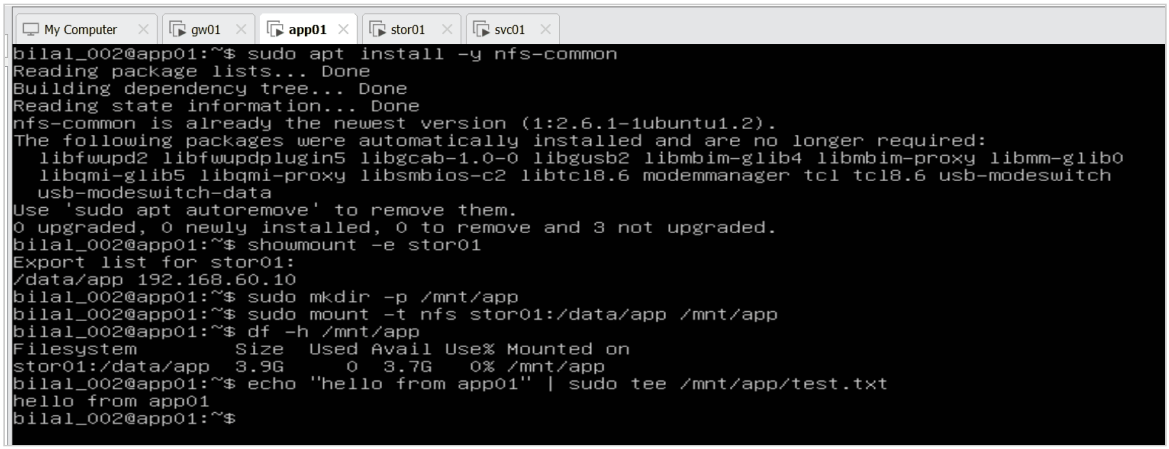


nfs server

6.2 app01 – the client, and the proof

```
sudo apt install -y nfs-common
showmount -e stor01                                    # what does stor01 share?
sudo mkdir -p /mnt/app
sudo mount -t nfs stor01:/data/app /mnt/app
df -h /mnt/app                                         # stor01:/data/app, 3.9 GB
echo "hello from app01" | sudo tee /mnt/app/test.txt   # write on app01
```

📸 Evidence – app01:



nfs client

```
cat /data/app/test.txt     # on stor01 — the same text
ls -l /data/app            # owner nobody:nogroup — root_squash at work
```

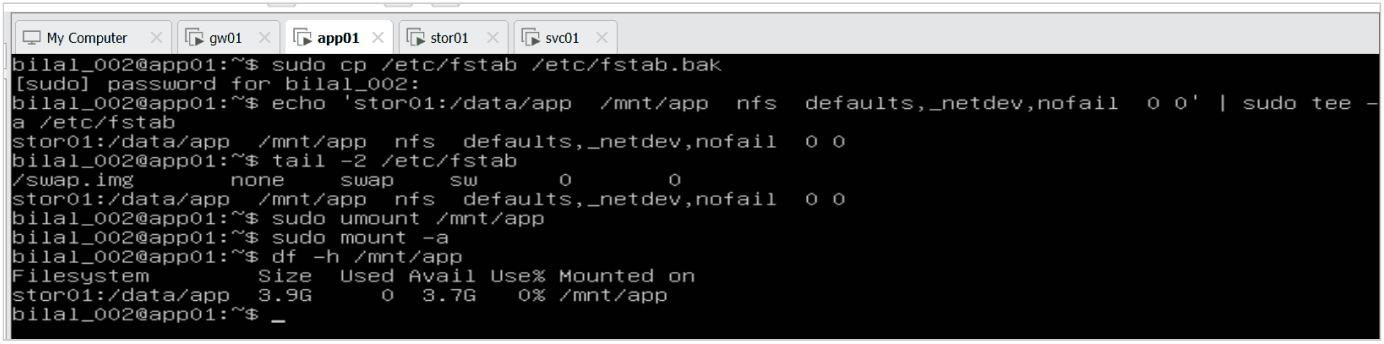
📸 Evidence – stor01 sees the file at once:

stor01 sees file

6.3 Permanent on app01

```
sudo cp /etc/fstab /etc/fstab.bak
echo 'stor01:/data/app  /mnt/app  nfs  defaults,_netdev,nofail  0 0' | sudo tee -a /etc/fstab
sudo umount /mnt/app
sudo mount -a
df -h /mnt/app
```

📸 **Evidence – mounted from fstab:**



app01 fstab

Lesson: a server must never fail to boot because **another** server is down.

Key understanding (in my own words)

> Acme's application runs on app01, and its data and backups live on stor01. stor01 needed storage that can **grow** and an easy way to **share** files with app01.
>
> **LVM.** `lsblk` showed two empty 5 GB disks, sdb and sdc. A raw disk cannot be used directly, so I built LVM layer by layer: `pvcreate` made both disks **physical volumes**; `vgcreate` put them into one **volume group**, `vgdata` – a 10 GB **pool** (a VG can be built from one or many PVs); `lvcreate` cut two 3 GB **logical volumes** from the pool, `lvdata` and `lvbackups`, so full backups cannot fill the app's space; `mkfs.ext4` formatted them and `mount` attached them to `/data` and `/backups`. Mounts disappear after a reboot, so I backed up `/etc/fstab` and added each volume's **UUID** from `blkid`. One wrong character in a UUID taught me to always test with `mount -a` before a reboot. Finally `lvextend -L +1G -r` grew `/data` from 3 to 4 GB while it was mounted – the main reason LVM exists.
>
> **NFS.** On stor01 I installed the NFS server, created `/data/app` owned by `nobody:nogroup`, and shared it in `/etc/exports` with **only app01**, read/write. `root_squash` treats root from app01 as "nobody" on stor01. On app01 I installed the NFS client, checked the share with `showmount -e stor01` and mounted it on `/mnt/app`; a file written on app01 appeared at once on stor01. In app01's fstab I used `_netdev` (wait for the network) and `nofail` (boot even if stor01 is down).
>
> **In one line:** stor01 now has storage that grows without downtime (LVM) and shares it with app01 over the network (NFS).

7. Phase 2 – backups: archive, mirror, versions, restore

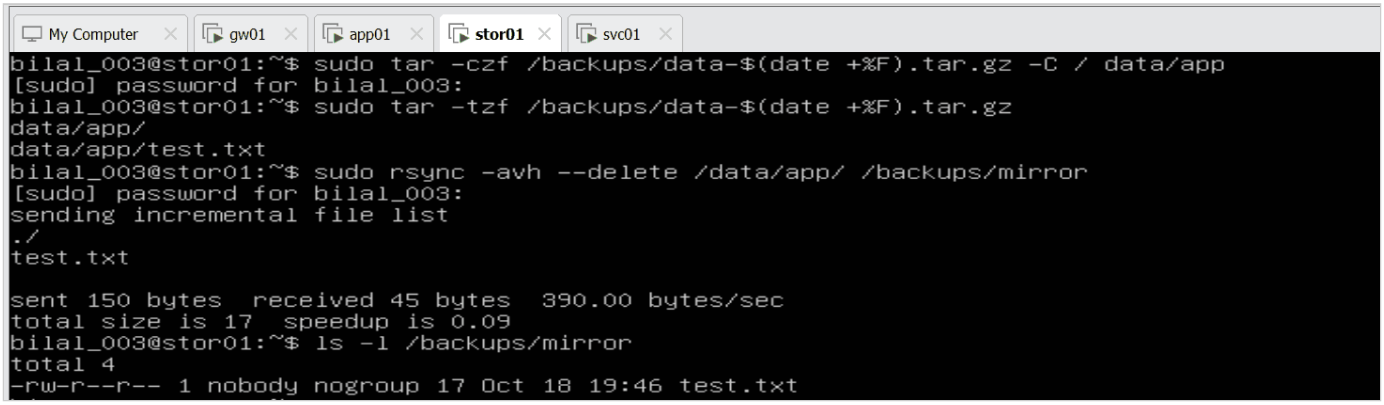
💡 **In plain words:** a backup is a copy kept somewhere safe, so a lost file can come back. I made three kinds, all in `/backups` on stor01 (the `lvbackups` volume), and then proved they work by restoring a deleted file – through a real ticket.

```
stor01
/data/app/                      ← the original (shared with app01 over NFS)
/backups/
├── data-2026-10-18.tar.gz      ← ARCHIVE: one compressed file per day          (file)
├── mirror/                     ← MIRROR: exact copy of /data/app, only "now"   (real folder)
├── v1/  v2/  v3/  2026-10-19/  ← VERSIONS: one folder per run, history        (real folders)
└── latest -> 2026-10-19        ← shortcut to the newest version               (soft link)
```

7.1 Archive (tar) and mirror (rsync)

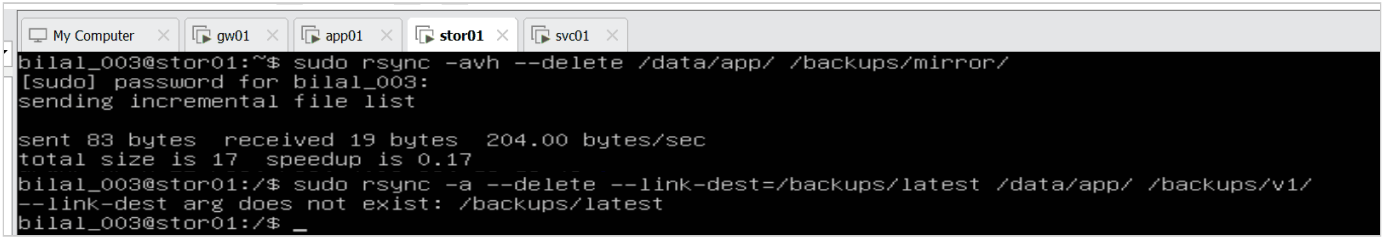
```
sudo tar -czf /backups/data-$(date +%F).tar.gz -C / data/app   # pack /data/app into ONE dated, compressed file
tar -tzf /backups/data-$(date +%F).tar.gz                      # LIST what is inside — nothing is unpacked
sudo mkdir -p /backups/mirror                                   # a new, empty real folder
sudo rsync -avh --delete /data/app/ /backups/mirror/            # make it an exact copy of /data/app
```

📸 **Evidence – the archive lists `data/app/test.txt`; rsync copies `test.txt`, keeping owner `nobody` and the original time:**



tar and mirror

📸 **Evidence – run again with nothing changed: rsync copies nothing (only "sent 83 bytes"). Then the first versioned run warns `--link-dest arg does not exist` – normal, there is no previous version yet:**

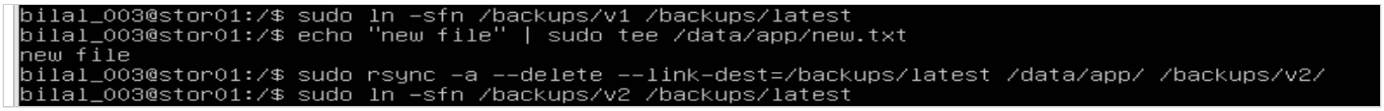


mirror rerun

7.2 Versions with hard links – history without wasting space

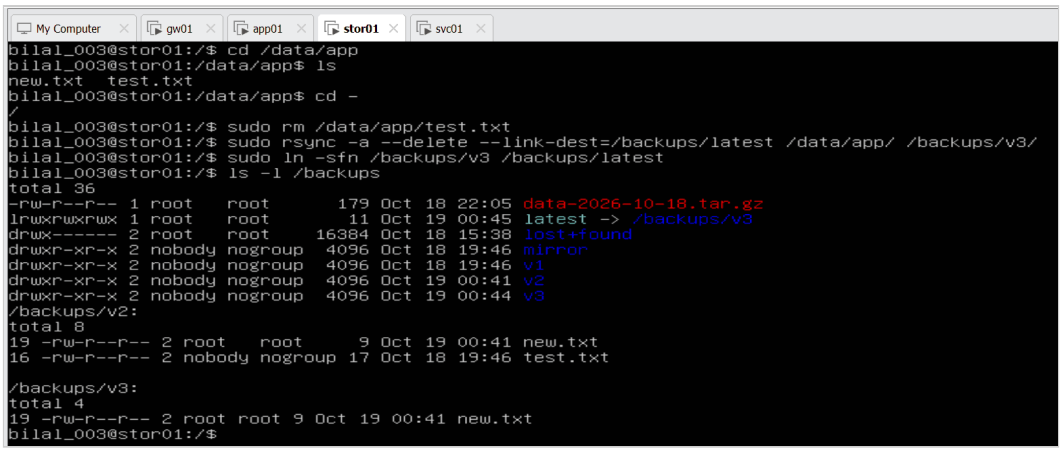
```
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v1/   # FROM /data/app  TO v1  COMPARE
WITH latest
        sudo ln -sfn /backups/v1 /backups/latest                                     # latest → v1
        echo "new file" | sudo tee /data/app/new.txt                                 # a change
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v2/
        sudo ln -sfn /backups/v2 /backups/latest                                     # latest → v2
        sudo rm /data/app/test.txt                                                   # the "accident"
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v3/
        sudo ln -sfn /backups/v3 /backups/latest                                     # latest → v3
        ls -l /backups
        ls -li /backups/v1 /backups/v2 /backups/v3
```

📸 **Evidence – v1, then a new file, then v2:**



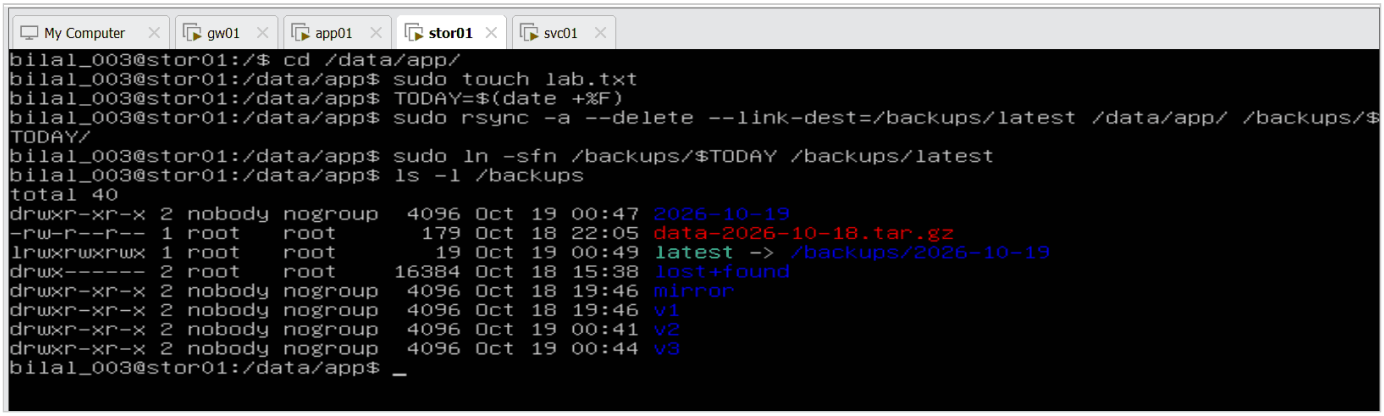
v1 v2

📸 **Evidence – after deleting test.txt, v3; `latest -> /backups/v3`; in v2 `test.txt` has inode 16 with link count 2 (shared with v1 – one copy on disk), `new.txt` inode 19 count 2 (shared with v3); v3 has no test.txt, but** v1 and v2 still do**:**



versions

📸 **Evidence – a real dated version, `/backups/2026-10-19`, and `latest` moved to it:**

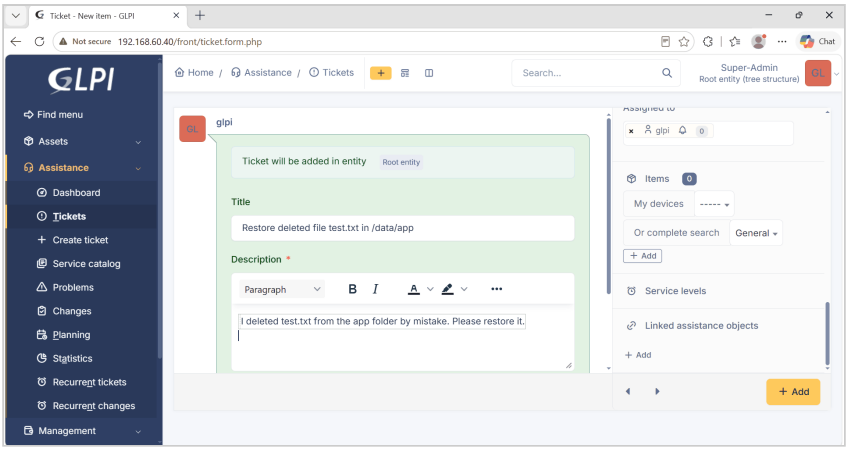


dated backup

7.3 Restore drill – through a ticket

💡 **The story:** Alice deletes `test.txt` by mistake and opens a ticket. Normal employees cannot touch `/backups` – it belongs to root and is handled by IT. The technician picks the version taken **before** the deletion and restores it.

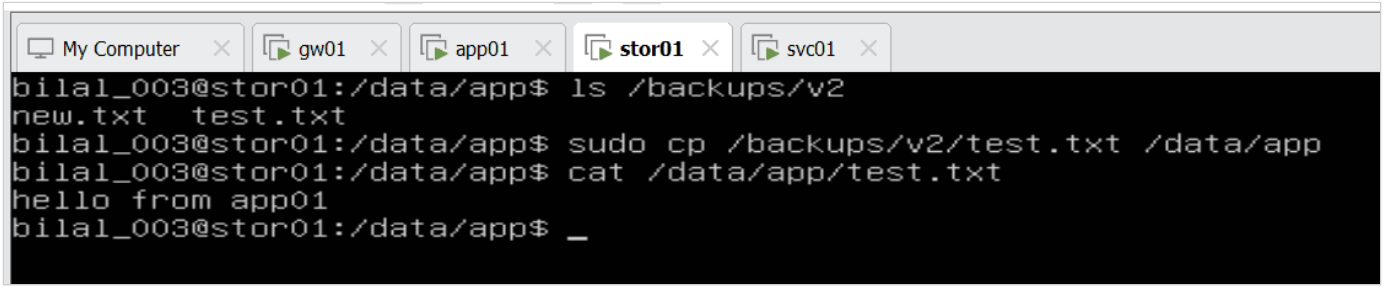
📸 **Evidence – Request ticket created (category *Storage & Backup*):**



ticket

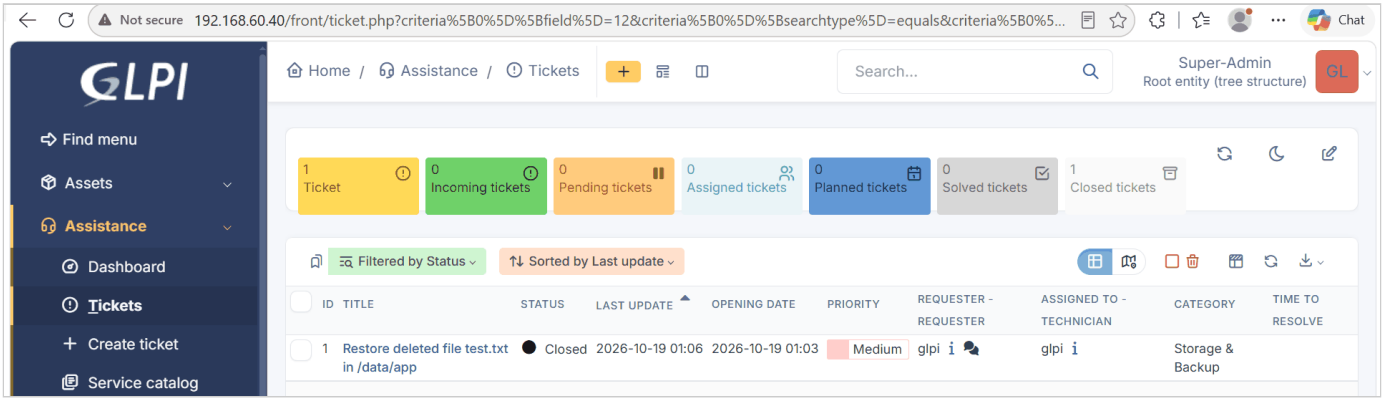
```
ls /backups/v2                          # v3 no longer has the file — v2 does
sudo cp /backups/v2/test.txt /data/app  # restore: OUT of the backup, BACK to /data/app
cat /data/app/test.txt                  # "hello from app01"
```

📸 **Evidence – restored and readable:**



restore

📸 **Evidence – ticket #1, Request, Storage & Backup,** Closed**:**



ticket closed

Key understanding (in my own words)

In `/backups` I keep three kinds of backup. The **archive** (`tar`) is one compressed **file** per day, for example `data-2026-10-18.tar.gz`; `tar -tzf` only lists what is inside. The **mirror**, `/backups/mirror`, is a **real folder**: `rsync -avh --delete` fills it with independent copies of `/data/app`. It is not live – it is updated only when rsync runs, and then copies only what is new or changed, like a patch; with `--delete`, files removed from the original are also removed from the mirror, so it only knows “now”.

For **history**, each rsync run creates a **new version folder** (v1, v2, v3, or one per date). It takes the files **from** `/data/app`, writes them **to** the new folder, and **compares with** `/backups/latest`, the previous version. Changed or new files are really copied; unchanged files become **hard links** to the previous version, so every version looks complete but only the changes use space. Old versions are never changed. `/backups/latest` is the only **shortcut** (soft link); after each run `ln -sfn` moves it to the newest version, so the next run always compares with the most recent one.

A backup only counts once it has been restored. Alice deleted `test.txt` by mistake and opened a ticket. Normal employees cannot touch `/backups` – IT handles it at the administrative level. As the technician I found that v3, taken after the deletion, no longer had the file, but v2 did. I restored it with `sudo cp /backups/v2/test.txt /data/app/`, checked it with `cat`, wrote the solution in the ticket and closed it. The drill proves that the backups hold real files, that I can pick the right version, and that a lost file comes back working.

🗒️ All the files that make changes survive a reboot in this project: persistence cheat sheet.

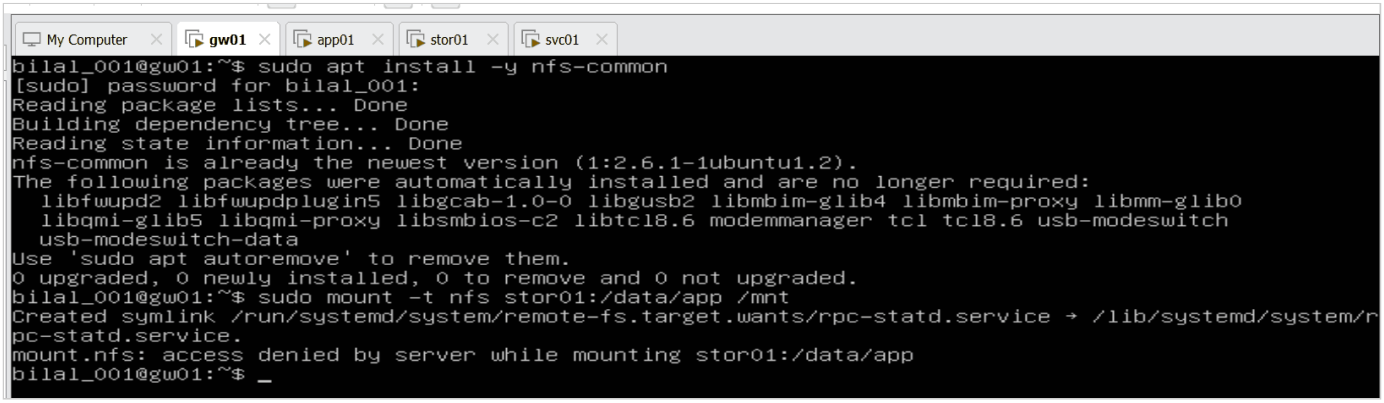
Day 5 – 20 Oct: snapshot rollback, job control, systemd timer and socket

1. Break test – reachable is not allowed (Step 2.2)

💡 **In plain words:** gw01 has stor01 in `/etc/hosts` and can ping it – but it is not listed in stor01's `/etc/exports`. Can it use the share anyway?

```
        # on gw01
        sudo apt install -y nfs-common                  # NFS client tools (already installed)
        sudo mount -t nfs stor01:/data/app /mnt         # /mnt = standard empty folder for temporary mounts
```

📸 **Evidence – `access denied by server`:**



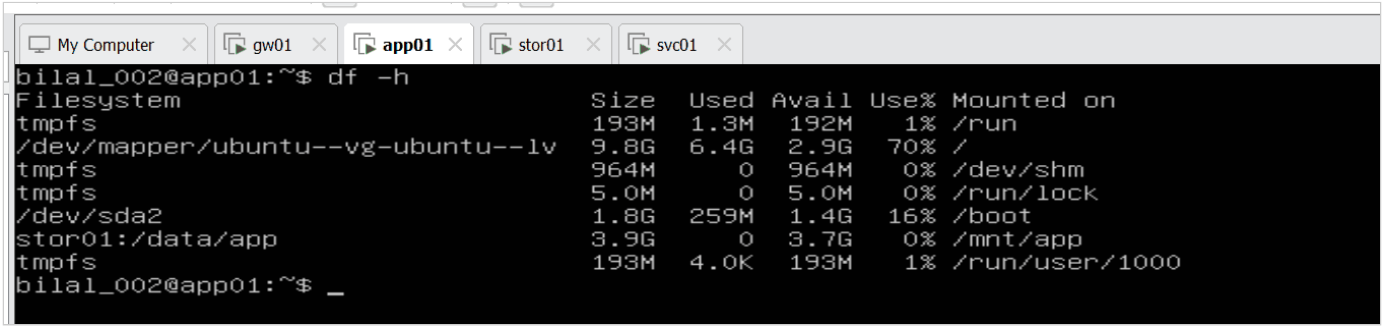
access denied

Lesson: being able to reach a server is not permission to use its shares. Only the IPs listed in `/etc/exports` (here only app01, 192.168.60.10) may mount `/data/app`.

2. Where the app's data really lives

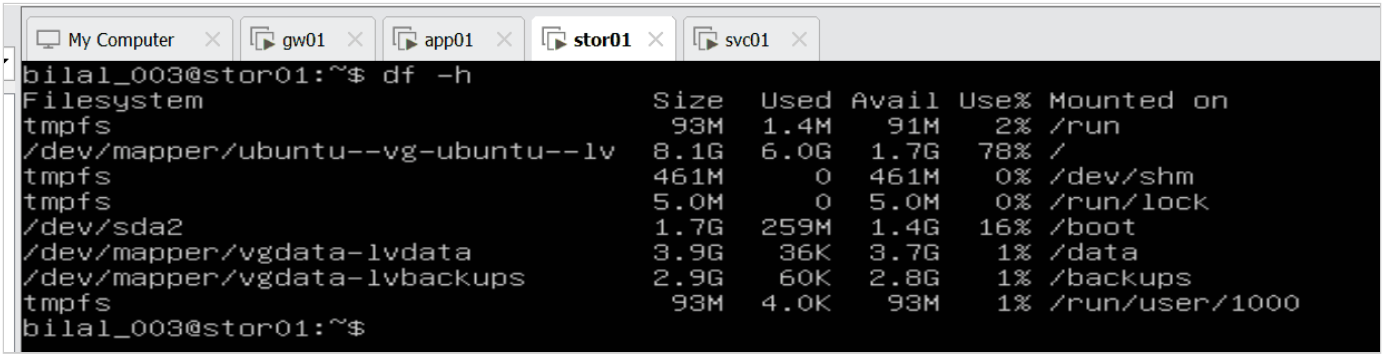
```
app01: /mnt/app  ═ NFS ═▶  stor01: /data/app  ─inside─▶  /data   ─mounted from─▶  lvdata (logical volume)
                                                          /backups ─mounted from─▶  lvbackups
```

📸 **Evidence – app01: `stor01:/data/app` on `/mnt/app`:**



app01 df

📸 **Evidence – stor01: `lvdata` on `/data`, `lvbackups` on `/backups`:**



stor01 df

3. LVM snapshot – the undo button (Step 2.4)

💡 **The story:** a risky change is coming to `/data` (an update, a cleanup). A mistake made from app01 lands directly on `lvdata`, and the rsync mirror – which keeps only the current state – would copy it at its next run. The dated versions still hold good files, but restoring file by file is slow. So **before** the change: take a snapshot – a frozen state of the whole volume – and roll back in seconds if it goes wrong.

3.1 Take the snapshot, make the “mistakes”

```
# on app01 — release the share first (see the issue below)
sudo umount /mnt/app

# on stor01
ls /data/app                                              # lab.txt new.txt test.txt
sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata        # -s snapshot of lvdata; 1 GB room for changes
sudo lvs
echo "bad change" | sudo tee /data/app/oops.txt           # mistake 1
sudo rm /data/app/test.txt                                # mistake 2
ls /data/app                                              # lab.txt new.txt oops.txt
```

📸 **Evidence:**

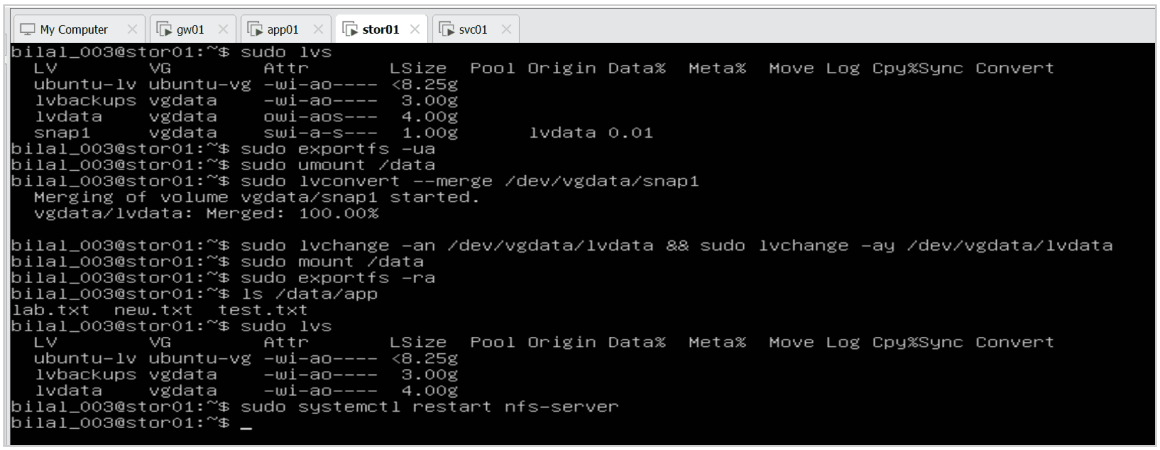
```
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  test.txt
bilal_003@stor01:~$ sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata
[sudo] password for bilal_003:
  Logical volume "snap1" created.
bilal_003@stor01:~$ sudo lvs
  LV        VG         Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  ubuntu-lv ubuntu-vg -wi-ao---- <8.25g
  lvbackups vgdata     -wi-ao----  3.00g
  lvdata    vgdata     owi-aos---  4.00g
  snap1     vgdata     swi-a-s---  1.00g       lvdata 0.01
bilal_003@stor01:~$ echo "bad change" | sudo tee /data/app/oops.txt
bad change
bilal_003@stor01:~$ sudo rm /data/app/test.txt
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  oops.txt
bilal_003@stor01:~$ _
```

snapshot taken

3.2 Roll back

```
# on stor01
sudo exportfs -ua                                         # stop sharing /data/app
sudo umount /data                                         # nobody may use the volume during the merge
sudo lvconvert --merge /dev/vgdata/snap1                  # back to the frozen state
sudo lvchange -an /dev/vgdata/lvdata && sudo lvchange -ay /dev/vgdata/lvdata   # deactivate / activate
sudo mount /data
sudo exportfs -ra                                         # share again
ls /data/app                                              # lab.txt new.txt test.txt — oops.txt gone
sudo lvs                                                  # snap1 gone — merged into lvdata
```

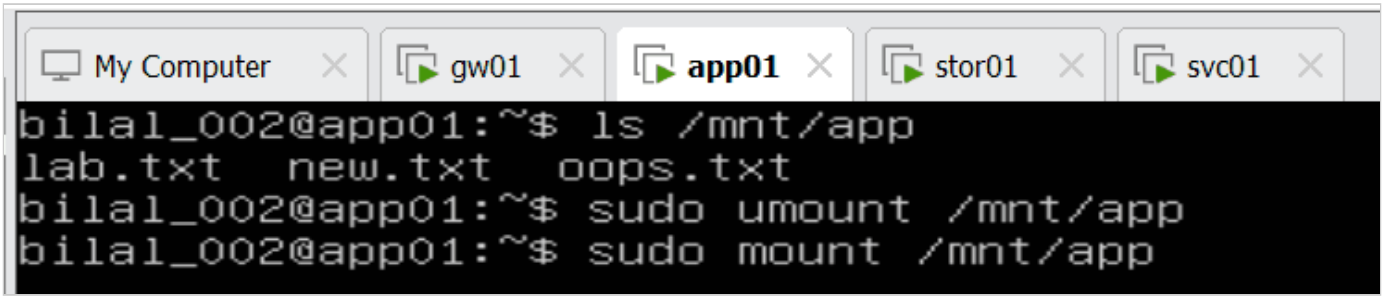
📸 **Evidence – `Merged: 100.00%`; test.txt back, oops.txt gone; snap1 gone:**



rollback

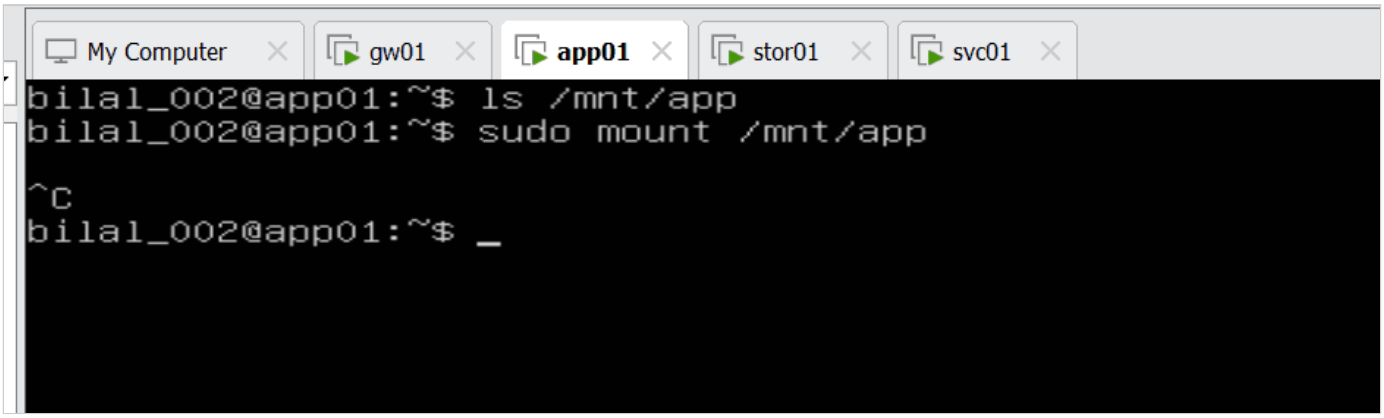
3.3 Issue – app01 hangs after the rollback (stale NFS mount)

📸 **Evidence – app01 still mounted during the rollback (oops.txt visible):**



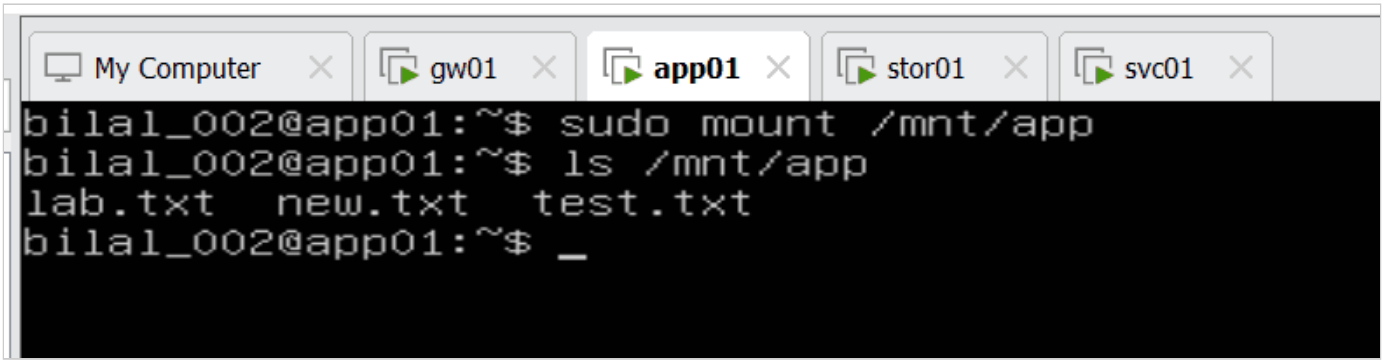
still mounted

📸 **Evidence – the mount hangs:**



hang

📸 **Evidence – after the fix, app01 sees the rolled-back files:**



fixed

5. VMware snapshots

Key understanding (in my own words)

> On app01, `/mnt/app` is a folder. On stor01, `/data/app` is a folder. They are connected over NFS, so everything done in `/mnt/app` on app01 really happens in `/data/app` on stor01. On stor01, the logical volume `lvdata` is mounted on `/data`, so every file in `/data`, including `/data/app`, is stored on `lvdata`. The work happens on `lvdata`; the backups are stored on another logical volume, `lvbackups`, mounted on `/backups`.
>
> If someone on app01 makes a wrong or risky change in `/mnt/app`, the mistake lands directly on `lvdata`. The rsync mirror only keeps the current state, so at its next run it would copy the mistake too. A backup is a set of separate copies of the files on `lvbackups`: from it I can restore files one by one, from any day, but that is slow for a big mistake.
>
> So before a risky change I take an **LVM snapshot**: a frozen state of the whole `lvdata` volume, taken at the moment I create it. If the change goes wrong, I roll back the whole volume at once. To roll back, nobody may be using the volume: first app01 unmounts `/mnt/app`, then stor01 stops sharing (`exportfs -ua`) and unmounts `/data`. I learned this the hard way – app01 was still mounted during my rollback, was left with a stale mount and hung, until I restarted the NFS server and force-unmounted the client.
>
> Both survive a reboot, but a snapshot lives on the same disks and becomes invalid if its room fills up: **snapshot = quick undo, short-term, same disks; backup = recovery, long-term, separate volume.** And gw01 showed that being able to reach a server is not permission to use its shares.

6. Phase 3 starts – automation (Step 3.1: job control)

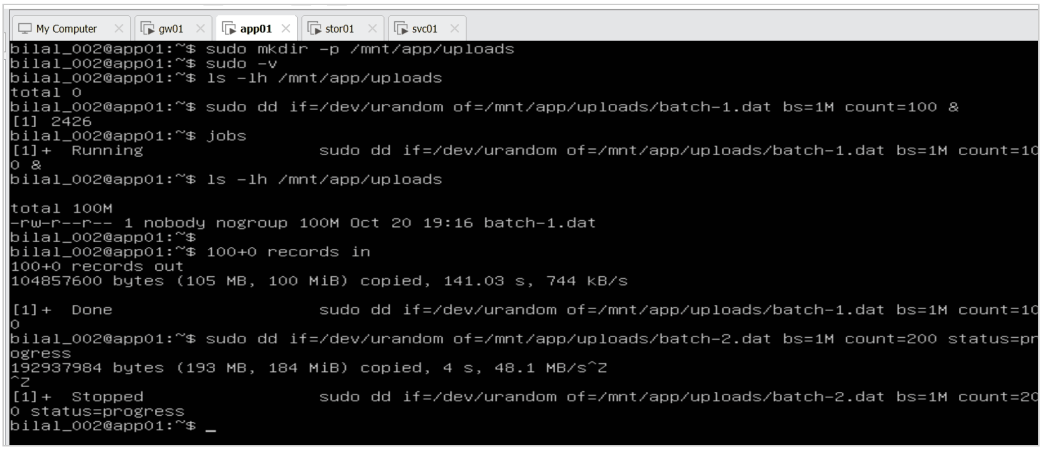
💡 **The story:** so far every command ran by hand, and long commands blocked the terminal. Acme's customers upload large files from **app01**; through NFS they land in `/data/app/uploads` on **stor01**, where they must be backed up. Long jobs like these should **not block my terminal**, **not slow down the app**, and **not die when my connection drops**.

```
app01: /mnt/app/uploads  ═══ NFS ═══►  stor01: /data/app/uploads (lvdata)  ──backup──►  /backups (lvbackups)
       "customers upload"                                                      nohup tar from an SSH session
```

6.1 Customer uploads on app01 – foreground, background, pause, priority, kill

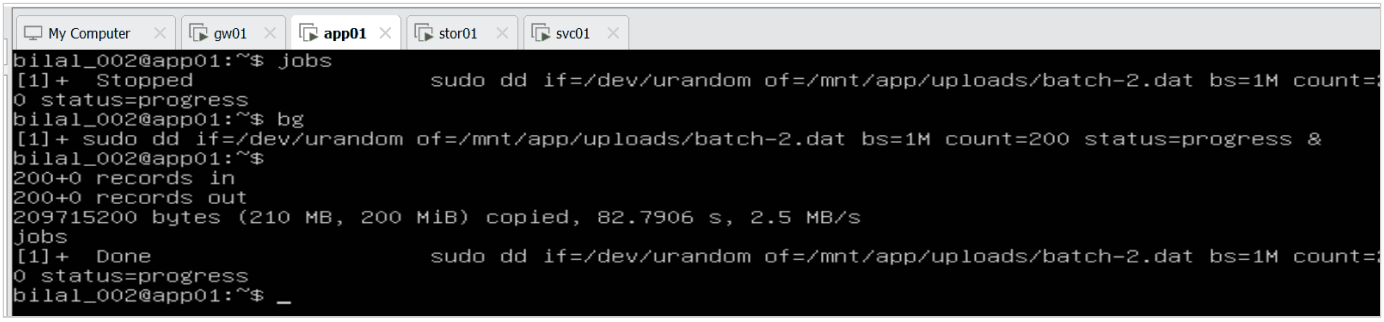
```
        # app01
        sudo rm -rf /mnt/app/uploads && sudo mkdir -p /mnt/app/uploads   # clean start (really /data/app/uploads on
stor01)
        sudo -v                                                          # cache the password — background sudo jobs
cannot ask for it

        # batch-1: in the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100 &
        jobs                                                             # [1]+ Running — the prompt is free
        ls -lh /mnt/app/uploads                                          # keep working while it uploads

        # batch-2: forgot the & — pause and send to the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-2.dat bs=1M count=200 status=progress
        #   Ctrl+Z → [1]+ Stopped
        jobs; bg; jobs                                                   # Stopped → Running → Done
```

📸 **Evidence – batch-1 in the background (`Running` → `Done`), batch-2 paused with Ctrl+Z (`Stopped`):**



bg and ctrl+z

📸 **Evidence – `bg`: it continues in the background and finishes:**



bg done

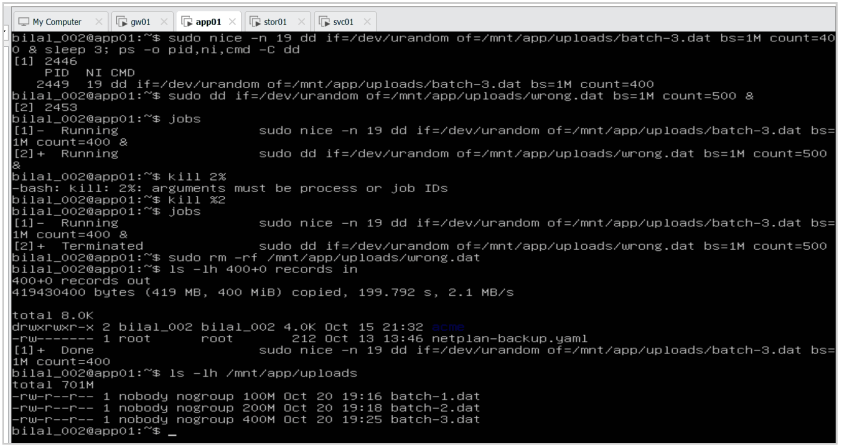
Issue – Ctrl+Z and Ctrl+C seemed to do nothing

📸 **Evidence:**

ctrl+z waits

```
        # batch-3: low priority — start dd and check its nice value in the same line
        sudo nice -n 19 dd if=/dev/urandom of=/mnt/app/uploads/batch-3.dat bs=1M count=400 & sleep 3; ps -o pid,ni,cmd -
C dd

        # wrong.dat: started by mistake — stop only that job
        sudo dd if=/dev/urandom of=/mnt/app/uploads/wrong.dat bs=1M count=500 &
        jobs                                # [1] batch-3 Running, [2] wrong.dat Running
        kill %2                             # [2] Terminated — batch-3 keeps running
        sudo rm -f /mnt/app/uploads/wrong.dat
        ls -lh /mnt/app/uploads             # batch-1 100M, batch-2 200M, batch-3 400M
```

📸 **Evidence – `NI 19`; two jobs at once, only `%2` killed; the three batches, owned by `nobody` (root_squash):**

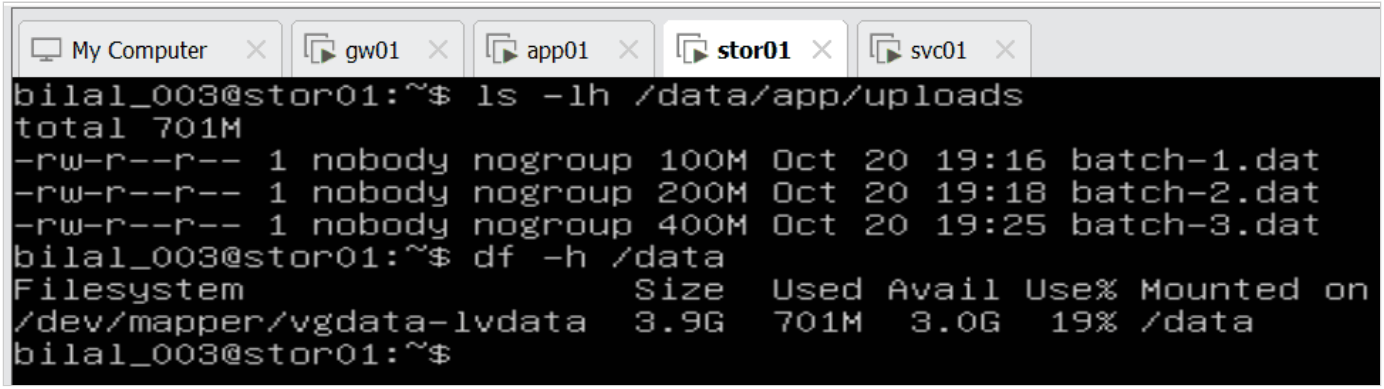


nice kill result

6.2 On stor01 – the uploads arrived; a backup that survives a logout

```
        # stor01
        ls -lh /data/app/uploads          # the three batches, 701 MB
        df -h /data                       # 19 % used
```

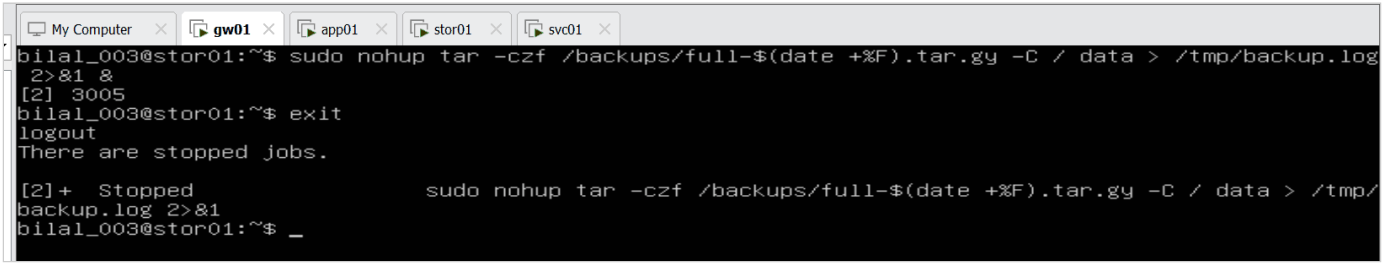
📸 **Evidence:**



uploads arrived

Issue – the background backup went “Stopped”

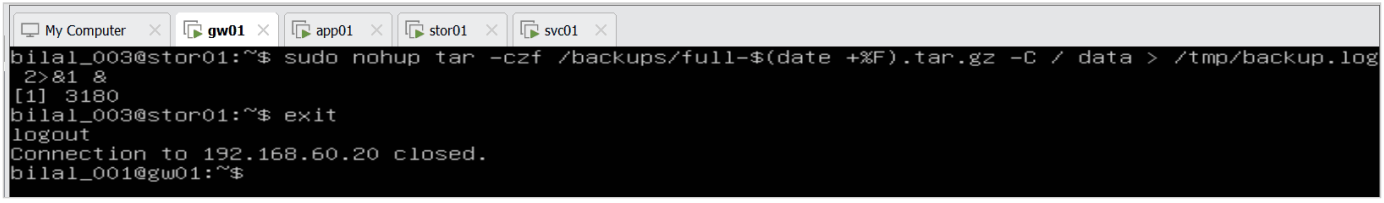
📸 **Evidence:**



stopped

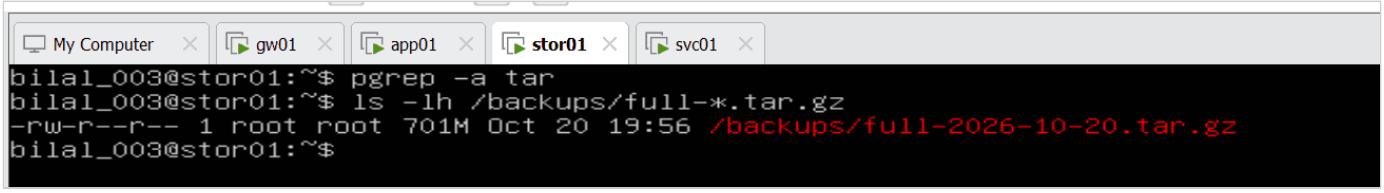
```
# gw01 — plays "my laptop"
ssh stor01
sudo -v
sudo nohup tar -czf /backups/full-$(date +%F).tar.gz -C / data > /tmp/backup.log 2>&1 &
exit                                    # really logged out: "Connection to 192.168.60.20 closed."
ssh stor01
pgrep -a tar                            # nothing — finished
ls -lh /backups/full-*.tar.gz           # 701M, finished at 19:56 — after I had left
sudo rm -f /backups/full-*.tar.gz       # free the space again
```

📸 Evidence – started, logged out:



nohup logout

📸 Evidence – the archive was finished while I was gone:



finished after logout

Key understanding (in my own words)

Acme's customers upload files from app01 through `/mnt/app`, which NFS stores in `/data/app/uploads` on stor01. Long jobs like these uploads and backups should not block my terminal or slow down the app.

Foreground and background: any command can run in the foreground or the background. In the foreground, the terminal is busy until the command finishes. With `&` it runs in the background, and I get the prompt back at once. If I forget the `&`, `Ctrl+Z` pauses the job and `bg` lets it continue in the background; `fg` brings it back. `jobs` lists my jobs with their numbers. A wrong job is stopped with `kill %number` (job number, same terminal) or `kill PID` (process ID, from anywhere). On a network disk (NFS), a job waiting for a write reacts to Ctrl+Z or kill only after that write finishes.

Priority: `nice` and `renice` set a job's priority from -20 (most important) to 19 (politest). All jobs run at the same time, but when the CPU is busy, lower numbers get more CPU time. `nice` sets it at the start; `renice` changes it while the job runs. That is how a backup can run during the day without slowing down the app.

Surviving a logout: a job started from an SSH session is normally killed when the connection closes. With `nohup … &` it keeps running on the server after I log out. I started a 700 MB backup on stor01 from gw01, logged out, came back, and the archive had been finished while I was gone. Two lessons: run `sudo -v` first, because a background job cannot answer a password prompt (it becomes "Stopped"); and check `jobs` before `exit`, because Bash will not log out while a job is stopped. The archive was as big as the data (701 MB), because random files like customer uploads hardly compress.

7. Step 3.2 – disk-full alert: a systemd service and timer

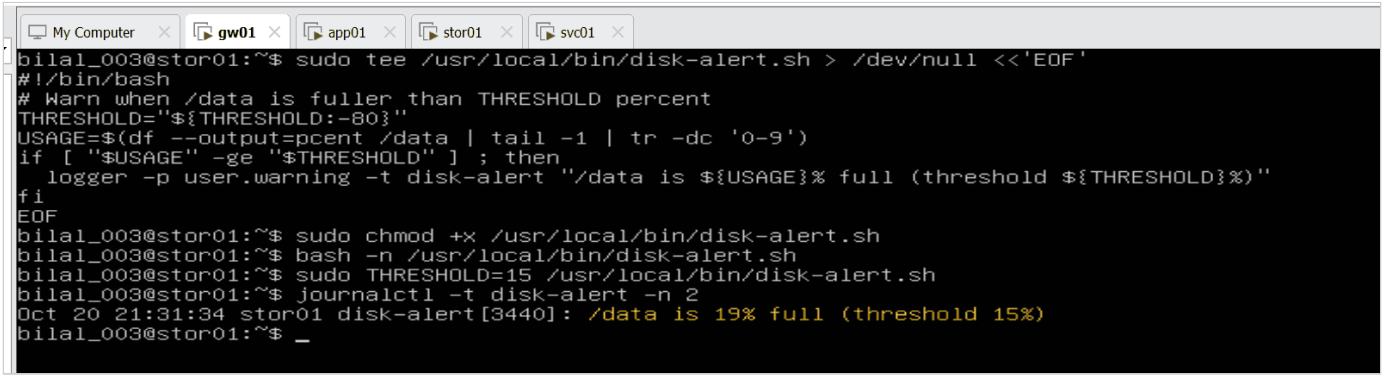
💡 **The story:** files uploaded on app01 – today by me as a simulation, later by customers once the app is published through gw01 (Phase 7) – land in `/mnt/app`, which is really `/data` on stor01 (`lvdata`). If `/data` fills up, uploads fail and the app breaks, often before anyone in IT notices. In Phase 2 I can grow `/data` with LVM – but only if someone knows it is filling up. So, like a phone warning "storage almost full", stor01 must **watch its own disk and warn by itself** – no human, no manual command.

```
disk-alert.timer ──every 5 min──► disk-alert.service ──runs──► disk-alert.sh ──if too full──► journal (system log)
     WHEN                              HOW                          WHAT
```

7.1 The script – WHAT to check

```
sudo tee /usr/local/bin/disk-alert.sh > /dev/null <<'EOF'
#!/bin/bash
# Warn when /data is fuller than THRESHOLD percent
THRESHOLD="${THRESHOLD:-80}"
USAGE=$(df --output=pcent /data | tail -1 | tr -dc '0-9')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
  logger -p user.warning -t disk-alert "/data is ${USAGE}% full (threshold ${THRESHOLD}%)"
fi
EOF
sudo chmod +x /usr/local/bin/disk-alert.sh
bash -n /usr/local/bin/disk-alert.sh              # syntax check without running – no output = OK
sudo THRESHOLD=15 /usr/local/bin/disk-alert.sh    # one manual run with a low test limit
journalctl -t disk-alert -n 2
```

📸 **Evidence – syntax OK; manual run: `/data is 19% full (threshold 15%)`:**



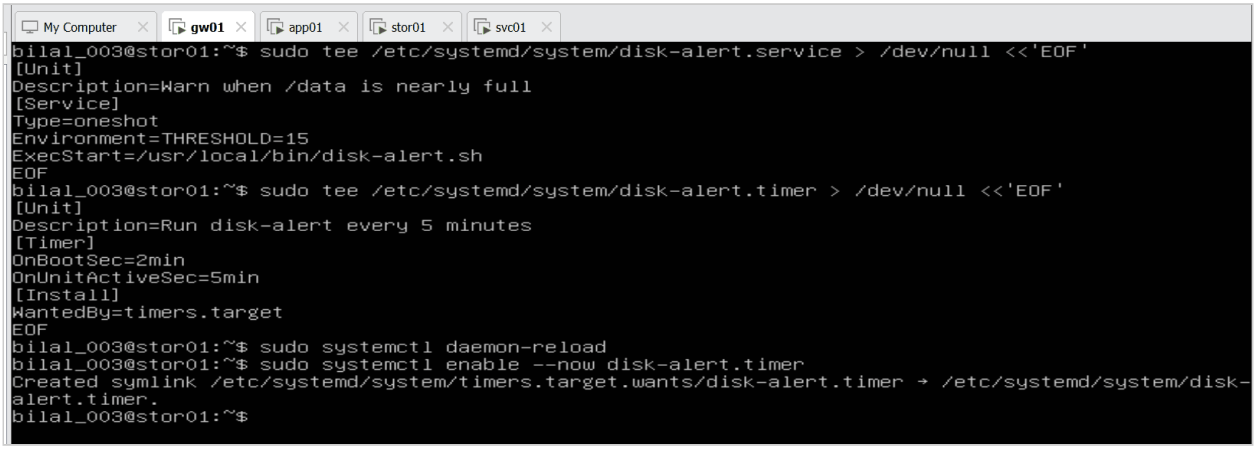
script manual run

Lesson: my first version had two typing mistakes (a `)` inside the quotes, no spaces inside `[ ]`). `bash -n` checks a script **before** it runs – use it every time.

7.2 The service (HOW) and the timer (WHEN)

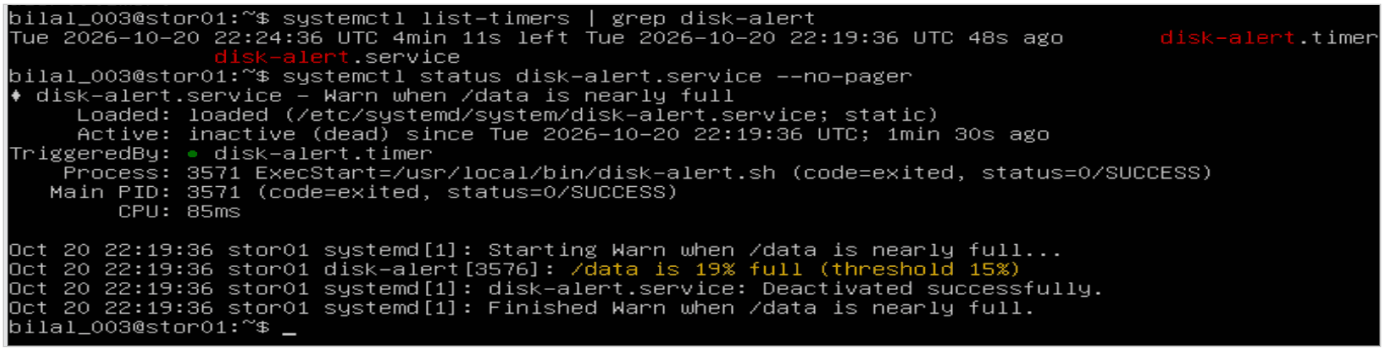
```
sudo tee /etc/systemd/system/disk-alert.service > /dev/null <<'EOF'
[Unit]
Description=Warn when /data is nearly full
[Service]
Type=oneshot
Environment=THRESHOLD=15
ExecStart=/usr/local/bin/disk-alert.sh
EOF

sudo tee /etc/systemd/system/disk-alert.timer > /dev/null <<'EOF'
[Unit]
Description=Run disk-alert every 5 minutes
[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload                      # systemd reads the new files
sudo systemctl enable --now disk-alert.timer      # start now + at every boot
systemctl list-timers | grep disk-alert
systemctl status disk-alert.service --no-pager
```

📸 **Evidence – units created, timer enabled:**



service timer

📸 **Evidence – `list-timers`: NEXT in 4 min, LAST 48 s ago; `status`:** TriggeredBy: disk-alert.timer**, `status=0/SUCCESS`, then `inactive (dead)` until the next run:**

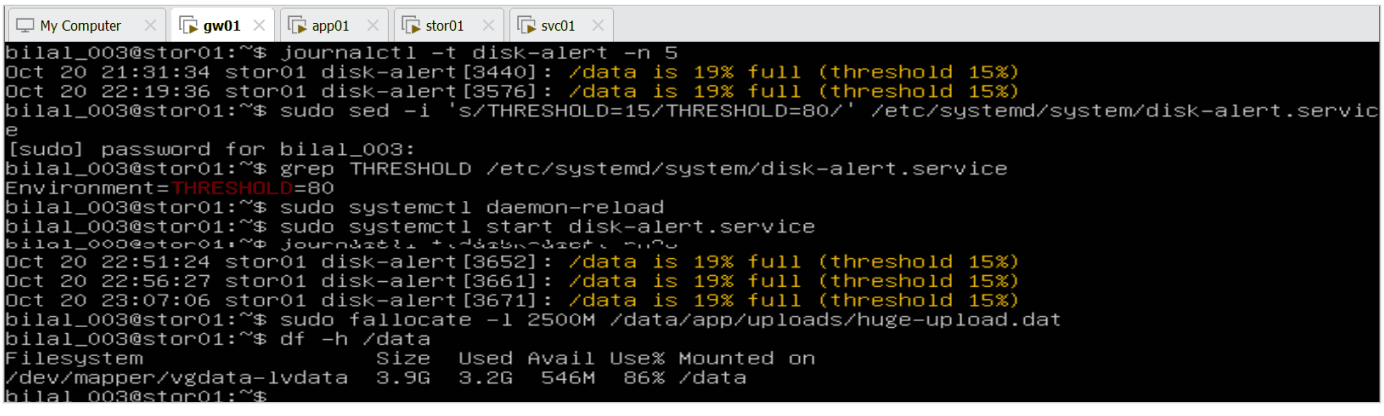


triggered by timer

7.3 The real limit – silent when fine, loud when full

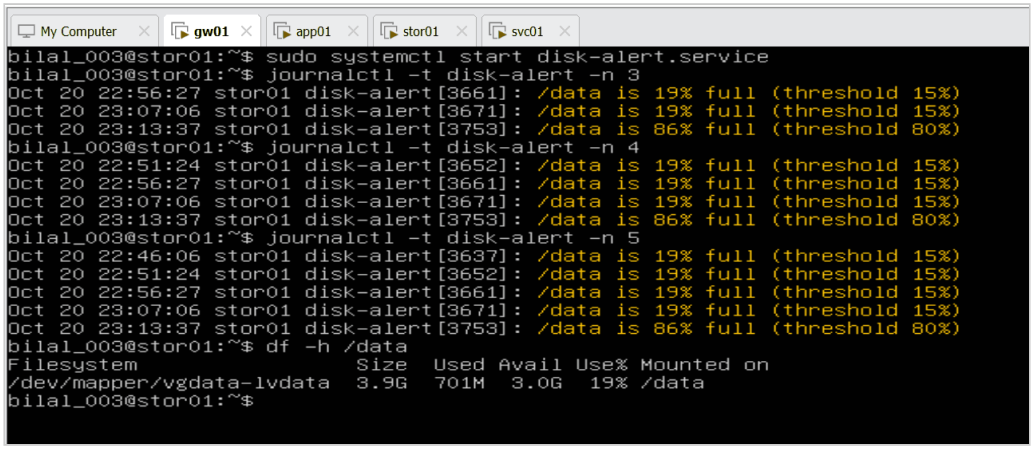
```
sudo sed -i 's/THRESHOLD=15/THRESHOLD=80/' /etc/systemd/system/disk-alert.service
sudo systemctl daemon-reload                          # always after changing a unit file
sudo systemctl start disk-alert.service               # 19 % < 80 % → no new line: no news is good news
journalctl -t disk-alert -n 3                         # 22:51, 22:56, 23:07 — runs by the timer, by themselves
sudo fallocate -l 2500M /data/app/uploads/huge-upload.dat    # simulate a huge upload — reserves 2.5 GB instantly
df -h /data                                           # 86 %
sudo systemctl start disk-alert.service
journalctl -t disk-alert -n 3                         # "/data is 86% full (threshold 80%)"
sudo rm /data/app/uploads/huge-upload.dat
df -h /data                                           # back to 19 %
```

📸 **Evidence – limit 80 %, silent at 19 %; the timer's own runs; the "huge upload" takes `/data` to 86 %:**



silent then 86

📸 **Evidence – the alert fires: `/data is 86% full (threshold 80%)`; after clean-up 19 %:**



alert 86

Key understanding (in my own words)

Files uploaded on app01 (today by me as a simulation, later by customers once the app is published through gw01) go into `/mnt/app`, which is really `/data` on stor01, the `lvdata` volume. If `/data` fills up, new uploads fail and the app breaks, often before anyone in IT notices. In Phase 2 I could grow `/data` with LVM, but only if someone knows it is filling up. So, like a phone that warns “storage almost full”, stor01 should **watch its own disk and warn us by itself**, without a human running a command. The script `/usr/local/bin/disk-alert.sh` reads how full `/data` is with `df`, keeps only the number with `tr -dc '0-9'` (so `19%` becomes `19`), and compares it with a threshold: 80 % normally, 15 % for my test. If the usage is greater than or equal to the threshold, `logger` writes a **warning to the system log**, tagged `disk-alert`, which I read with `journalctl -t disk-alert`. I checked the script with `bash -n` and ran it once by hand.

Then I turned it into automation: `disk-alert.service` says *how* to run it (`Type=oneshot` – run once and finish), and `disk-alert.timer` says *when* – 2 minutes after boot and then every 5 minutes. `systemctl enable --now` started the timer and made it survive reboots; `status` showed *TriggeredBy: disk-alert.timer* and *SUCCESS*. The timer wakes the service, the service runs the script, and the script writes a warning only if `/data` is too full. With the test limit, warnings appeared every few minutes by themselves. With the real limit of 80 % (`sed` + `daemon-reload`) the script stayed silent at 19 %; a simulated 2.5 GB upload (`fallocate`) pushed `/data` to 86 %, and the alert *“/data is 86% full (threshold 80%)”* appeared. stor01 now watches its own disk, and the warning comes early enough to grow the volume or clean up.

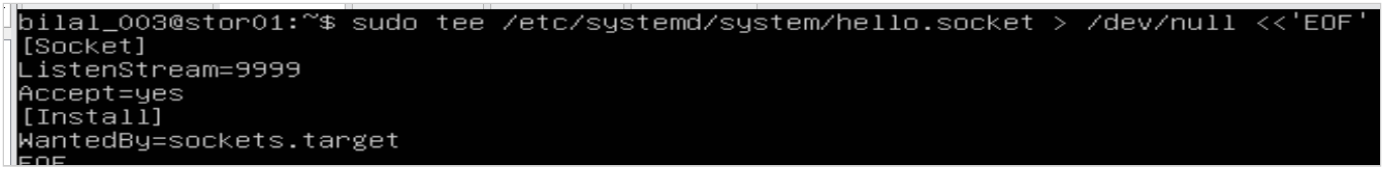
8. Step 3.3 – socket activation: start a service only when someone connects

💡 **The story:** a rarely used service does not need to run all day and use memory. Instead, **systemd itself listens on the port** and starts the program only when a client connects – then it ends again. Real systems use this: `ssh.socket` (newer Ubuntu), `docker.socket`, `cups.socket` (printing).

```
client connects to :9999 ──► hello.socket (systemd listens)
                               │ same name "hello" → template hello@.service
                               ▼
                     one copy starts → echo → output back to the client → ends
```

```
sudo tee /etc/systemd/system/hello.socket > /dev/null <<'EOF'
[Socket]
ListenStream=9999
Accept=yes
[Install]
WantedBy=sockets.target
EOF
sudo tee /etc/systemd/system/hello@.service > /dev/null <<'EOF'
[Service]
ExecStart=/bin/sh -c 'echo "hello from stor01 — started on demand"'
StandardOutput=socket
EOF
sudo systemd-analyze verify /etc/systemd/system/hello@.service   # check the unit before testing
sudo systemctl daemon-reload
sudo systemctl enable --now hello.socket
ss -tlnp | grep 9999                                             # LISTEN *:9999 — systemd holds the port
nc localhost 9999                                                # → hello from stor01 — started on demand
```

📸 **Evidence – the socket unit:**



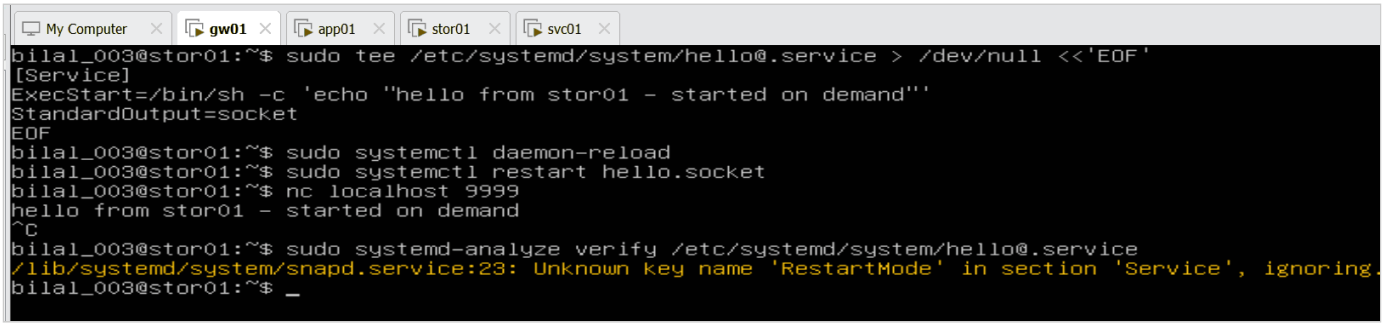
socket unit

📸 **Evidence – enabled; `LISTEN *:9999`:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now hello.socket
Created symlink /etc/systemd/system/sockets.target.wants/hello.socket → /etc/systemd/system/hello.so
cket.
bilal_003@stor01:~$ ss -tlpn | grep 9999
LISTEN 0      4096               *:9999             *:*
bilal_003@stor01:~$ nc localhost 9999
```

listening

📸 **Evidence – the greeting on connect; `systemd-analyze verify` shows no error for my file (the yellow line is about Ubuntu's own `snapd.service`):**



on demand

Lessons from my mistakes: the service was first named `hello.service` (with `Accept=yes` it must be the template `hello@.service`), the section was misspelled (`[Servuce]`), and `ExecStart` lacked the leading `/` – each time `nc` just waited. `ExecStart` needs a full path, `cat` the unit once, and `systemd-analyze verify` catches such mistakes before testing.

Key understanding (in my own words)

> In systemd, everything it manages is a **unit**: a small text file in `/etc/systemd/system/` that describes one thing, and the ending of the name says what kind. A **`.service`** says *how* to run a program, a **`.timer`** says *when* to start a service, a **`.socket`** says *which network port* to listen on, and a **`.target`** is a *group* of units, like `multi-user.target` (server mode). After creating or changing a unit, `systemctl daemon-reload` makes systemd read it; `enable` starts it at every boot and `--now` also starts it immediately. In Step 3.2 I used a service plus a timer for the disk alert. In Step 3.3 I used a **socket**: a rarely used service does not need to run all day, so systemd itself listens on the port (`hello.socket`, port 9999) and starts the program **only when someone connects**. The socket and the service are linked by their name (`hello`). With `Accept=yes`, each connection gets its own short-lived copy of the template `hello@.service` (the `@` means template), and `StandardOutput=socket` sends the program's output straight back through the connection to the client. `nc localhost 9999` returned *"hello from stor01 – started on demand"*; before and after, nothing was running. Real systems use the same idea, for example `ssh.socket`, `docker.socket` and `cups.socket`. My mistakes taught me three checks: the template needs the `@`, `ExecStart` needs a full path starting with `/`, and `systemd-analyze verify` checks a unit file before testing – like `bash -n` for scripts.

Day 6 – 21 Oct: custom target, cron vs timer, nightly backup – Phase 3 complete

1. Revision – what a systemd unit really is

💡 **In plain words:** before going on, I explained Steps 3.2 and 3.3 again in my own words and corrected what I had misunderstood.

```
TIMER:   disk-alert.timer ──same name──► disk-alert.service ──ExecStart──► disk-alert.sh ──logger──► journal
            WHEN                              HOW                                WHAT
SOCKET:  client connects :9999 ──► hello.socket ──same name──► one copy of hello@.service ──► answer to client
──► copy ends
```

Key understanding (in my own words)

systemd is the program that starts and manages services on a Linux server. Everything it manages (SSH, Docker, my disk alert) is a **unit**: a small settings file that says how something should run. Linux's own units are in `/lib/systemd/system/` (for example `ssh.service` and `ssh.socket`), and the units I create for automation go in `/etc/systemd/system/`. The ending of the name says the type: **`.service`** = **how** to run a program, **`.timer`** = **when** to run it, **`.socket`** = **which port** to listen on, and **`.target`** = a **group** of units that start together. After creating or changing a unit I run `systemctl daemon-reload`, and `enable --now` starts it now and at every boot.

Timer (Step 3.2): the script `/usr/local/bin/disk-alert.sh` checks how full `/data` is and writes a warning to the journal only if usage reaches the threshold (80 %). `disk-alert.service` says **how** to run it (`Type=oneshot`, `ExecStart=` the script), and `disk-alert.timer` says **when** – 2 minutes after boot, then every 5 minutes. The timer finds its service **by the same first name** (`disk-alert`); `WantedBy=timers.target` only makes the timer start at every boot. The chain is **timer → service → script → journal**.

Socket (Step 3.3): here the trigger is a **connection**, not a time. A rarely used service does not need to run all day, so **systemd itself listens on the port** and starts the program only when someone connects. `hello.socket` says `ListenStream=9999` and finds its service by the same first name, `hello`. With **`Accept=yes`**, every connection gets **its own short-lived copy** of the service – that is why the service is a **template**, `hello@.service`: the `@` means one file, many copies, one per connection. `ExecStart=/bin/sh -c 'echo …'` is the program, and **`StandardOutput=socket`** sends its output straight back through the connection to the client. I tested it with `nc localhost 9999` and got *"hello from stor01 – started on demand"*; the copy ended, so nothing was running before or after. The chain is **client connects → socket → one copy of the service → answer → copy ends**. Real servers use this too: with `ssh.socket`, systemd holds port 22 and starts `ssh.service` on the first connection (with `Accept=no`, one sshd handles all connections itself – no template). On stor01, `ssh.socket` is disabled and `ssh.service` runs all the time – the classic choice for a server admins use daily. My three lessons: a per-connection service needs the `@`, `ExecStart` needs a full path starting with `/`, and `systemd-analyze verify` checks a unit before I use it.

2. Step 3.4 – a custom target: one command for the whole Acme stack

💡 **The story:** stor01 now has several Acme units of my own – the disk alert, the hello socket, later the nightly backup. During maintenance an admin should not have to remember every name. One group, **`acme-stack.target`**, starts them all with one command – the same idea as `multi-user.target`, but for Acme.

```
sudo tee /etc/systemd/system/acme-stack.target > /dev/null <<'EOF'
[Unit]
Description=Acme services group
Wants=disk-alert.timer hello.socket
AllowIsolate=yes
EOF
sudo systemd-analyze verify /etc/systemd/system/acme-stack.target
sudo systemctl daemon-reload

# test: stop both members, then start the whole group with ONE command
sudo systemctl stop disk-alert.timer hello.socket
systemctl is-active disk-alert.timer hello.socket      # inactive inactive
sudo systemctl start acme-stack.target
systemctl is-active disk-alert.timer hello.socket      # active active
systemctl list-dependencies acme-stack.target          # the group as a tree
```

📸 **Evidence – both members stopped, one `start` of the target, both active again; the group tree (the yellow line is Ubuntu's own `snapd.service`, not my file):**

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/acme-stack.target > /dev/null <<'EOF'
[Unit]
Description=Acme services group
Wants=disk-alert.timer hello.socket
AllowIsolate=yes
EOF
[sudo] password for bilal_003:
bilal_003@stor01:~$ sudo systemd-analyze verify /etc/systemd/system/acme-stack.target
/lib/systemd/system/snapd.service:23: Unknown key name 'RestartMode' in section 'Service', ignorin
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl stop disk-alert.timer hello.socket
bilal_003@stor01:~$ systemctl is-active disk-alert.timer hello.socket
inactive
inactive
bilal_003@stor01:~$ sudo systemctl start acme-stack.target
bilal_003@stor01:~$ systemctl is-active disk-alert.timer hello.socket
active
active
bilal_003@stor01:~$ systemctl list-dependencies acme-stack.target
acme-stack.target
● ├─disk-alert.timer
● └─hello.socket
bilal_003@stor01:~$
```

acme-stack target

Key understanding (in my own words)

> A **target** is a unit that **groups other units** – services, timers or sockets – so that starting the target starts all its members with one command. Linux's own targets are in `/lib/systemd/system/`: `timers.target` holds all the timers (that is why each timer has `WantedBy=timers.target`), and `multi-user.target` is the normal **server state** – networking, SSH and all other services, with text login and no desktop. `graphical.target` is `multi-user.target` plus the graphical desktop, so targets can contain other targets; my servers boot to `multi-user.target`. On stor01 I created my own group, `acme-stack.target`, and `Wants=disk-alert.timer hello.socket` made both units its members. To test it, I stopped both (`inactive inactive`), started only `acme-stack.target`, and both became `active` again; `systemctl list-dependencies acme-stack.target` showed the group as a tree. In a real company this is how an admin brings a whole application stack back after maintenance with one command – for example `company-app.target` – instead of remembering every service name.

3. Reference – looking inside the real server

💡 **In plain words:** to see that these ideas are not only lab examples, I looked at how Ubuntu itself uses sockets, targets and timers on stor01.

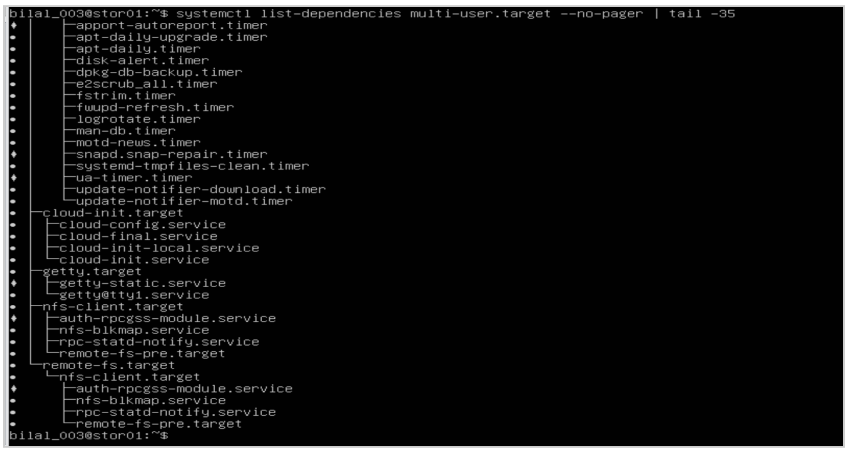
```
        # stor01
        systemctl list-sockets                                  # 29 sockets held by systemd — hello.socket on
[::]:9999
        sudo systemctl status ssh.socket ssh.service            # socket mode or always-on?
        systemctl list-dependencies multi-user.target --no-pager  # what "server mode" contains
        ls /etc/systemd/system/multi-user.target.wants/         # members I or packages enabled
        ls /etc/systemd/system/timers.target.wants/
        ls /etc/systemd/system/sockets.target.wants/
```

📸 **Evidence – `ssh.socket` disabled and inactive, `ssh.service` enabled and running since boot (5.2 MB); `ExecStartPre=/usr/sbin/sshd -t` tests the configuration before starting:**

```
bilal_003@stor01:~$ sudo systemctl status ssh.socket ssh.service
[sudo] password for bilal_003:
♦ ssh.socket - OpenBSD Secure Shell server socket
     Loaded: loaded (/lib/systemd/system/ssh.socket; disabled; vendor preset: enabled)
     Active: inactive (dead)
     Listen: [::]:22 (Stream)
   Accepted: 0; Connected: 0;

● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-10-21 16:40:15 UTC; 36min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1031 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1067 (sshd)
      Tasks: 1 (limit: 964)
     Memory: 5.2M
        CPU: 1.939s
     CGroup: /system.slice/ssh.service
             └─1067 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

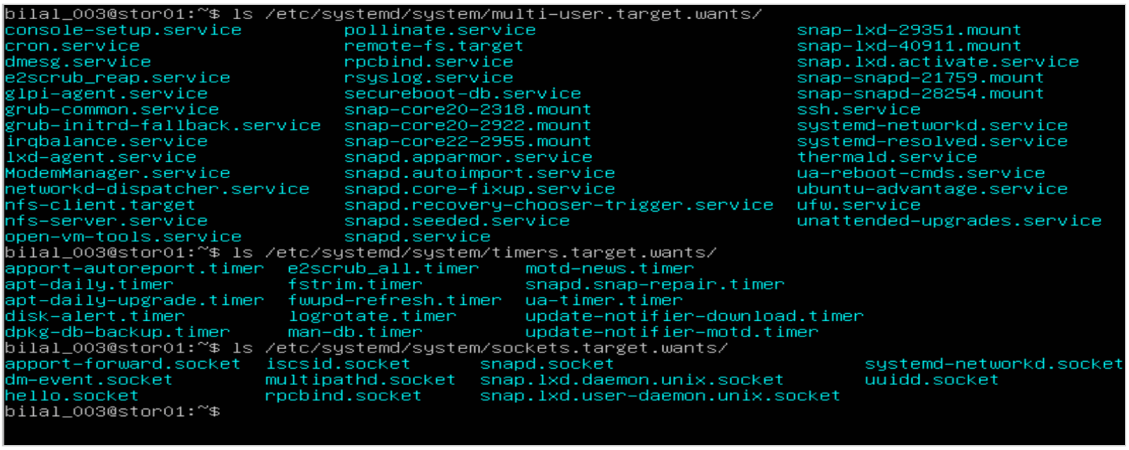
ssh socket vs service

📸 **Evidence – `multi-user.target` contains other targets: `timers.target` with my `disk-alert.timer` next to Ubuntu's own timers (`apt-daily`, `logrotate`, `fstrim`), `getty.target` with the console login `getty@tty1.service` (a template again), and `nfs-client.target` / `remote-fs.target`:**



multi-user tree

📸 **Evidence – `enable` = a link in a `….wants/` folder: `ssh.service`, `nfs-server.service`, `glpi-agent.service` in `multi-user.target.wants/`; `disk-alert.timer` in `timers.target.wants/`; `hello.socket` in `sockets.target.wants/`:**



wants folders

Key understanding (in my own words)

> `systemctl list-dependencies multi-user.target` showed me what "server mode" really is: a tree of units. Its direct members are services like `cron`, `rsyslog`, `nfs-server`, `rpcbind`, `ssh`, `glpi-agent` and `open-vm-tools`, plus `.mount` units – mounts are units too. It also contains **other targets**: `timers.target`, where my `disk-alert.timer` sits next to Ubuntu's own timers (`apt-daily`, `logrotate`, `fstrim`); `sockets.target` with `hello.socket`; `getty.target`, whose `getty@tty1.service` is the console login and also a template with `@`; and `remote-fs.target` for NFS. The `….wants/` folders in `/etc/systemd/system/` show how membership works: `enable` simply creates a link in the folder of the target named in `WantedBy=`. On stor01, `ssh.socket` is disabled and `ssh.service` always runs, and its `ExecStartPre=sshd -t` tests the configuration before every start. So a target is a group, groups can contain groups, `enable` adds a member, and booting the server simply means systemd starting this whole tree.

4. Step 3.5 part A – cron, the old way

💡 **The story:** before systemd timers, Linux ran scheduled jobs with **cron**. Many companies still have hundreds of cron jobs – backups, reports, clean-ups – so an admin must be able to read and write them. I ran the disk check with cron for a few minutes, compared it with the timer, then removed it.

```
        crontab -e                          # my job list – first time: choose 1 (nano)
        # added at the bottom:
        */2 * * * * THRESHOLD=15 /usr/local/bin/disk-alert.sh

        crontab -l                                  # the line is there
        grep CRON /var/log/syslog | tail -3         # cron started the job
        journalctl -t disk-alert -n 3               # the warnings the script wrote
        crontab -e                                  # afterwards: line deleted again
```

```
*/2    *     *     *      *        command
min    hour   day   month  weekday          * = every · */2 = every 2 · 0 2 * * * = daily at 02:00
```

📸 **Evidence – my crontab line; `syslog`: cron started it as `bilal_003` at 20:08, next to Ubuntu's own hourly cron jobs run as root:**

```
bilal_003@stor01:~$ crontab -l
# m h  dom mon dow   command
*/2 * * * * THRESHOLD=15 /usr/local/bin/disk-alert.sh

bilal_003@stor01:~$ grep CRON /var/log/syslog | tail -3
Oct 21 18:17:01 stor01 CRON[9116]: (root) CMD (   cd / && run-parts --report /etc/cron.hourly)
Oct 21 19:17:01 stor01 CRON[9215]: (root) CMD (   cd / && run-parts --report /etc/cron.hourly)
Oct 21 20:08:01 stor01 CRON[9479]: (bilal_003) CMD (THRESHOLD=15 /usr/local/bin/disk-alert.sh)
bilal_003@stor01:~$
```

crontab and syslog

📸 **Evidence – the script's warnings at 20:08 and 20:10 – exactly every 2 minutes (the 86 % line is yesterday's test, before a reboot):**

```
bilal_003@stor01:~$ journalctl -t disk-alert -n 3
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
-- Boot 6e4ad773005e4624b8f2adda0aba8d00 --
Oct 21 20:08:01 stor01 disk-alert[9485]: /data is 19% full (threshold 15%)
Oct 21 20:10:02 stor01 disk-alert[9498]: /data is 19% full (threshold 15%)
bilal_003@stor01:~$
```

cron every 2 min

```
cron (every 2 min) ──runs──► disk-alert.sh ──THRESHOLD=15 given──► 19 ≥ 15? yes ──logger──► journal
```

Lesson: the crontab line only says **when** to run the script – not where warnings go. The **script** decides that: `THRESHOLD="${THRESHOLD:-80}"` takes the value I pass (15), and only if usage ≥ threshold does `logger` write to the journal. That is why the same script works unchanged with cron and with a systemd timer. The server clock is UTC, so 20:08 is 22:08 Berlin time.

Key understanding (in my own words)

> Before systemd timers, Linux ran scheduled jobs with **cron**, and many companies still use it, so an admin must be able to read it. Each user has their own job list, the **crontab**. `crontab -e` opens it in an editor (nano), and every job is **one line**: five time fields (minute, hour, day of month, month, weekday) followed by the command. On stor01 I added `*/2 * * * * THRESHOLD=15 /usr/local/bin/disk-alert.sh`. `*/2` means every 2 minutes, and each `*` means every hour, day, month and weekday. Cron, like a timer, only **runs** the script; the crontab line does not say where warnings go. The **script** decides that. `THRESHOLD="${THRESHOLD:-80}"` takes the value I pass (15), or 80 if none is given. `df` reads how full `/data` is (19 %), and only if 19 ≥ 15 does `logger` write the warning to the journal. The chain is **cron → script → logger → journal**. `crontab -l` showed my line. `grep CRON /var/log/syslog` proved that cron started the job as `bilal_003` at 20:08, next to Ubuntu's own hourly cron jobs run as root. `journalctl -t disk-alert` showed *"/data is 19% full (threshold 15%)"* at 20:08 and 20:10, exactly every 2 minutes. So the same script works unchanged with cron and with a systemd timer. Cron is simple, one line, but it has two weaknesses. First, **the logs are split**: the start of the job is in `/var/log/syslog`, and the script's warning is in the journal. Second, **missed runs are lost**: if the server is off at the scheduled time, cron simply skips that run and nothing tells me. A systemd timer solves both. `journalctl -u <name>.service` shows the start, the output and the result in one place, and `Persistent=true` runs a missed job at the next boot. Then I removed the cron line again with `crontab -e`.

5. Step 3.5 part B – the nightly backup runs by itself

💡 **The story:** in Phase 2 I made versioned backups by hand. A backup that depends on someone remembering it will one day be forgotten. Now stor01 backs up `/data/app` **every night at 02:00**, while nobody uploads – and if the server was off at 02:00, it **catches up at the next boot**.

```
nightly-backup.timer ──02:00──► nightly-backup.service ──runs──► nightly-backup.sh ──► /backups/<date> + latest + "finished" in journal
      WHEN                          HOW                          WHAT
```

```
# 1. The script — WHAT
sudo tee /usr/local/bin/nightly-backup.sh > /dev/null <<'EOF'
#!/bin/bash
# Nightly versioned backup of /data/app, keep 7 days
TODAY=$(date +%F)
rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/$TODAY/
ln -sfn /backups/$TODAY /backups/latest
find /backups -maxdepth 1 -type d -name '20*' -mtime +7 -exec rm -rf {} +
logger -t nightly-backup "backup of /data/app to /backups/$TODAY finished"
EOF
sudo chmod +x /usr/local/bin/nightly-backup.sh      # sudo — the file belongs to root
bash -n /usr/local/bin/nightly-backup.sh            # syntax check — for scripts only

# 2. The service — HOW
sudo tee /etc/systemd/system/nightly-backup.service > /dev/null <<'EOF'
[Unit]
Description=Nightly backup of /data/app
[Service]
Type=oneshot
ExecStart=/usr/local/bin/nightly-backup.sh
EOF

# 3. The timer — WHEN
sudo tee /etc/systemd/system/nightly-backup.timer > /dev/null <<'EOF'
[Unit]
Description=Nightly backup at 02:00
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
[Install]
WantedBy=timers.target
EOF

sudo systemd-analyze verify /etc/systemd/system/nightly-backup.*   # check unit files
sudo systemctl daemon-reload
sudo systemctl enable --now nightly-backup.timer
systemctl list-timers | grep nightly

# 4. Test now — don't wait until 02:00
sudo systemctl start nightly-backup.service
ls -l /backups
journalctl -u nightly-backup.service -n 5

# 5. Add the trigger to the Acme group
sudo systemctl add-wants acme-stack.target nightly-backup.timer
systemctl list-dependencies acme-stack.target
```

📸 **Evidence – the timer unit:**

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/nightly-backup.timer > /dev/null <<'EOF'
[Unit]
Description=Nightly backup at 02:00:00
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
[Install]
WantedBy=timers.target
EOF
```

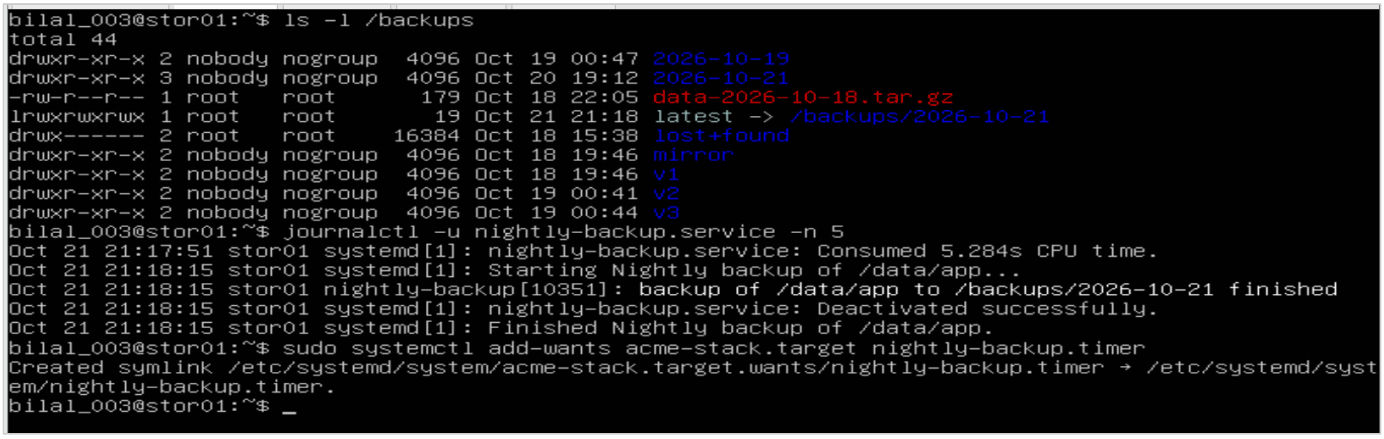
timer unit

📸 **Evidence – `enable --now` creates the link in `timers.target.wants/`; `verify` shows no error for my files; next run Thu 02:00 UTC:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now nightly-backup.timer
Created symlink /etc/systemd/system/timers.target.wants/nightly-backup.timer → /etc/systemd/system/n
ightly-backup.timer.
bilal_003@stor01:~$ sudo systemd-analyze verify /etc/systemd/system/nightly-backup.*
/lib/systemd/system/snapd.service:23: Unknown key name 'RestartMode' in section 'Service', ignoring.
bilal_003@stor01:~$ systemctl list-timers | grep nightly
Thu 2026-10-22 02:00:00 UTC 4h 43min left n/a                          n/a          nightly-backup.ti
mer           nightly-backup.service
bilal_003@stor01:~$ _
```

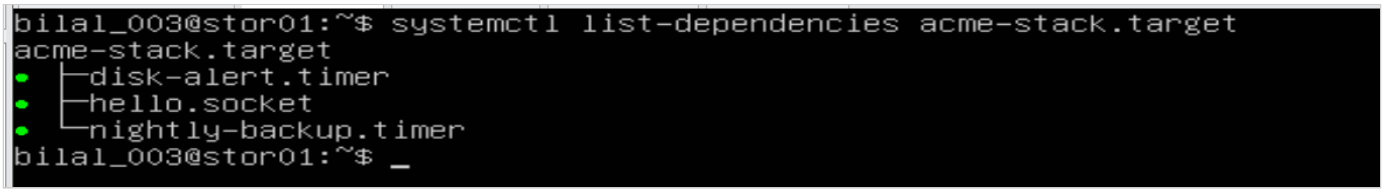
enable verify list-timers

📸 **Evidence – `/backups/2026-10-21` created, `latest → /backups/2026-10-21`; the journal shows start, "finished" and *Deactivated successfully* in one place; the timer added to `acme-stack.target`:**



backup folder journal add-wants

📸 **Evidence – the Acme group now has three members:**



three members

Lessons: `bash -n` is for **scripts** only – a `.service` or `.timer` file is checked with `systemd-analyze verify`. A heredoc must end with exactly the word it started with (`<<'EOF'` … `EOF`); a mistyped start word leaves the shell waiting at `>` – press Ctrl+C and type it again. The first run copied the 700 MB of new uploads (5.3 s CPU); a second run was instant because nothing had changed – the hard-link idea at work.

Key understanding (in my own words)

> The nightly backup now runs by itself on stor01, with the pattern **timer → service → script → journal**. The script `/usr/local/bin/nightly-backup.sh` does what I did by hand in Phase 2. `rsync -a --delete --link-dest=/backups/latest` copies `/data/app` into a folder named with today's date, and unchanged files become hard links that take no extra space. `ln -sfn` moves the `latest` link to the new folder, `find … -mtime +7` deletes versions older than 7 days, and `logger` writes "finished" to the journal. `nightly-backup.service` says **how** (`Type=oneshot`), and `nightly-backup.timer` says **when**: `OnCalendar=*-*-* 02:00:00`, every night, plus `Persistent=true`, which runs a missed backup at the next boot if the server was off. `systemd-analyze verify` checked the unit files (`bash -n` is only for scripts), and `enable --now` created the link in `timers.target.wants/`. `list-timers` shows the next run at 02:00 UTC (04:00 in Berlin, because the server runs on UTC). I didn't wait for the night: `systemctl start nightly-backup.service` created `/backups/2026-10-21`, moved `latest` to it, and `journalctl -u nightly-backup.service` showed the start, the "finished" message and *Deactivated successfully* all in one place. That is the advantage over cron, together with the catch-up. Finally, `systemctl add-wants acme-stack.target nightly-backup.timer` made the backup the third member of the Acme group, next to `disk-alert.timer` and `hello.socket`. A target can hold **any kind of unit**: services, timers, sockets, mounts or other targets. What I add depends on how the job should run. A service that must run **all the time**, like the NFS server or a web server, goes into the target **as a service**; `multi-user.target` holds `ssh.service` and `nfs-server.service` this way. A job that runs **at a time** or **on a connection** goes in through its **trigger**, the timer or socket, which starts the service with the same name only when needed. That is why I added `nightly-backup.timer` and not `nightly-backup.service`. If the service were in the target, every start of the target would run a backup immediately, for example in the middle of the day after maintenance.

Day 7 – 24 Oct: Phase 4 – slow app, runaway process, pipes

1. A question from industry – too many backups, and backup logs that grow too big

💡 **In plain words:** a friend who works as a Linux administrator asked: in a real company backups are taken all the time – what happens when the backups, and the backup **logs**, become huge? These are two different problems with two different fixes.

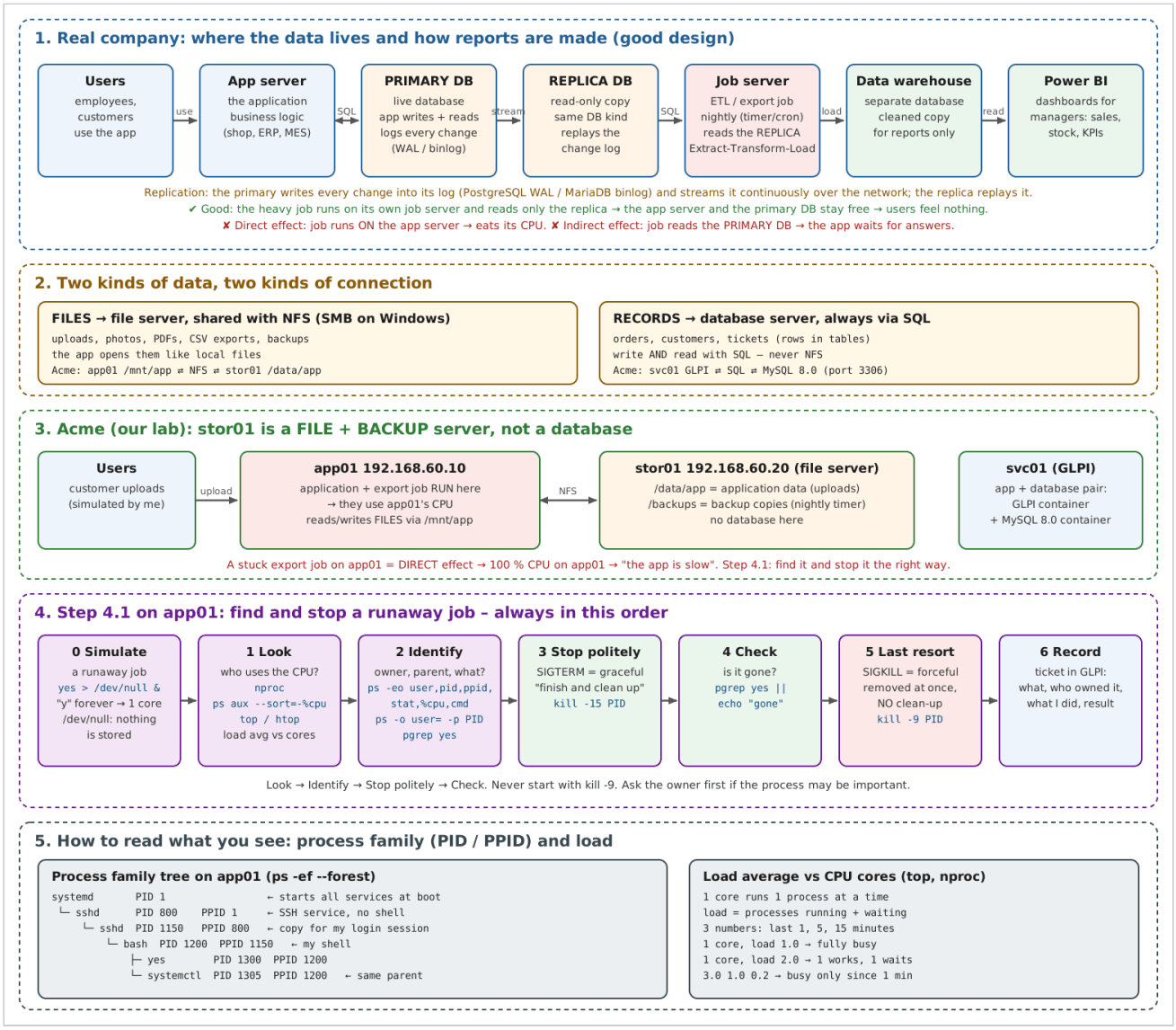
2. Real-world troubleshooting practice – how the incident drills fit in

💡 **In plain words:** to train like an on-call administrator, each phase will end with an **incident drill on my own Acme servers**: something realistic breaks, and I fix it like a real ticket – triage → hypotheses → commands → fix → verify → write-up in GLPI. The scenarios come from a hands-on troubleshooting lab (SadLab) and an L2/L3 incident workbook; only the cases that match this platform are used.

3. Phase 4 starts – Step 4.1: where a slow application comes from

💡 **The story:** app01 is Acme's application server. In real companies the most common daily problems are a **process that eats all the CPU**, a **job that hangs**, or a program that **won't stop properly**. Phase 4 teaches how to **find** the guilty process, **stop it the right way**, and finally **cap** what a service may use, so one bad program cannot freeze the whole server. Before touching commands, I worked out where such a process comes from in a real company and in Acme.

📸 **Architecture – where the data lives, how reports are made (primary → replica → job server → data warehouse → Power BI), files vs records, Acme, the order of work on app01, the process family and the load average:**



Step 4.1 architecture

```
Real company:  Users → App server ⇄ PRIMARY DB → (WAL/binlog stream) → REPLICA → Job server (ETL) → Data warehouse → Power BI
Replication:   the app writes SQL only to the primary; the primary logs every change and streams the log to the replica, which replays it
Acme:          Users → app01 (runs the app) ⇄ NFS (files) ⇄ stor01 (stores files + backups)  ·  svc01: GLPI ⇄ SQL ⇄ MySQL 8.0
Step 4.1:      Simulate → Look → Identify → Stop politely → Check → (kill -9 last) → Record
```

```
sudo apt install -y htop stress-ng
nproc                                                    # how many CPU cores
yes > /dev/null &                                        # simulate a runaway job
ps aux --sort=-%cpu | head -5                            # who uses most CPU
top                                                      # load average, %CPU, STAT — q to quit
htop                                                     # same, clearer — q to quit
ps -eo user,pid,ppid,stat,%cpu,cmd --sort=-%cpu | head -5  # owner, parent, state, command
ps -o user= -p $(pgrep yes)                              # who owns it
ps -ef --forest | less                                   # the process family tree
kill -15 $(pgrep yes)                                    # SIGTERM — graceful stop
pgrep yes || echo "gone"                                 # check
```

Issue – `apt install` hangs at `0% [Working]`

📸 **Evidence:**

```
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
Ign:4 http://de.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 stress-ng amd64 0.13.12-2ubun
tu1
Ign:1 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libipsec-mb1 amd64 1.2-1
Ign:2 http://de.archive.ubuntu.com/ubuntu jammy/universe amd64 libjudydebian1 amd64 1.0.5-5
Ign:3 http://de.archive.ubuntu.com/ubuntu jammy/main amd64 libsctp1 amd64 1.0.19+dfsg-1build1
0% [Working]_
```

apt hangs

📸 **Evidence – 2 CPU cores; the runaway job started as job [1], PID 1918:**

```
bilal_002@app01:~$ sudo apt install -y htop stress-ng
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
htop is already the newest version (3.0.5-7build2).
stress-ng is already the newest version (0.13.12-2ubuntu1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0
  libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager tcl tcl8.6 usb-modeswitch
  usb-modeswitch-data
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
bilal_002@app01:~$ nproc
2
bilal_002@app01:~$ yes > /dev/null &
[1] 1918
bilal_002@app01:~$ jobs
[1]+  Running                 yes > /dev/null &
```

nproc yes

📸 **Evidence – Look: `yes` at the top with 99.9 % CPU, state R, owner `bilal_002` (shown cut as `bilal_0+`); without the minus in `--sort=-%cpu`, ps sorts lowest first:**

```
bilal_002@app01:~$ ps aux --sort=-%cpu | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
bilal_0+    1918 99.9  0.0   5768  1044 tty1     R    18:57   2:40 yes
root         711  0.5  0.4 314816  9312 ?        Ssl  14:59   1:15 /usr/bin/vmtoolsd
root        1557  0.3  0.0      0     0 ?        I    16:27   0:32 [kworker/0:0-events]
root        1550  0.2  0.0      0     0 ?        I    16:27   0:25 [kworker/1:1-events]
bilal_002@app01:~$
```

ps aux

📸 **Evidence – `htop`: core 0 at 100 %, core 1 at 0 % – `yes` fills exactly one of the two cores; load average 1.09:**

```
  0[||||||||||||||||||||||||||||||||100.0%]   Tasks: 32, 39 thr; 2 running
  1[                                  0.0%]   Load average: 1.09 0.60 0.24
  Mem[|||||||||||||||                395M/1.88G]   Uptime: 04:02:03
  Swp[                                 0K/1.78G]

    PID USER       PRI  NI  VIRT   RES   SHR S CPU%▽MEM%   TIME+  Command
   1918 bilal_002   20   0  5768  1044   956 R 137.  0.1  3:33.42 yes
   1930 bilal_002   20   0  8236  4356  3464 R  0.9  0.2  0:00.15 htop
      1 root        20   0   98M 11620  8208 S  0.0  0.6  0:08.34 /sbin/init
    521 root        19  -1 47828 17912 16764 S  0.0  0.9  0:05.08 /lib/systemd/systemd-journald
```

htop

📸 **Evidence – `top`: load average `1.06, 0.63, 0.26` – about 1.0 over the last minute and still rising over 5 and 15 minutes; `2 running, 0 zombie`:**

```
top - 19:01:27 up  4:02,  1 user,  load average: 1.06, 0.63, 0.26
Tasks: 214 total,   2 running, 212 sleeping,   0 stopped,   0 zombie
%Cpu(s): 12.9 us, 20.7 sy,  0.0 ni, 66.4 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1927.6 total,    916.0 free,    393.4 used,    618.2 buff/cache
MiB Swap:   1824.0 total,   1824.0 free,      0.0 used.   1364.2 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1918 bilal_0+  20   0    5768   1044    956 R  99.7   0.1   3:58.04 yes
   1932 bilal_0+  20   0   10612   4080   3252 R   0.7   0.2   0:00.04 top
      1 root      20   0  100884  11620   8208 S   0.0   0.6   0:08.34 systemd
```

top

📸 **Evidence – Identify: PID 1918, PPID 1295; `echo $$` = 1295, so the parent is my bash; the family tree: `/bin/login` (console tty1) → `-bash` → `yes`. Over SSH, the parent of bash would be `sshd`:**

```
bilal_002@app01:~$ ps -eo user,pid,ppid,stat,%cpu,cmd --sort=-%cpu | head -5
USER         PID    PPID STAT %CPU CMD
bilal_0+    1918    1295 R    99.9 yes
root         711       1 Ssl    0.5 /usr/bin/vmtoolsd
root        1557       2 I      0.3 [kworker/0:0-events]
root        1550       2 I      0.2 [kworker/1:1-mpt_poll_0]
bilal_002@app01:~$ ps -o user= -p $(pgrep yes)
bilal_002
bilal_002@app01:~$ pgrep yes
1918
bilal_002@app01:~$ echo $$
1295
bilal_002@app01:~$ ps -ef --forest | grep -B3 "[y]es"
root         902       1  0 14:59 ?        00:00:00 /usr/libexec/udisks2/udisksd
root         911       1  0 14:59 tty1     00:00:00 /bin/login -p --
bilal_0+    1295     911  0 15:05 tty1     00:00:00  \_ -bash
bilal_0+    1918    1295 99 18:57 tty1     00:07:48      \_ yes
bilal_002@app01:~$
```

identify

📸 **Evidence – Stop politely with SIGTERM:**

```
root         911       1  0 14:59 tty1     00:00:00 /bin/login -p --
bilal_0+    1295     911  0 15:05 tty1     00:00:00  \_ -bash
bilal_0+    1918    1295 99 18:57 tty1     00:07:48      \_ yes
bilal_002@app01:~$ kill -15 $(pgrep yes)
```

kill 15

📸 **Evidence – Check: “gone”, no jobs left, the load average falling (`0.25`, 0.64, 0.43 – the 1-minute value drops first):**

```
bilal_002@app01:~$ pgrep yes || echo "gone"
gone
bilal_002@app01:~$ jobs
bilal_002@app01:~$ uptime
 19:07:52 up  4:08,  1 user,  load average: 0.25, 0.64, 0.43
bilal_002@app01:~$
```

gone

Lesson: read the output **above** a success message. When `pgrep` was typed without a name it failed with an error – and `||` printed “gone” anyway. Only `pgrep yes || echo "gone"` is real proof.

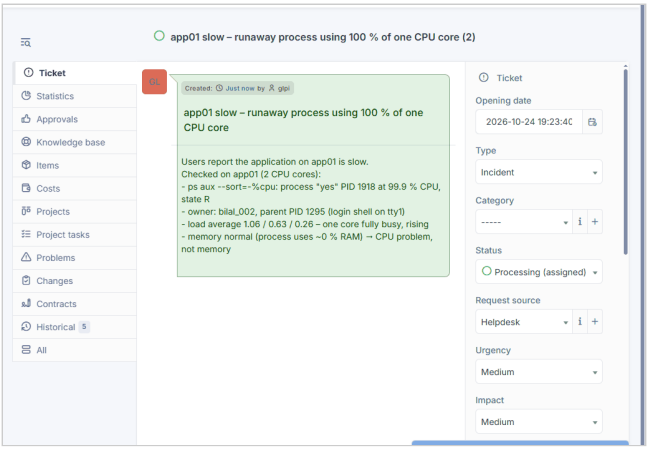
3.2 Record – GLPI ticket #2

📸 **Evidence – GLPI runs as an app + database pair on svc01: `glpi/glpi` publishes port 80 to the network; the database `mysql:8.0` shows only `3306/tcp` – not published, reachable only by GLPI inside Docker:**

```
bilal_004@svc01:~$ sudo docker ps
[sudo] password for bilal_004:
CONTAINER ID   IMAGE              COMMAND                  CREATED       STATUS          PORTS
                    NAMES
a2b8794fa4c9   glpi/glpi:latest   "/opt/glpi/entrypoin…"   7 days ago    Up 7 minutes    0.0.0.0:80->8
0/tcp, [::]:80->80/tcp   helpdesk-glpi-1
b721b3867130   mysql:8.0          "docker-entrypoint.s…"   7 days ago    Up 7 minutes    3306/tcp, 330
60/tcp                   helpdesk-db-1
bilal_004@svc01:~$ _
```

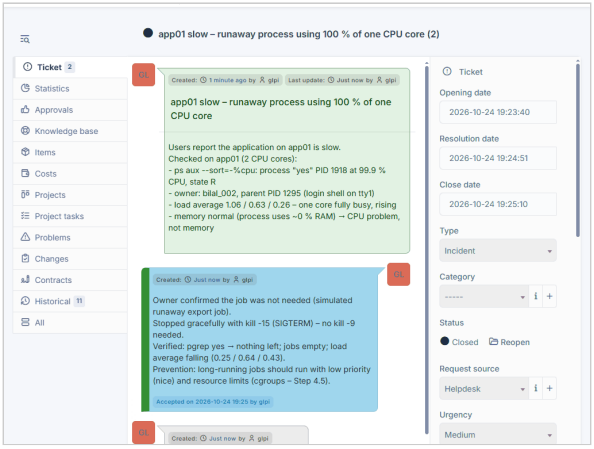
docker ps

📸 **Evidence – Incident ticket #2 created (Type Incident, Urgency/Impact Medium):**



ticket created

📸 Evidence – solution added and approved, ticket closed (opened 19:23:40, solved 19:24:51, closed 19:25:10 UTC):



ticket closed

Lesson: in a ticket, **Requester = who** reported it (a person), **Items = what** is broken (a computer from the inventory). In a company everyone works under a personal named account, not the built-in `glpi` super-admin.

Key understanding – the lab result (in my own words)

On app01 I first installed `htop` and `stress-ng`. The first attempt hung at `0% [Working]` with repeated `Ign:` lines because **gw01 was switched off**: gw01 is app01's only gateway to the internet (default route 192.168.60.1, NAT), so it is a single point of failure, and after powering it on the install worked. `nproc` showed **2 cores**. I created a runaway job with `yes > /dev/null &` (job [1], PID 1918). `ps aux --sort=-%cpu | head -5` put `yes` at the top with 99.9 % CPU, owner `bilal_002` (shown as `bilal_0+` because the name was cut off), state `R` (running), almost no memory (a CPU problem, not a memory problem) and a growing CPU TIME; without the minus in `--sort=-%cpu`, ps sorts lowest first and shows only idle processes. `htop` showed one core at 100 % and the other at 0 %, so `yes` fills exactly one of the two cores. `top` showed a load average of `1.06, 0.63, 0.26`: about 1.0 over the last minute (one core fully busy) and still rising over 5 and 15 minutes. `ps -eo user,pid,ppid,stat,%cpu,cmd` showed PID 1918 with PPID 1295, and `echo $$` printed 1295, so the parent of `yes` was my bash. `ps -ef --forest` showed the whole family: systemd (PID 1) → `/bin/login` (911, because I worked on the VMware console tty1; over SSH it would be `sshd`) → `-bash` (1295) → `yes` (1918). I stopped it gracefully with `kill -15` (SIGTERM): bash reported `[1]+ Terminated`, `pgrep yes || echo "gone"` printed "gone", `jobs` was empty, and `uptime` showed the load falling to `0.25, 0.64, 0.43`, the 1-minute value dropping first. I also learned to read the output above a success message: when I mistyped `pgrep` without a name, it failed with an error, and `||` printed "gone" anyway. Finally I recorded the incident in GLPI as ticket #2 – Type Incident, Requester = the person who reported it, Items = the computer that is broken – with the findings, the solution and a prevention note (low priority with `nice`, limits with cgroups), and closed it. `docker ps` on svc01 showed that GLPI is a real app + database pair: the `glpi/glpi` container publishes port 80, while its **MySQL 8.0** database shows only `3306/tcp`, not published, so users reach the app and only the app reaches the database.

Key understanding – the concept (in my own words)

1. The problem. The most common complaint about an application server is *"the application is slow"*. Very often the cause is one program using all of the server's CPU. To fix it, I must understand where the data lives, which job can cause the problem, and how to find and stop that job safely.

2. Where the data lives: files and records. Applications use two kinds of data. **Files** (uploads, photos, PDFs, CSV exports, backups) are kept on a **file server** and shared over the network with **NFS** (SMB on Windows); the application opens them as if they were on its own disk. **Records** (orders, customers, tickets) are rows in tables inside a **database**, such as MariaDB or PostgreSQL. The application always talks to a database with **SQL**, both to write ("save this order") and to read ("show me today's orders"), and never with NFS. A server whose main job is running database software is called a **database server**. One database program can hold many separate databases (for example `shop`, `hr` and `glpi`), and two different database programs can even run on the same server, because each uses its own port (MariaDB 3306, PostgreSQL 5432). Most applications come as an **app + database pair**: the app contains the pages, buttons and **business logic** (the rules of what happens, for example "when an order is placed, reduce the stock"), and the database stores the data.

3. How a real company makes reports, and how it goes wrong. Users work in the application (a web shop, ERP or MES) on the **application server**. It writes and reads the live data in the **primary database**, always with SQL and only on the primary. The primary first writes every change into its **change log**, a numbered diary on disk (PostgreSQL: the **WAL**, write-ahead log; MariaDB: the **binlog**), and then changes the table. It streams that log **continuously over the network** to a **replica**, a read-only copy of the **same kind** of database (PostgreSQL → PostgreSQL, MariaDB → MariaDB), which **replays** the same changes on its copy, normally less than a second behind. This is **replication**. Every night an automatic **ETL job** (Extract,

Transform, Load), started by a timer or cron, runs on a separate **job server**. It **extracts** millions of rows from the **replica**, **transforms** them (cleans and combines them) and **loads** them into a **data warehouse**, a separate database used only for reports. Moving data between different kinds of database like this is ETL, not replication. **Power BI** reads the data warehouse and shows dashboards to managers, such as sales, stock and KPIs. Typical jobs: *"export today's orders to a CSV file for accounting"*, *"create the monthly sales report"*, *"send new customers to another system every hour"*. This design keeps the heavy job away from the live system, so users feel nothing. If a company skips it, the heavy job slows the application in one of two ways. With a **direct effect**, the job runs **on the application server itself** and uses its CPU. With an **indirect effect**, the job reads the **primary database**, which becomes so busy that the application waits for its answers. A replica protects performance but is **not a backup**: if someone deletes data by mistake, the deletion is copied to the replica within seconds as well.

4. The Acme platform. Acme is smaller. **stor01 is a file and backup server, not a database server**: it holds `/data/app` (the customer uploads, on the LVM volume `lvdata`) and `/backups` (copies made by the nightly backup timer). app01 reads and writes these files through the NFS mount `/mnt/app`. The application and any export job **run on app01**, so a stuck job there is a **direct effect**: it uses app01's CPU, and the application becomes slow. Acme's real app + database pair is GLPI on **svc01**: every ticket, user and inventoried computer is a row in its **MySQL 8.0** database (a close relative of MariaDB), and GLPI reads and writes them with SQL. Only GLPI's port 80 is published to the network; the database port 3306 is reachable only by the GLPI container inside Docker, so users reach the app and only the app reaches the database.

5. Runaway job, hung job and the load average. A job causes trouble when it is **stuck in a loop**. A loop is a part of a program that repeats the same steps, such as "read the next row, write it, repeat", until it reaches its stop point, for example the last row. Because of a bug, the stop point never comes, so the job repeats forever at 100 % CPU, like a nightly export still running at 10 a.m. That is a **runaway job**: busy and out of control. A **hung job** is different: it uses almost no CPU because it is stuck **waiting**, like my Ctrl+Z waiting on NFS in Phase 3. A normal scheduled job, like my nightly backup, starts, finishes and stops. One **CPU core** runs one process at a time, and `nproc` shows how many cores a server has. The **load average** is the number of processes that want the CPU at the same moment, both running and waiting. `top` shows it as three numbers, the averages over the last **1, 5 and 15 minutes**. With 1 core, a load of 1.0 means fully busy, and 2.0 means one process is working while another waits. `3.0, 1.0, 0.2` means "busy only since a minute ago"; `3.0, 3.0, 3.0` means "busy for at least 15 minutes".

6. Processes, PID and PPID. Every running program is a **process** with its own number, the **PID**. The **PPID** is the PID of the process that **started** it, its parent. Processes form a family tree, shown by `ps -ef --forest`. **systemd** (PID 1) starts all services at boot, so services such as `sshd` have PPID 1 and no shell. When I log in over SSH, sshd starts a copy of itself for my session, and that copy starts my **shell**, bash, the program that reads the commands I type and runs them. Every command I start, for example `yes` or `systemctl restart sshd`, gets its **own PID**, and its **PPID is my bash**. Processes started by the same bash therefore share the same PPID.

7. Finding and stopping the job on app01, always in this order. Simulate → Look → Identify → Stop politely → Check → (kill -9 only as a last resort) → Record. Simulate: I first installed `htop` (a clearer live process viewer) and `stress-ng` (a test tool that creates CPU, memory or disk load on purpose, used later for limits). `yes > /dev/null &` creates a runaway job: `yes` prints the letter "y" without end, so it alone uses one full core; `>` sends this output to `/dev/null`, a special file where everything disappears and nothing is stored (it is not the `/bin` folder, which holds programs); and `&` runs it in the background. **Look:** `nproc` shows the number of cores, and `ps aux --sort=-%cpu | head -5`, `top` and `htop` list the processes using the most CPU. `yes` appears at the top with about 100 %, and the load average rises. **Identify:** `ps -eo user,pid,ppid,stat,%cpu,cmd` shows the **owner** (the user who started it), the PID, the PPID, the state (`R` = running), the CPU use and the **command** (`cmd`, the command line that started it, which shows what the program is). `ps -o user= -p $(pgrep yes)` shows only the owner, and `pgrep yes` shows only the PID. **Stop politely:** `kill -15` sends **SIGTERM**, a **graceful** stop: the program is asked to finish and clean up. **Check:** `pgrep yes || echo "gone"` prints "gone" when nothing is left (`||` means "if the first command finds nothing, run the second"). **Last resort:** `kill -9` sends **SIGKILL**, a **forceful** stop. The kernel removes the process at once and its clean-up never runs, so leftovers such as a lock file or a half-written file can remain; I use it only if a process ignores SIGTERM. **Record:** in a company, before stopping anything, I check who owns the process and whether it is important, asking the owner if needed. Afterwards I write a GLPI ticket: what happened, who owned the process, what I did and the result.

4. Step 4.2 – pipes: output → input → output

💡 **The story:** a pipe (`|`) passes the output of one program straight into the next as its input – no file in between. Admins use it every day, and it saves real disk space when moving big data: compress and send in one go, instead of first writing a temporary archive.

```
command 1 ──output──► | ──input──► command 2 ──output──► | ──input──► command 3 ──output──► screen
```

4.1 Hands-on – stream a compressed copy to another server (gw01 → stor01)

```
# gw01
sudo apt install -y pv
tar czf - /etc 2>/dev/null | pv | ssh stor01 "tar tzf - | wc -l"
```

📸 Evidence – 548 KiB streamed in 1 s; stor01 counted 1,563 files; nothing was stored on either server. `pv` shows `<=>` instead of % because a stream's size is not known in advance:

```
bilal_001@gw01:~$ tar czf - /etc 2>/dev/null | pv | ssh stor01 "tar tzf - | wc -l"
 548KiB 0:00:01 [ 283KiB/s] [   <=>
1563
bilal_001@gw01:~$
```

tar pv ssh

```
ps aux | sort -k3 -nr | head -3            # top 3 CPU users
grep bash /etc/passwd | wc -l              # how many users have a bash shell
pv shop.sql.gz | gunzip | mysql shop       # restore a dump with progress, no unpacked temp file
```

4.2 Reference only – named pipe (FIFO)

```
mkfifo /tmp/jobs                                                  # create the named pipe
ls -l /tmp/jobs                                                   # prw-r--r-- … 0 … /tmp/jobs
while read job < /tmp/jobs; do echo "processing: $job"; done      # terminal 1: the reader (worker)
echo "build-report" > /tmp/jobs                                   # terminal 2: the writer
rm /tmp/jobs                                                      # remove it

# the same transfer as above, as two separately started parts
mkfifo /tmp/stream
tar czf /tmp/stream /etc &                                        # writes "a file" — really the pipe
pv /tmp/stream | ssh stor01 "tar tzf - | wc -l"                   # reads it
rm /tmp/stream
```

Key understanding (in my own words)

A **pipe** (`|`) makes the output of one command the input of the next – **output → input → output** – with no file in between, like an assembly line, and only the last output reaches the screen. In `ps aux | sort -k3 -nr | head -3`, `ps` lists all processes, `sort` orders them by CPU and `head` keeps the top three. In real work pipes save time and disk space: on gw01, `tar czf - /etc | pv | ssh stor01 "tar tzf - | wc -l"` packed and compressed `/etc` into the pipe (`tar` packs many files into one stream: **c** create, **x** extract, **t** list, **z** gzip, **f -** = the pipe instead of a file), `pv` (**pipe viewer**) showed how much data passed and how fast (548 KiB in 1 second; a moving `<=>` instead of a percentage, because a stream's size is not known in advance), `ssh` carried the stream to stor01, and there `tar tzf -` listed what arrived and `wc -l` counted 1,563 files – nothing was stored on either server. With `tar xzf - -C /data` instead, stor01 would have unpacked it, which is a real copy to a new server without any temporary archive; the pipe saves the temporary copy, but the target still needs space for what is finally stored there. `pv` is also used to show progress and time left for long jobs like restoring a database dump (`pv shop.sql.gz | gunzip | mysql shop`). One bar `|` is a pipe; two bars `||` mean "or" – run the second command only if the first fails. A **named pipe (FIFO)** is a pipe with a name in the filesystem: a tube between two programs on the same server that stores nothing (size 0, type `p` in `ls -l`), hands data over in order and only once, and makes the writer wait until a reader is there. It is needed only when two programs cannot be joined with `|` in one command line – for example a tool that accepts only a file name, or two programs started separately. It is rare in daily admin work, so I kept it as a reference: when I see a `p` at the start of an `ls -l` line, I know it is a pipe, not a normal file.

Day 8 – 25 Oct: signals, lock, trap, queue system, worker, L1 → L2 escalation

1. Step 4.3 – the concept: two copies, signals, lock and trap

💡 **In plain words:** jobs that run by themselves – nightly backups, exports – must never run **twice at the same time**, and when they are stopped they must not leave broken files behind. Three tools solve this: a **lock**, **signals** and **trap**.

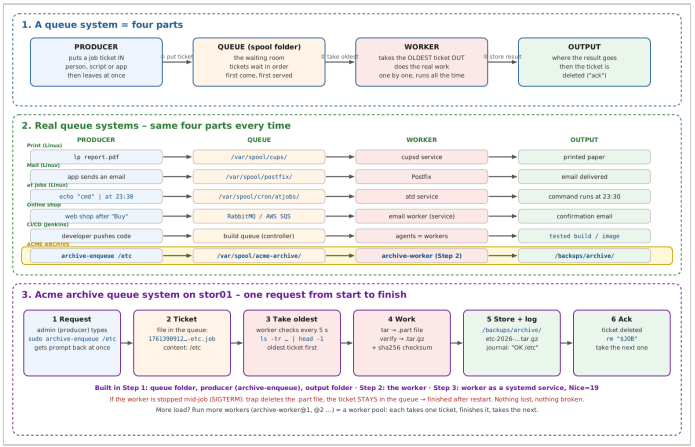
Key understanding (in my own words)

A nightly backup or export started by a **timer or cron** must never run **twice at the same time**: if the timer's backup runs from 02:00 to 03:00 and I start it by hand at 02:05, two copies would write to the same place and produce a broken result. The protection is a **lock** – a "someone is working on this right now" sign. Some programs have a lock **built in** (apt and dpkg – which is why a second apt, often Ubuntu's automatic updates running in the background, answers *"Could not get lock"*), but simple tools like `cp` or `rsync` and my own scripts have **none**, so I add it myself: a lock file checked at the start, `flock -n` (a kernel lock released automatically even after `kill -9`), or simply a systemd service, which never runs twice. A lock only stops a **second copy from starting**; it never stops anyone from terminating the job, and once the first copy has finished and removed its lock, the job can run again normally. A job is stopped by a **signal** from the kernel: **SIGTERM** (`kill -15`, "please stop"; `systemctl stop` sends it), **SIGINT** (Ctrl+C) or **SIGKILL** (`kill -9`, "stop now"). Stopping affects resources in two ways: **CPU and memory** are freed by the kernel whichever signal is used, but files the job created on **disk** – its lock, temporary files, a half-written archive – stay behind unless clean-up code removes them. **Linux processes have no clean-up by default**: the default action of SIGTERM is "die immediately", and SIGTERM only **gives the program a chance** to clean up. rsync, apt and databases have that code built in, while `cp`, `scp` and a plain Bash script do not – stopping a 20 GB `cp` halfway leaves a broken half-file that looks complete. **`trap`** adds clean-up code to a script: `trap cleanup SIGINT SIGTERM` means "on SIGTERM or Ctrl+C, first run my `cleanup` steps, then exit", and it runs only the steps I write. Trap does not stop or restart the process – the signal stops it, the trap cleans the disk before it ends, so the next start begins clean. **SIGKILL cannot be trapped**, so after `kill -9` the leftovers stay; a leftover lock is a **stale lock**, and the fix is always: prove nothing is running (`pgrep -af <job>`), remove the lock by hand, start the job again. Good jobs also write into a temporary **`.part` file** and rename it only when it is complete and verified, so a half-written file never looks finished.

2. Step 4.3 – the concept: queue systems, producers and workers

💡 **In plain words:** a **lock** says "busy, go away". A **queue with a worker** says "wait in line, I'll do it for you": everyone drops a request and leaves at once, and one program works through the requests one by one, at low priority, without losing any.

📸 **Architecture – a queue system has four parts; Linux already has several (print, mail, `at`), and the same pattern runs online shops and Jenkins:**



queue systems

Key understanding (in my own words)

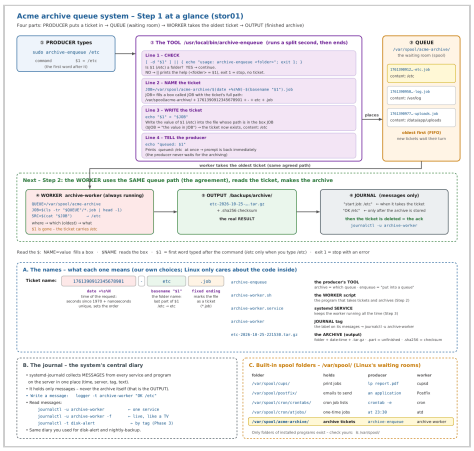
A **queue system** has four parts working together. A **producer** – a person, a script or an application – creates a **job ticket**, a small instruction such as *"archive /etc"*, places it in the **queue** and **gets control back at once**: it never waits for the job to be finished. The queue is a waiting room on disk called a **spool** folder; Linux keeps its own under **`/var/spool/`** – print jobs from `lp` wait in `/var/spool/cups/` for the worker `cupsd`, emails wait in `/var/spool/postfix/` for Postfix, `at` jobs wait in `/var/spool/cron/atjobs/` for `atd`. A **worker**, a program that runs all the time (usually a systemd service), takes the tickets **in order of arrival** – the oldest first, **FIFO**, first in, first out – does the work, stores the result in the agreed **output** place, and only then deletes the ticket (the **ack**, "done"), so if it fails or is stopped halfway the ticket stays and nothing is lost. Linux does not build a queue by itself: someone must place the tickets, and the worker must be written to take them out. Developers write workers; admins run and manage them – a systemd service with `Restart=on-failure`, more copies (a **worker pool**) when the load grows, logs in the journal, a safe stop with SIGTERM. Workers exist for three reasons: **nobody waits**, **the load stays under control**, **nothing**

is lost. The same pattern runs a web shop (the shop puts *“send the confirmation email for order 501”* into RabbitMQ or AWS SQS and an email worker sends it) and **Jenkins** (the controller keeps the build queue, *“Waiting for next available executor”*, and agents are the workers). A **timer** is driven by **time** – one fixed job at a fixed time; a **socket** answers each **live connection** at once while the caller waits; a **worker** is driven by **stored requests** – any job, any time, one by one, while the requester has already left. Real automation grows step by step: do it by hand, script it, schedule it, and finally let a worker do it for everyone – repeating the same proven steps like a tireless user, without deciding anything new as an AI agent would. **`nice`** answers a different question from the queue: the queue decides the **order**, nice (-20 most important … 19 politest) decides **how much CPU** a running job gets.

3. Lab – the Acme archive queue system on stor01

💡 **The story:** at Acme, admins and scripts request archives (“archive /etc before I change the config”, “archive the uploads”). Instead of running heavy `tar` jobs by hand, they drop a **ticket** into a queue, and one **archive worker** turns the tickets into verified archives one by one, at low priority, so the customer application is never slowed down.

📸 **Architecture – one request from start to finish, the names, the journal and the built-in spool folders:**



step 1 overview

3.1 Step 1 – the queue and the producer

```
# stor01
sudo mkdir -p /var/spool/acme-archive /backups/archive
sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }
JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
sudo chmod +x /usr/local/bin/archive-enqueue
```

📸 **Evidence:**

```
bilal_003@stor01:~$ sudo mkdir -p /var/spool/acme-archive /backups/archive
bilal_003@stor01:~$ sudo -v
bilal_003@stor01:~$ sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }

JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
bilal_003@stor01:~$ sudo bash -n /usr/local/bin/archive-enqueue
bilal_003@stor01:~$ sudo chmod +x /usr/local/bin/archive-enqueue
bilal_003@stor01:~$ _
```

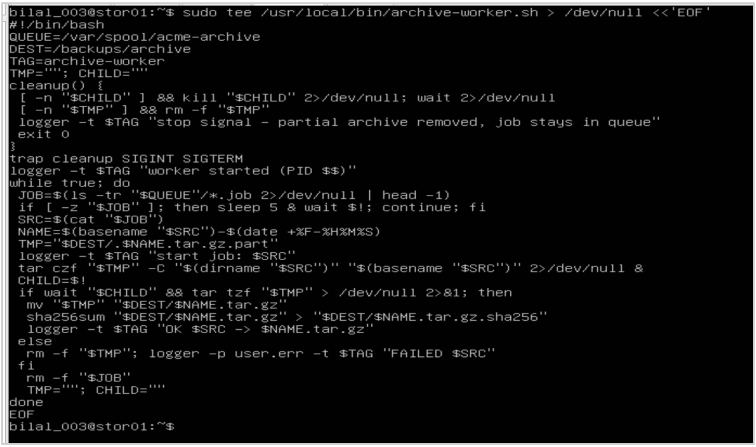
step 1

Lesson: my first version had `data` instead of `date`. `bash -n` did not catch it – it checks only the grammar of a script, not whether its commands exist. Only a test run proves a script works.

3.2 Step 2 – the worker

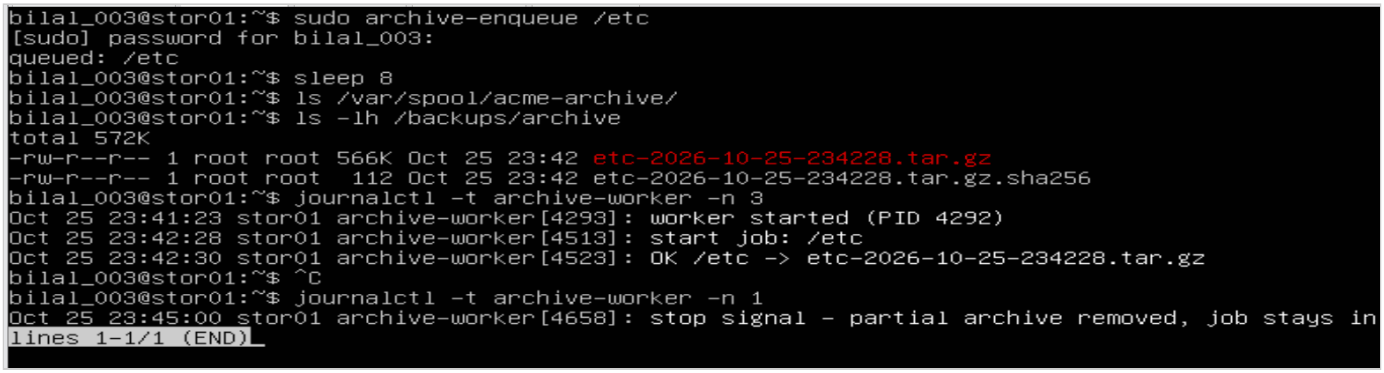
```
        QUEUE=/var/spool/acme-archive                          # WHERE — the agreed queue folder (same path as the
producer)
        DEST=/backups/archive                                  # OUTPUT
        trap cleanup SIGINT SIGTERM                            # TRAP — on SIGTERM/Ctrl+C: stop tar, delete the .part
file
        while true; do                                         # WORKER — runs forever
          JOB=$(ls -tr "$QUEUE"/*.job 2>/dev/null | head -1)   # WHICH — the oldest ticket first (FIFO)
          SRC=$(cat "$JOB")                                    # WHAT — read the folder from the ticket
          TMP="$DEST/.$NAME.tar.gz.part"                       # SAFE FILE — work into a .part file
          logger -t $TAG "start job: $SRC"                     # JOURNAL — message only
          mv "$TMP" "$DEST/$NAME.tar.gz"                       # OUTPUT — only a complete, verified archive gets the
real name
          logger -t $TAG "OK $SRC -> $NAME.tar.gz"             # JOURNAL — "OK" only after the archive is stored
          rm -f "$JOB"                                         # ACK — delete the ticket
        done
```

📸 **Evidence – the worker script:**



worker script

📸 **Evidence – manual test: worker running in one terminal; in the other `queued: /etc` at once, the queue empty again (ack), `etc-2026-10-25-234228.tar.gz` (566 K) + `.sha256` in the output, the journal *worker started* → *start job* → *OK* within 2 seconds; Ctrl+C → the trap's *"stop signal"* message:**



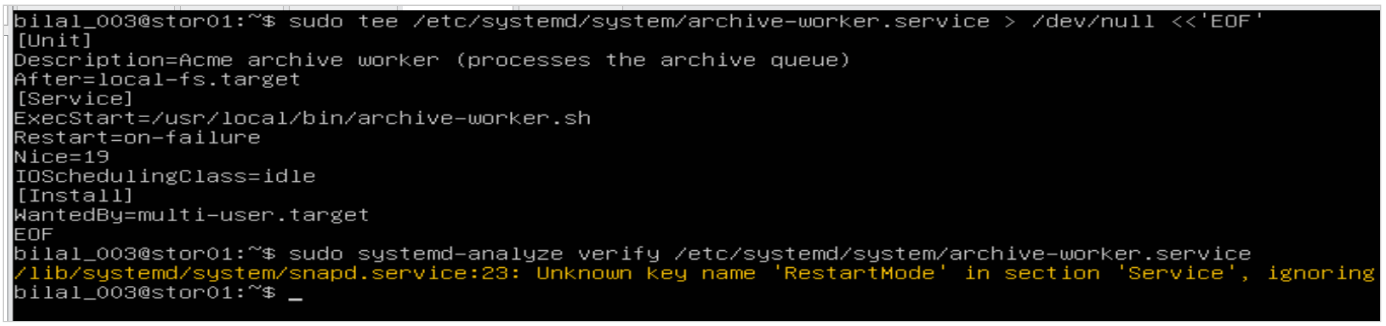
enqueue archive journal trap

Lessons: I first typed the worker by hand with five mistakes – `backups/archive` without the leading `/`, `[ -n "$CHILD"]` without the space before `]`, `SIGNIT`, `sha56sum` and, worst, **`*.JOB` instead of `*.job`**: Linux is case-sensitive, so the worker would have waited forever without any error. Long scripts are pasted, not retyped. The number in `archive-worker[4513]` is the PID of each `logger` call; the worker's own PID is the one in the message (4292).

3.3 Step 3 – the worker as a service, at low priority

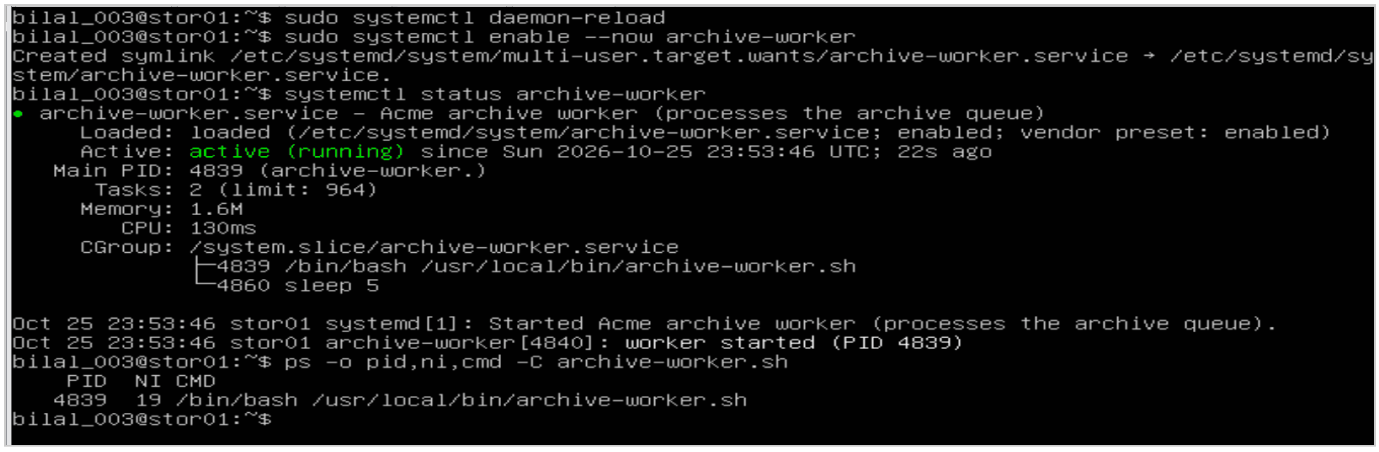
```
sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
sudo systemctl daemon-reload
sudo systemctl enable --now archive-worker
systemctl status archive-worker --no-pager
ps -o pid,ni,cmd -C archive-worker.sh
```

📸 **Evidence – the unit; `verify` finds nothing in my file (the yellow line is Ubuntu's own `snapd.service`):**



service unit

📸 **Evidence – `active (running)`, Main PID 4839, 1.6 MB; the cgroup holds the worker and its child `sleep 5` (empty queue, waiting); NI 19:**



service running

Key understanding – the lab so far (in my own words)

On stor01 I built the **Acme archive queue system**. In **Step 1** I created the **queue** `/var/spool/acme-archive/` (the waiting room for job tickets), the **output** `/backups/archive/` and the **producer's tool** `archive-enqueue` in `/usr/local/bin`, the folder for an admin's own commands. When I type `sudo archive-enqueue /etc`, `$1` – a box filled with the first word after the command – becomes `/etc`; `[ -d "$1" ]` checks it is a folder (otherwise `||` prints the help and `exit 1` stops); `JOB=` builds the ticket's path from the queue folder, the time of the request (`date +%s%N`) and the folder name (`basename`); `echo "$1" > "$JOB"` writes the folder into that ticket file, and *"queued: /etc"* comes back at once. The tool's name is only a label for humans – "enqueue" is the standard word for "put into a queue" – and what it does is decided only by its code. In **Step 2** I created the **worker** `archive-worker.sh`. It knows where to look because the **same queue path** is written at its top – producer and worker simply agree on it, as `cupsd` reads its spool folder from its settings file – and it reads the folder from the ticket with `cat`, because `$1` exists only for the split second the producer runs. In a manual test it turned my ticket into `etc-2026-10-25-234228.tar.gz` with a `.sha256` checksum within 2 seconds, wrote *start job* and *OK* to the **journal** (the server's central diary, which holds only messages, never the archive itself), deleted the ticket (the ack), and on Ctrl+C its trap logged *"stop signal"* before exiting. In **Step 3** I made it a **systemd service** with `Restart=on-failure`, `Nice=19` and `IOSchedulingClass=idle`; it now runs all the time as PID 4839 at **NI 19**, its cgroup showing it waiting (`sleep`

5) for the next ticket. Because a service runs only once, systemd itself is the lock – no second worker can start.

3.4 Step 4 – three jobs in the queue

```
# Terminal 1 (stor01 console): journalctl -u archive-worker -f
# Terminal 2 (ssh stor01 from gw01):
df -h /backups
sudo archive-enqueue /data/app/uploads
sudo archive-enqueue /etc
sudo archive-enqueue /var/log
ls /var/spool/acme-archive/
journalctl -t archive-worker -n 10
```

📸 **Evidence – three tickets waiting, named by request time so the oldest comes first:**

```
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  701M  2.1G  26% /backups
bilal_003@stor01:~$ sudo archive-enqueue /data/app/uploads
[sudo] password for bilal_003:
queued: /data/app/uploads
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sudo archive-enqueue /var/log
queued: /var/log
bilal_003@stor01:~$ ls /var/spool/acme-archive
1792974483564978473-uploads.job  1792974503083839724-etc.job  1792974511474086151-log.job
bilal_003@stor01:~$ ps -o pid,ni,cmd -C tar
    PID  NI CMD
bilal_003@stor01:~$
```

three tickets

📸 **Evidence – the live `-u` view shows *start job: /data/app/uploads* but not its *OK*:**

```
bilal_003@stor01:~$ journalctl -u archive-worker -f
Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
```

journal -u

📸 **Evidence – `journalctl -t` (by tag) shows every message: uploads 00:28:08 → OK 00:29:04 (56 s), `/etc` 1 s, `/var/log` 11 s – strictly in arrival order (FIFO), one at a time:**

```
bilal_003@stor01:~$ journalctl -t archive-worker -n 10
Oct 25 23:42:28 stor01 archive-worker[4513]: start job: /etc
Oct 25 23:42:30 stor01 archive-worker[4523]: OK /etc -> etc-2026-10-25-234228.tar.gz
Oct 25 23:45:00 stor01 archive-worker[4658]: stop signal - partial archive removed, job stays in qu>
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
Oct 26 00:28:08 stor01 archive-worker[6564]: start job: /data/app/uploads
Oct 26 00:29:04 stor01 archive-worker[6586]: OK /data/app/uploads -> uploads-2026-10-26-002808.tar.>
Oct 26 00:29:04 stor01 archive-worker[6594]: start job: /etc
Oct 26 00:29:05 stor01 archive-worker[6604]: OK /etc -> etc-2026-10-26-002904.tar.gz
Oct 26 00:29:05 stor01 archive-worker[6612]: start job: /var/log
Oct 26 00:29:16 stor01 archive-worker[6622]: OK /var/log -> log-2026-10-26-002905.tar.gz
lines 1-10/10 (END)
```

journal -t

Lesson: `logger` runs as a very short-lived process; sometimes it ends before the journal can link the message to the **service**, so `journalctl -u archive-worker` can miss a line. **`journalctl -t archive-worker`** filters by the **tag** set with `logger -t` and always shows them all.

3.5 Step 5 – a request at any time: worker vs timer

```
sudo archive-enqueue /etc        # "archive /etc NOW – before I change the config"
journalctl -t archive-worker -n 2
```

📸 **Evidence – queued, started and finished in the same second (00:34:12) – no waiting for a 02:00 timer:**

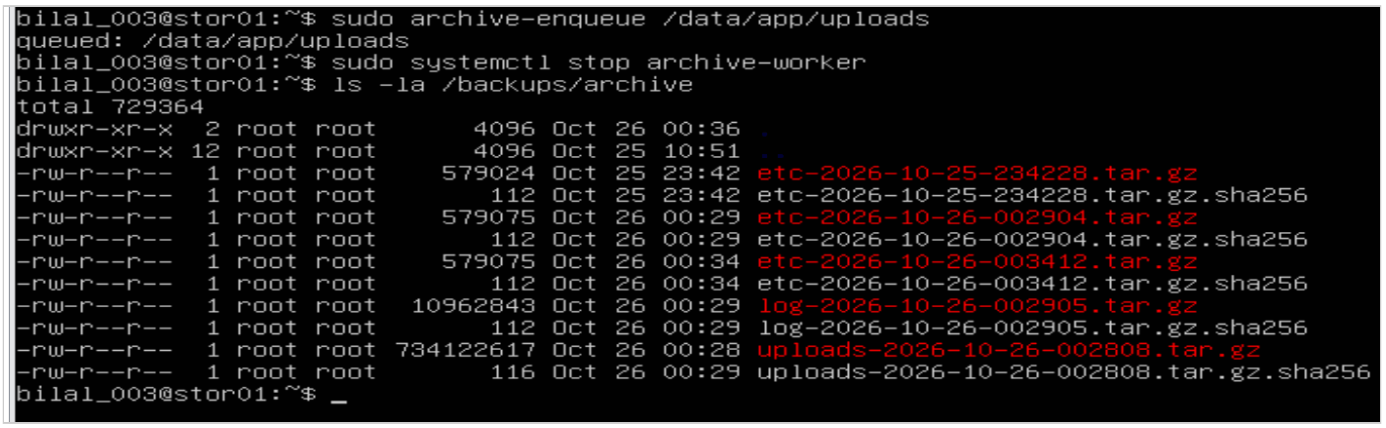
```
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sleep 3
bilal_003@stor01:~$ journalctl -t archive-worker -n 2
Oct 26 00:34:12 stor01 archive-worker[6922]: start job: /etc
Oct 26 00:34:12 stor01 archive-worker[6932]: OK /etc -> etc-2026-10-26-003412.tar.gz
bilal_003@stor01:~$ _
```

any time

3.6 Step 6 – stop the worker in the middle of a job (the trap test)

```
sudo archive-enqueue /data/app/uploads
sleep 5
sudo systemctl stop archive-worker          # SIGTERM while tar is compressing 700 MB
ls -la /backups/archive/                    # no .part file
ls /var/spool/acme-archive/                 # the ticket is still there
sudo systemctl start archive-worker
```

📸 **Evidence – after the stop: only complete archives in the output, no `.part` file:**

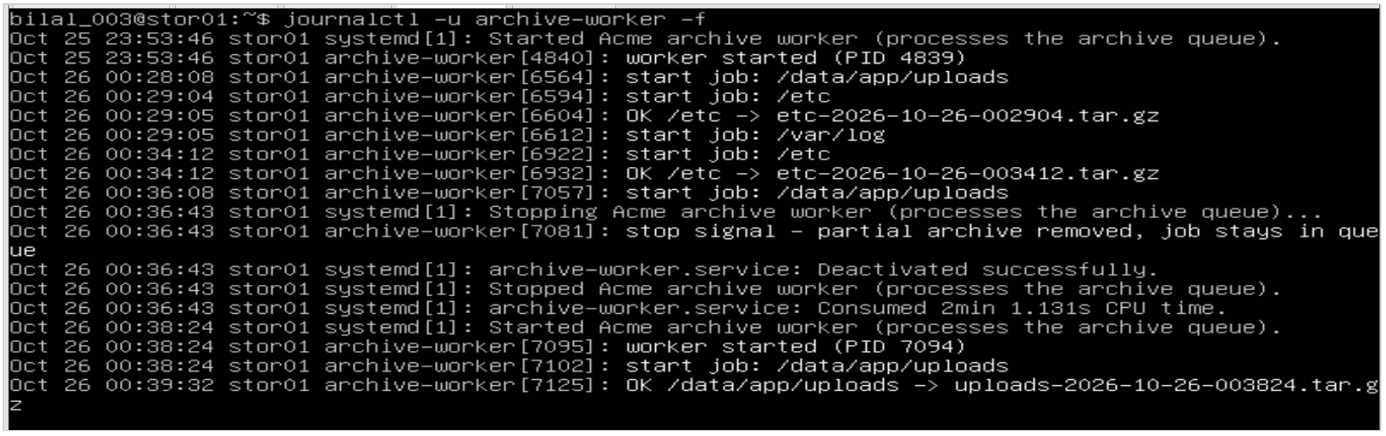


no part file

📸 **Evidence – the ticket stayed in the queue; the journal shows the trap's message:**

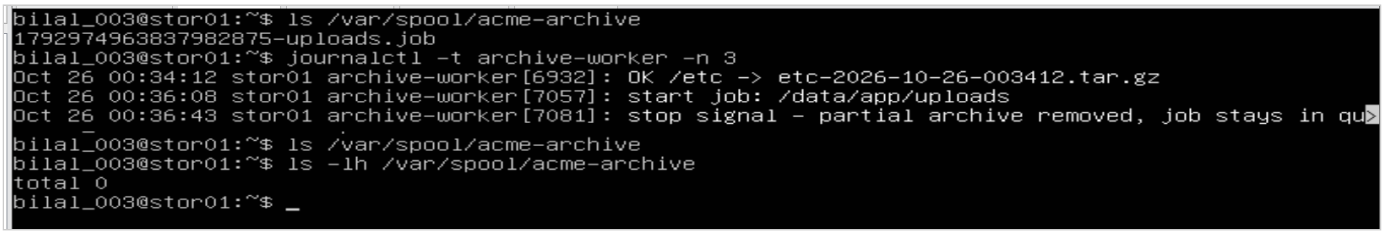
ticket stays

📸 **Evidence – the full story in the journal: start 00:36:08 → *Stopping…* → *stop signal - partial archive removed, job stays in queue* → *Deactivated successfully* (clean exit) → restart 00:38:24 (new PID 7094) → the same job → *OK* 00:39:32:**



stop restart ok

📸 **Evidence – the queue empty again after the restart:**



queue empty

3.7 Step 7 – a deliberate escalation: L1 finds it, L2 fixes it

💡 **The story:** a colleague left manual database dumps in `/backups`, filling it. The archive worker fails. As **L1** I find the cause and collect evidence, but I am not allowed to delete someone else's data or resize volumes – so I **escalate**. Then, as **L2 – Linux Storage**, I get the owner's approval, fix it and close the ticket.

```
# create the problem
sudo fallocate -l $(df --output=avail -B1 /backups | tail -1) /backups/db-dump-manual.sql
sudo archive-enqueue /etc                              # still OK — see the lesson below
sudo fallocate -l $(( $(stat -f -c '%f' /backups) * $(stat -f -c '%S' /backups) )) /backups/db-dump-manual-2.sql
sudo archive-enqueue /var/log                          # now FAILED

# L1 — look, identify, do not fix outside the role
journalctl -t archive-worker -n 3
df -h /backups
sudo du -sh /backups/* | sort -h | tail -3
ls -l /backups/db-dump-manual*.sql
```

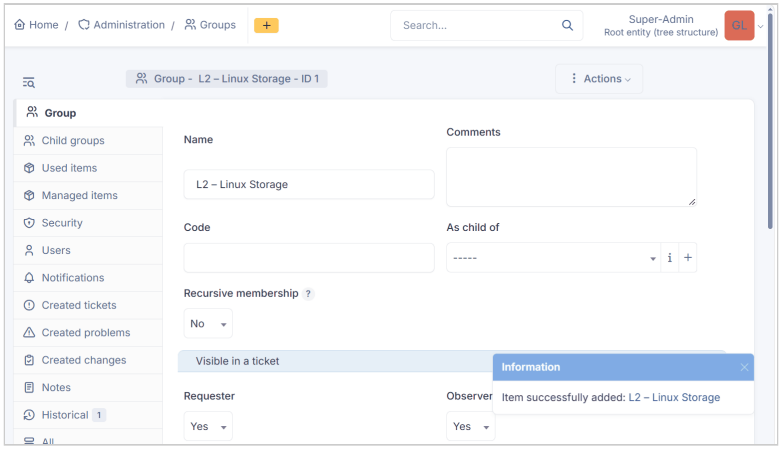
📸 **Evidence – L1: `FAILED /var/log`, `/backups` 100 %, the two dump files (owner root, 00:44 and 00:52):**

```
bilal_003@stor01:~$ sudo fallocate -l $(df --output=avail -B1 /backups | tail -1) /backups/db-dump-m
anual.sql
bilal_003@stor01:~$ sudo archive-enqueue /etc
queued: /etc
bilal_003@stor01:~$ sleep 5
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  2.8G     0 100% /backups
bilal_003@stor01:~$ sudo fallocate -l $(($(stat -f -c '%f' /backups)* $(stat -f -c '%S' /backups)))
/backups/db-dump-manual-2.sql
fallocate: fallocate failed: No space left on device
bilal_003@stor01:~$ df -h /backups
Filesystem                  Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  2.9G     0 100% /backups
bilal_003@stor01:~$ sudo archive-enqueue /var/log
queued: /var/log
bilal_003@stor01:~$ sleep 5
bilal_003@stor01:~$ journalctl -t archive-worker -n 3
Oct 26 00:44:38 stor01 archive-worker[7408]: OK /etc -> etc-2026-10-26-004437.tar.gz
Oct 26 00:53:25 stor01 archive-worker[7872]: start job: /var/log
Oct 26 00:53:25 stor01 archive-worker[7879]: FAILED /var/log
bilal_003@stor01:~$ sudo du -sh /backups/* | sort -h | tail -3
669M    /backups/db-dump-manual.sql
701M    /backups/2026-10-21
1.4G    /backups/archive
bilal_003@stor01:~$ ls -l /backups/db-dump-manual*.sql
-rw-r--r-- 1 root root 160464896 Oct 26 00:52 /backups/db-dump-manual-2.sql
-rw-r--r-- 1 root root 701198336 Oct 26 00:44 /backups/db-dump-manual.sql
bilal_003@stor01:~$
```

l1 evidence

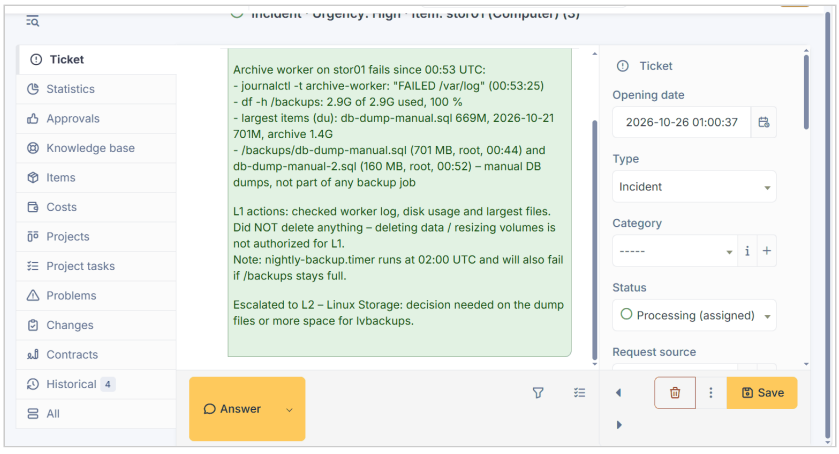
Lesson – the 5 % root reserve: after the first dump `df` already showed **Avail 0 / 100 %**, yet the `/etc` archive still succeeded. ext4 keeps about **5 % reserved for root**; `df`'s "Avail" and "Use%" count only the space normal users can have, and the worker runs as root. Only after the second file filled **all** free blocks (`stat -f`) did the job fail. In a real incident this is why root services often keep working a little longer while users already see *"No space left on device"*.

📸 **Evidence – the L2 group:**



group

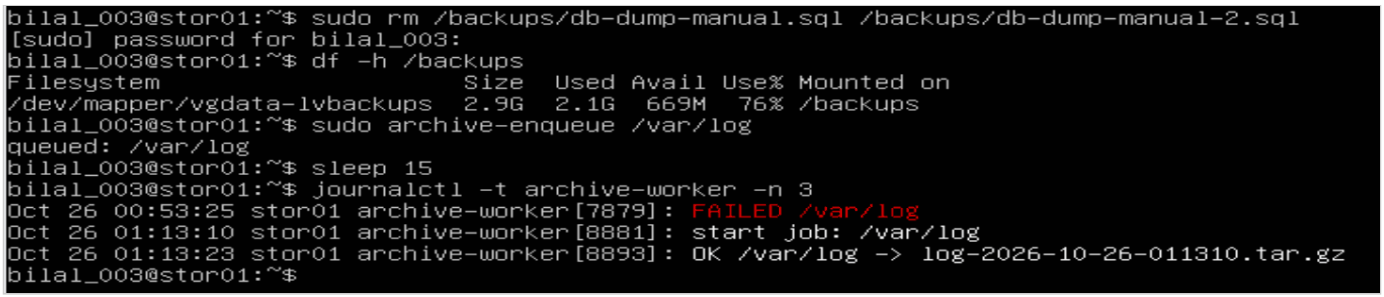
📸 **Evidence – ticket #3 escalated: Incident · Urgency High · Item stor01 · Processing (assigned) to the L2 group:**



ticket 3

```
sudo rm /backups/db-dump-manual.sql /backups/db-dump-manual-2.sql
df -h /backups                          # 76 %, 669 MB free
sudo archive-enqueue /var/log           # re-queue the failed job
journalctl -t archive-worker -n 3
```

📸 **Evidence – space back, the failed job re-queued and `OK /var/log` at 01:13:23:**



l2 fix

Lesson – a lost job: the `/var/log` ticket disappeared after `FAILED`, because the worker deletes the ticket after a failure too. Real queue systems move failed jobs to a **failed queue** (a *dead-letter queue*) so they can be retried. That is a design improvement for **L3**.

Key understanding – Steps 4 to 7 (in my own words)

> With the worker running as a service, I queued three requests – `/data/app/uploads` (about 700 MB), `/etc` and `/var/log`. Each `archive-enqueue` answered *"queued"* at once, and the worker processed them strictly **in arrival order (FIFO)** and **one at a time**: uploads in 56 seconds, `/etc` the moment it finished, then `/var/log`; the later tickets simply waited. `journalctl -u archive-worker` missed one *OK* line, because a message from a very short-lived `logger` process cannot always be linked to the service, while `journalctl -t archive-worker` – filtering by the tag – showed every message. A request at any time (*"archive /etc now"*) was queued, started and finished within the same second, something a 02:00 timer cannot do. Then the real test: I stopped the worker with `systemctl stop` while it was compressing 700 MB. systemd sent SIGTERM, the worker's **trap** logged *"stop signal – partial archive removed, job stays in queue"*, the service was *Deactivated successfully*, `/backups/archive/` contained **no `.part` file**, and the uploads ticket was **still in the queue** – because the ticket is deleted only after success. After `systemctl start`, the new worker took that same ticket and finished it: nothing broken, nothing lost. Finally I practised a deliberate **escalation**. Two forgotten manual database dumps filled `/backups` – the first only up to the **5 % root reserve**, so `df` showed 100 % while the root-run worker could still write, and only the second made the job fail with *FAILED /var/log*. As **L1** I found the cause with `journalctl`, `df`, `du` and `ls`, did **not** delete someone else's data or resize the volume, and opened a High-urgency GLPI incident with all evidence, assigned to the group **L2 – Linux Storage**. As **L2** I recorded the owner's approval as a followup, removed the dumps (76 % used again), re-queued the failed job (*OK /var/log*), and wrote the solution – cause, decision, fix, verification, prevention – before closing the ticket. The failed `/var/log` ticket had been deleted by the worker, which showed a real design gap: production queues keep failed jobs in a **dead-letter queue** for retry. **L1's job is not to fix everything, but to find the cause, collect clear evidence, stay within its authority and hand over well; L2 fixes on the servers; L3 removes the root cause.**

Day 9 – 26 Oct: clean-up · how processes are born – PID, fork, exec, wait, orphan, zombie

1. Step 4.3 Step 9 – clean-up after the queue lab (stor01)

💡 **In plain words:** after the escalation drill of Day 8 the backup volume must be back to normal, and the real nightly backup must still work.

```
        # stor01
        journalctl -u nightly-backup.service --since "today" --no-pager | tail -5
        sudo rm -r /backups/archive/*
        df -h /backups
```

📸 **Evidence:**

```
bilal_003@stor01:~$ journalctl -u nightly-backup.service --since "today" --no-pager | tail -5
Oct 26 02:00:07 stor01 systemd[1]: Starting Nightly backup of /data/app...
Oct 26 02:00:07 stor01 nightly-backup[11231]: backup of /data/app to /backups/2026-10-26 finished
Oct 26 02:00:07 stor01 systemd[1]: nightly-backup.service: Deactivated successfully.
Oct 26 02:00:07 stor01 systemd[1]: Finished Nightly backup of /data/app.
bilal_003@stor01:~$ df -h /backups
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvbackups  2.9G  701M  2.1G  26% /backups
bilal_003@stor01:~$
```

Nightly backup at 02:00:07 succeeded; /backups back to 26 %

2. Step 4.4 – the concept: where processes and PIDs come from

2.1 What is a process and a PID?

💡 **In plain words:** a **process** is a program that is running right now – my bash, `ls`, `tar`, sshd, the archive worker. Every process gets a **PID** (its own number) and a **PPID** (the number of its parent, the process that started it). **Every process has a parent**, all the way up to **systemd, PID 1**.

2.2 The PID pool

```
        cat /proc/sys/kernel/pid_max      # size of the pool
        cat /proc/loadavg                 # last number = the PID given most recently
        ps -e --no-headers | wc -l        # how many PIDs are in use right now
```

2.4 From boot to my shell on app01

```
kernel → systemd (1) → getty@tty1 —exec→ login (911) —fork→ bash (1295) → my commands
```

```
while I work:  ls ends   → leaves an exit note → bash reads it  → back to the prompt
when I exit:   bash ends → leaves an exit note → login reads it → login ends
               → systemd starts a fresh getty → new "login:" screen (new PIDs)
```

2.5 Why fork + exec? bash works through a representative

```
bash (1295) ── asks the KERNEL "copy me" (fork) ──► representative (new PID 4810)
     │                                        exec ──► becomes ls (still 4810)
     │                                finishes → leaves the exit note
     └─ told (SIGCHLD) → READS the note (wait) → 4810 freed → prompt back
```

3. Hands-on on app01 – watching processes being born

3.1 Who am I?

```
        echo $$                   # $$ = the PID of this shell
```

📸 **Evidence:**

echo $$ prints 1295, the PID of my bash

3.2 My family tree

```
        pstree -ps $$             # -p = show PIDs, -s = show all parents
```

📸 **Evidence:**

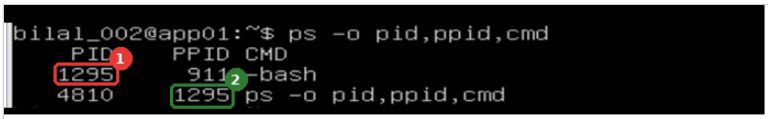
1. systemd (PID 1) – the first process, ancestor of all
2. login – started my bash when I logged in
3. my bash (1295) – me
4. pstree – my child, born when I pressed Enter

systemd(1) → login(911) → bash(1295) → pstree(4808)

3.3 Watch a birth

```
        ps -o pid,ppid,cmd          # ps = process status, -o = which columns
```

📸 **Evidence:**



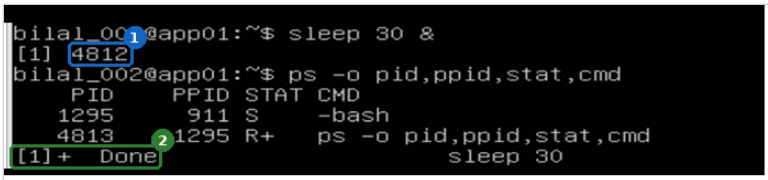
1. my bash = PID 1295
2. ps has PPID 1295 → ps is a CHILD of my bash (fork + exec)

1295 911 -bash, and ps with PPID 1295

3.4 A child in the background – and a good parent

```
        sleep 30 &                  # & = run in the background, prompt comes back at once
        ps -o pid,ppid,stat,cmd
```

📸 **Evidence:**



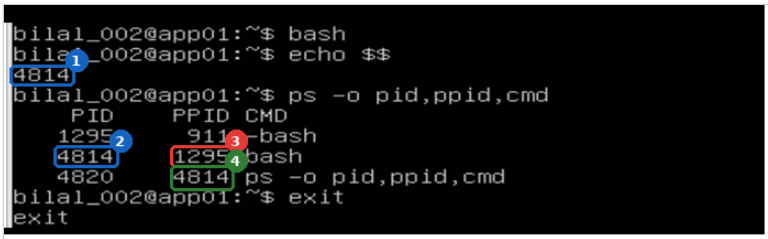
1. sleep started in the background with PID 4812 (& = give me the prompt back)
2. sleep is not in the list: I ran ps after more than 30 s. bash READ the exit note → "Done" → no zombie (good parent)

sleep 4812 in the background; [1]+ Done – no zombie

3.5 A child of a child

```
        bash                        # a new bash inside my bash — waits for me until exit
        echo $$
        ps -o pid,ppid,cmd
        exit
```

📸 **Evidence:**



1. new bash inside my bash got PID 4814
2. new bash 4814 ...
3. ... has parent 1295 (my first bash) = child
4. ps has parent 4814 = GRANDCHILD: 1295 → 4814 → 4820. "exit" closed the inner bash.

new bash 4814 with parent 1295; ps 4820 with parent 4814

3.6 exec – the copy turns into another program (reference)

```
bash -c 'echo "my PID is $$"; exec sleep 30' &     # -c = run these commands, then end
ps -o pid,ppid,cmd                                 # sleep has the SAME PID that echo printed
```

3.7 Orphan – the parent dies, the child keeps running

```
bash -c 'sleep 60 &'       # the inner bash starts sleep, then ends at once
ps -o pid,ppid,stat,cmd -C sleep
```

📸 **Evidence:**

1. sleep 60, PID 4830 – its parent (inner bash) already exited
2. PPID = 1 → systemd ADOPTED it
3. STAT S = still alive (sleeping), not Z → ORPHAN, not zombie

sleep 60 with PPID 1 and STAT S – an orphan adopted by systemd

5. Zombies – essentials

📸 **Reference (example output from the lab guide – PID numbers differ on a real server):**

EXAMPLE - what app01 should show (your PID numbers will be different)

```
bilal_002@app01:~$ ps -eo pid,ppid,stat,cmd | grep -E "STAT| Z|defunct"
    PID    PPID STAT CMD
   5013    5012 Z    [python3] <defunct>
   5016    1295 S+   grep --color=auto -E STAT| Z|defunct
bilal_002@app01:~$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   1295     911 Ss   -bash
   5012    1295 S    python3 /home/bilal_002/zombie.py
   5013    5012 Z    [python3] <defunct>
   5020    1295 R+   ps -o pid,ppid,stat,cmd
```

1. the zombie child, PID 5013
2. its parent is 5012 = python3 zombie.py
3. STAT Z = zombie
4. <defunct> = "dead". The process is finished, only its exit note waits
5. family view: bash 1295 → python3 5012 (alive, sleeping) → 5013 (Z, dead)

Example: a zombie in ps – STAT Z, defunct, PPID points to the parent

EXAMPLE - what app01 should show (your PID numbers will be different)

```
bilal_002@app01:~$ kill 5012
[1]+  Terminated              python3 ~/zombie.py
bilal_002@app01:~$ ps -eo pid,ppid,stat,cmd | grep defunct
   5027    1295 S+   grep --color=auto defunct
bilal_002@app01:~$ top -bn1 | head -3 | grep Tasks
Tasks: 110 total,   1 running, 109 sleeping,   0 stopped,   0 zombie
bilal_002@app01:~$ rm ~/zombie.py
```

1. stop the PARENT (5012), the real fix
2. only the grep line itself is left → the zombie is GONE. systemd (PID 1) adopted it and read its exit note at once
3. top: "0 zombie"

Example: after stopping the parent the zombie is gone – top shows 0 zombie

6. Key understanding (in my own words)

Every running program is a **process** with a **PID** (its own number) and a **PPID** (its parent's number). The kernel hands out PIDs like a **ticket machine**, in **increasing order**, up to `pid_max` (about 4.2 million on Ubuntu); a freed number is reused only after the counter wraps around. At boot the kernel starts **only systemd (PID 1)**, and systemd starts every service – including **getty**, a process that watches the terminal **tty1** and shows "login:". When I type my username, getty **execs** into **login** (same PID); after the password, login **forks** a copy, the copy execs into my **login shell** `-bash`, and login waits in the background as its parent. bash never runs a program itself: it asks the **kernel** to make a **copy** of it with a **new PID** (**fork**), the copy turns into the command and keeps that PID (**exec**), and bash stays alive; built-ins like `cd` and `echo` are the exception – bash does them itself. When a child ends, it automatically leaves an **exit note** (its exit code, `echo $?`); the kernel tells the parent with **SIGCHLD**, the parent **reads** the note (**wait**), and the PID is freed. `exit` closes the chain the same way: bash ends, login reads its note and ends, and systemd starts a fresh getty. An **orphan** is a child that is **still running** after its parent died – systemd adopts it (PPID 1). A **zombie** is a child that has **finished** but whose parent **never read the note** – a bug (no `wait()`), a stuck parent, or a container without init. It shows **STAT Z,** `<defunct>`, uses no CPU or memory and only holds its PID – rare, but thousands of zombies can use up all PIDs, and then nothing new can start, not even an SSH login. `kill -9` cannot remove a zombie because it is already dead; **the fix is the parent** – find it through the zombie's PPID, restart that service, and systemd adopts the zombie and cleans it up, then open a ticket for the developers.

Day 10 – 1 Nov: cgroups, cache and swap, container networking, where limits are set and stored

1. Warm-up – the PID pool on gw01

```
cat /proc/sys/kernel/pid_max      # 4194304 — size of the PID pool
cat /proc/loadavg                 # 0.01 0.07 0.03 1/238 1823
ps -e --no-headers | wc -l        # 209 processes
```

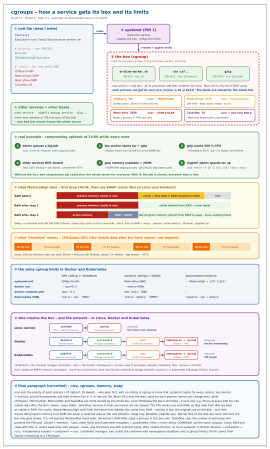
2. Step 4.5 – the concept: cgroups

💡 **In plain words:** `nice` decides **who goes first**; a **cgroup** decides **how much at most**. Every service runs in its own **box**, and a limit on the box is a hard ceiling for every process inside it – so one bad job cannot freeze the whole server.

2.4 The archive worker as an example (stor01)

```
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Nice=19                  # priority — per process
IOSchedulingClass=idle
CPUQuota=30%             # limits — for the whole box
MemoryHigh=150M
MemoryMax=200M
TasksMax=20
```

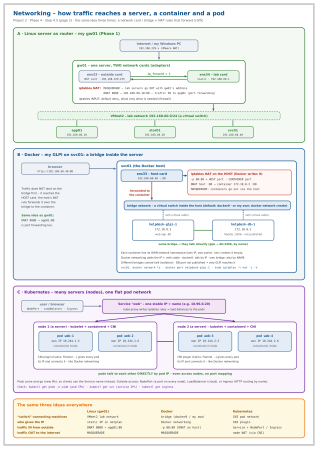
📸 **Diagram:**



cgroups – how a service gets its box and its limits

3. Container networking – the same idea as gw01

📸 **Diagram:**



Networking – Linux, Docker and Kubernetes

4. Key understanding (in my own words)

nice sets the priority of each process (-20 highest, 19 lowest) – who goes first, with no ceiling. A **cgroup** is a box that **systemd** creates for every service: one service = one box, and all its processes and their children live in it. In the unit file, `Nice=19` is only the start value for each process (`renice` can change one), while `CPUQuota`, `MemoryHigh`, `MemoryMax` and `TasksMax` are limits shared by the whole box. Over **CPUQuota** the box is **throttled** – it runs only e.g. 30 ms of every 100 ms, like mobile data after the limit: slower, never killed – and other services in their own boxes are not slowed. The CPU works only with **RAM**, so files read from disk are kept as copies in RAM, the **cache**. Above **MemoryHigh** (soft limit) the kernel first deletes this

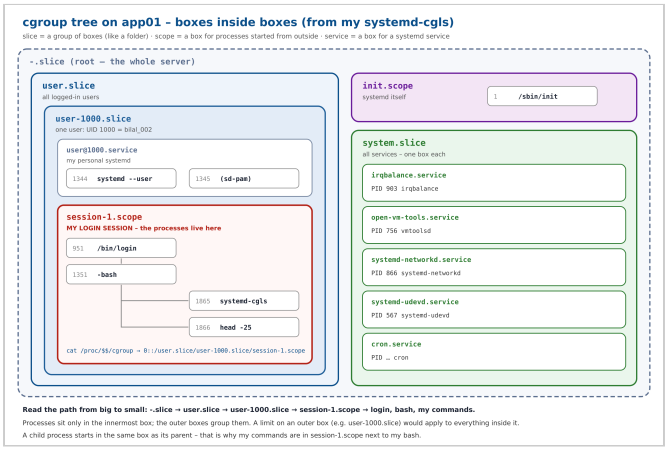
cache from RAM – nothing is lost, the originals are on the disk – and then moves idle program memory from RAM into **swap**, a reserved area on the disk (Ubuntu `/swap.img`, Windows `pagefile.sys`); normal files on the disk are never touched, the box only gets slower. If it still reaches **MemoryMax** (hard wall), the kernel's **OOM killer** stops a process in this box only. **TasksMax** caps the number of processes and protects the PID pool. **Docker** (`--memory`, `--cpus`, `mem_limit`) and **Kubernetes** (requests = guaranteed, limits = hard ceiling, `OOMKilled`) use the same cgroups: on Linux systemd creates the box; in Docker `dockerd → containerd → runc`; in Kubernetes `kubelet → containerd → runc`. containerd manages, runc builds the container with **namespaces** (isolation) and a **cgroup** (limits); the IP comes from Docker networking or a CNI plugin. Traffic to a container always reaches the **host's card** first, and a **NAT rule** forwards it – exactly like my gw01's `DNAT 8080 → app01:80`.

5. Lab Step 1 (app01) – the cgroup tree

```
free -h                 # RAM 1.9Gi · swap 1.8Gi
swapon --show           # /swap.img – the swap file on the disk
cat /proc/$$/cgroup     # 0::/user.slice/user-1000.slice/session-1.scope
systemd-cgls            # the whole tree of boxes
```

💡 **In plain words:** the server is one tree of boxes. Directly under the root, **side by side**, are `init.scope` (only PID 1 – systemd), `system.slice` (all services, one box each) and `user.slice` (all logged-in users). My shell lives in the innermost box: `user.slice → user-1000.slice → session-1.scope`.

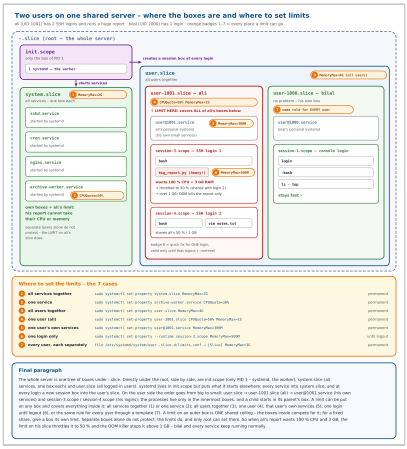
📸 **Diagram – my real tree on app01:**



cgroup tree on app01

6. Where a limit can be set – two users on one server

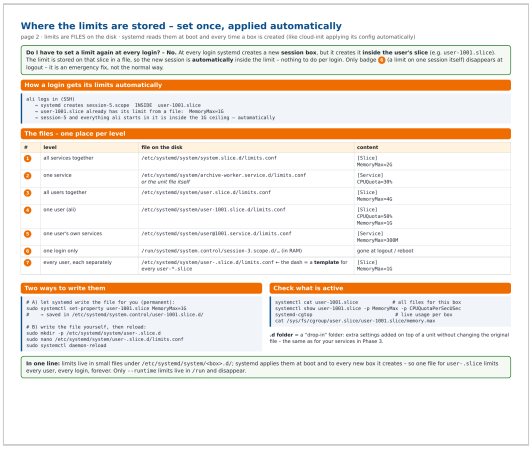
📸 **Diagram:**



Two users – where to set limits

7. Where the limits are stored – set once, applied automatically

📸 **Reference page:**



Where the limits are stored

In my own words: The whole server is one tree of boxes under `-.slice`: `init.scope`, `system.slice` and `user.slice` side by side. A limit can go on any box and covers everything inside it – all services, one service, all users, one user, a user's own services, one login, or every user through the `user-.slice` template. Limits are small files in `/etc/systemd/system/<box>.d/`; systemd applies them at boot and to every new box it creates, so I set them once and every login is covered automatically.

Day 11 – 3 Nov: memory limit + OOM killer, disk and RAM, mounts, live vs permanent

1. Lab Step 2 (app01) – a hard memory limit and the OOM killer

💡 **In plain words:** I gave a test program a box with a 100 MB memory wall and no swap, then asked it to use 300 MB. The kernel stopped it at the wall – inside its own box only. app01 and every service next to it kept running.

```
sudo systemd-run --scope -p MemoryMax=100M -p MemorySwapMax=0 stress-ng --vm 1 --vm-bytes 300M --timeout 1s
sudo journalctl -k --since "5 min ago" | grep -i -E "oom|killed" | tail -5
```

```
sudo systemd-run --scope  -p <LIMIT>   stress-ng <LOAD>  --timeout <TIME>
     └─ make a box ──┘  └ the limit ┘  └── fake load to test the limit ──┘
```

📸 **Evidence:**

My Computer | gw01 | app01 | stor01 | svc01

```
bilal_002@app01:~$ sudo systemd-run --scope -p MemoryMax=100M -p MemorySwapMax=0 stress-ng --vm 1 --
vm-bytes 300M --timeout 1s
Running scope as unit: run-ra966501bb27f4517a940f4c9533fc01a.scope
stress-ng: info:  [2687] setting to a 1 second run per stressor
stress-ng: info:  [2687] dispatching hogs: 1 vm
[18909.918912] Memory cgroup out of memory: Killed process 2689 (stress-ng) total-vm:349924kB, anon-
rss:99056kB, file-rss:792kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
[18910.113006] Memory cgroup out of memory: Killed process 2690 (stress-ng) total-vm:349924kB, anon-
rss:98792kB, file-rss:792kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
[18910.261201] Memory cgroup out of memory: Killed process 2691 (stress-ng) total-vm:349924kB, anon-
rss:98792kB, file-rss:792kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
[18910.402231] Memory cgroup out of memory: Killed process 2692 (stress-ng) total-vm:349924kB, anon-
rss:98792kB, file-rss:792kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
[18910.574483] Memory cgroup out of memory: Killed process 2693 (stress-ng) total-vm:349924kB, anon-
rss:98792kB, file-rss:792kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
[18910.712498] Memory cgroup out of memory: Killed process 2694 (stress-ng) total-vm:349924kB, anon-
rss:98792kB, file-rss:792kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
[18910.850120] Memory cgroup out of memory: Killed process 2695 (stress-ng) total-vm:349924kB, anon-
rss:98788kB, file-rss:856kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
stress-ng: info:  [2687] successful run completed in 1.10s
bilal_002@app01:~$ _
```

OOM test – stress-ng killed at 100 MB

My Computer | gw01 | app01 | stor01 | svc01

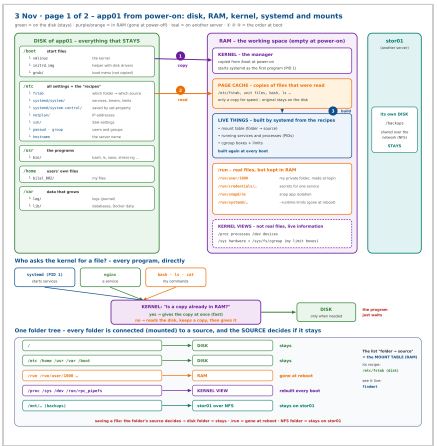
```
bilal_002@app01:~$ sudo journalctl -k --since "5 min ago" | grep -i -E "oom|killed" | tail -5
Nov 01 22:21:30 app01 kernel: stress-ng invoked oom-killer: gfp_mask=0xcc0(GFP_KERNEL), order=0, oom
_score_adj=1000
Nov 01 22:21:30 app01 kernel:  oom_kill_process.cold+0xb/0x10
Nov 01 22:21:30 app01 kernel: [  pid  ]   uid  tgid total_vm      rss pgtables_bytes swapents oom_sc
ore_adj name
Nov 01 22:21:30 app01 kernel: oom-kill:constraint=CONSTRAINT_MEMCG,nodemask=(null),cpuset=run-ra9665
01bb27f4517a940f4c9533fc01a.scope,mems_allowed=0,oom_memcg=/system.slice/run-ra966501bb27f4517a940f4
c9533fc01a.scope,task_memcg=/system.slice/run-ra966501bb27f4517a940f4c9533fc01a.scope,task=stress-ng
,pid=2695,uid=0
Nov 01 22:21:30 app01 kernel: Memory cgroup out of memory: Killed process 2695 (stress-ng) total-vm:
349924kB, anon-rss:98788kB, file-rss:856kB, shmem-rss:40kB, UID:0 pgtables:684kB oom_score_adj:1000
bilal_002@app01:~$
```

Journal proof – CONSTRAINT_MEMCG

2. How Linux uses the disk and RAM – from power-on

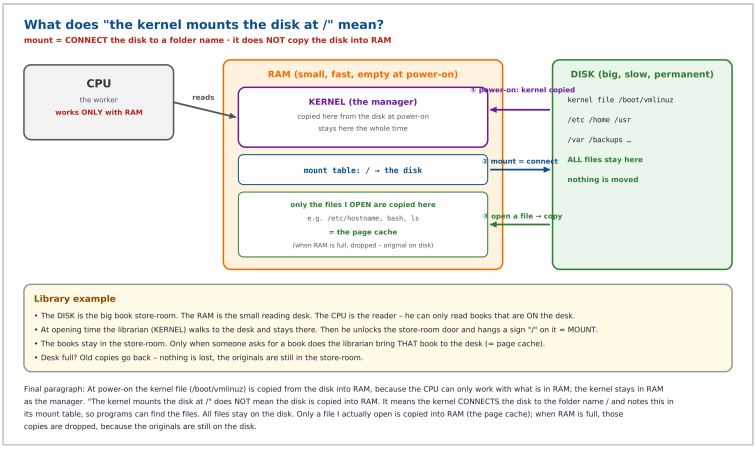
💡 **In plain words:** everything is stored on the disk; RAM starts empty. The kernel is copied into RAM and becomes the manager, systemd reads its settings from `/etc`, and builds the live things in RAM.

📸 **Diagram – the whole picture on one page:**



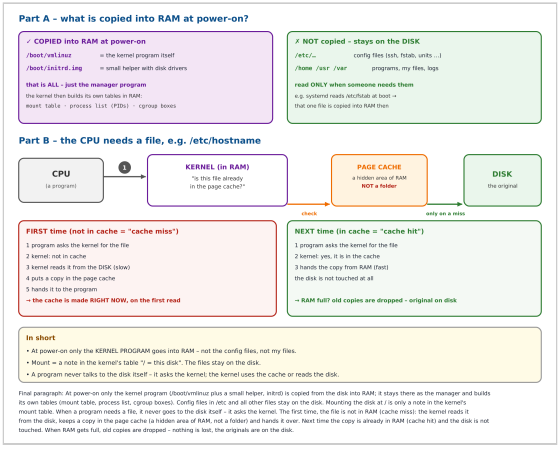
app01 from power-on – disk, RAM, kernel, systemd and mounts

2.1 Mount = connect, not copy



What mount means – the library example

2.2 When a program needs a file

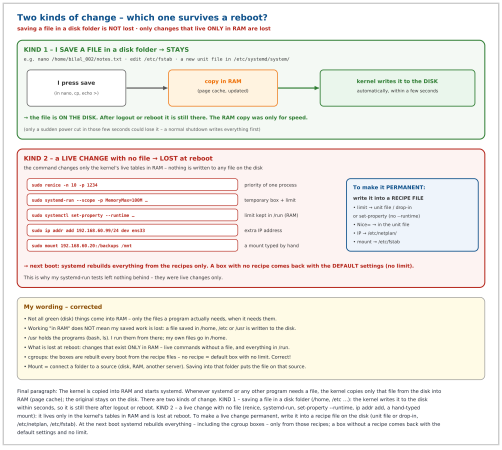


How a file is read – cache miss and cache hit

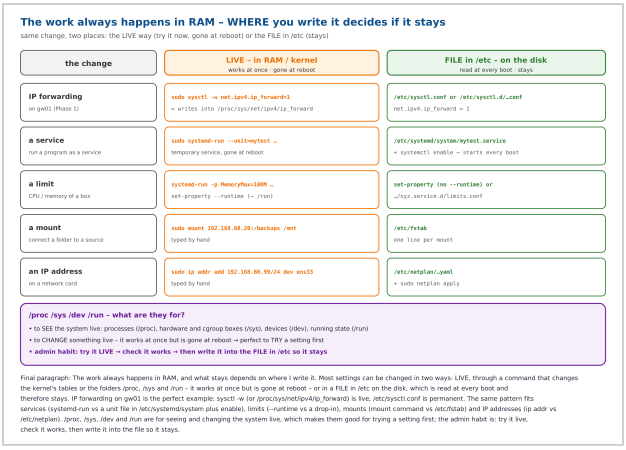
3. Live test vs permanent – where I write it decides

💡 **In plain words:** the work always happens in RAM. Saving a file in a disk folder is **not** lost – the kernel writes it to the disk within seconds. What is lost at reboot is a **live change with no file**. `/etc` is special because of its job: it holds the settings programs read every time they start.

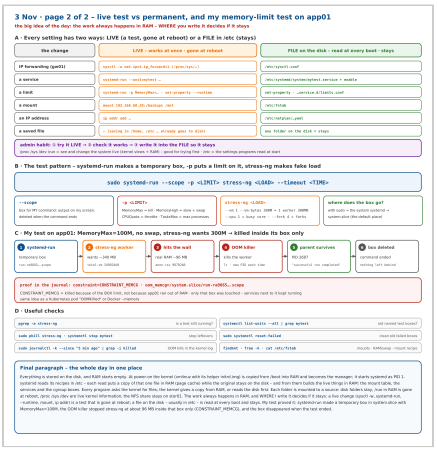
📸 **Diagrams:**



Two kinds of change



Live vs file



Page 2 of the overview – live test vs permanent and the OOM test

4. Key understanding (in my own words)

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (`vmlinuz` with its helper `initrd.img`) is copied from `/boot` into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in `/etc` – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, `/run` in RAM is gone at reboot, `/proc`, `/sys` and `/dev` are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and **where I write it decides if it stays**: a live change (`sysctl -w`, `systemd-run`, `--runtime`, `mount`, `ip addr`) is a test that is gone at reboot; a file on the disk – usually in `/etc` – is read at every boot and stays. My test proved it: `systemd-run` made a temporary box in system.slice with `MemoryMax=100M`, the OOM killer stopped stress-ng at about 96 MB inside that box only (`CONSTRAINT_MEMCG`), and the box disappeared when the test ended.